

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN**



KHÓA LUẬN TỐT NGHIỆP

ĐỀ TÀI:

**XÂY DỰNG HỆ THỐNG GIA SƯ LẬP TRÌNH CÁ
NHÂN HOÁ SỬ DỤNG MÔ HÌNH NGÔN NGỮ LỚN**

Giảng viên hướng dẫn: ThS. Nguyễn Thu Phương

Sinh viên thực hiện: Lưu Sỹ Trường

Lớp: Kỹ thuật phần mềm K21A

Mã sinh viên: DTC225180321

Thái Nguyên, năm 2026

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN**



KHÓA LUẬN TỐT NGHIỆP

ĐỀ TÀI:

**XÂY DỰNG HỆ THỐNG GIA SƯ LẬP TRÌNH CÁ
NHÂN HOÁ SỬ DỤNG MÔ HÌNH NGÔN NGỮ LỚN**

Giảng viên hướng dẫn: ThS. Nguyễn Thu Phương

Sinh viên thực hiện: Lưu Sỹ Trường

Lớp: Kỹ thuật phần mềm K21A

Mã sinh viên: DTC225180321

Thái Nguyên, năm 2026

ĐẠI HỌC THÁI NGUYÊN
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
THÔNG TIN VÀ TRUYỀN THÔNG

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

LỜI CAM ĐOAN

*(kèm theo Thông báo số: ... /TB-ĐH CNTT&TT ngày ... tháng năm 2026
của Hiệu trưởng Trường ĐH CNTT&TT)*

Em xin cam đoan đã thực hiện việc kiểm tra mức độ tương đồng nội dung (ĐA/KLTN) qua phần mềm Turnitin một cách trung thực và đạt kết quả mức độ tương đồng 10%. Bản đề tài (ĐA/KLTN) kiểm tra qua phần mềm là bản cứng đã nộp để bảo vệ trước hội đồng.

Nếu sai em xin hoàn toàn chịu trách nhiệm.

Xin chân thành cảm ơn./.

Thái Nguyên, ngày ... tháng ... năm 2026

TÁC GIẢ CỦA SẢN PHẨM

(Ký và ghi rõ họ tên)

Lưu Sỹ Trường

LỜI CẢM ƠN

Hoàn thành đồ án tốt nghiệp này là kết quả của một hành trình dài với sự đồng hành của nhiều người mà em vô cùng trân trọng.

Trước tiên, em muốn gửi lời cảm ơn đặc biệt đến cô Nguyễn Thu Phương, người thầy đã không quản ngại thời gian và công sức để chỉ dẫn em từng bước trong quá trình nghiên cứu và hoàn thiện đề tài. Những nhận xét sâu sắc cùng tinh thần trách nhiệm cao của cô đã giúp em nhìn nhận vấn đề rõ ràng hơn và tìm ra hướng đi phù hợp trong những lúc gặp khó khăn.

Em cũng xin bày tỏ sự tri ân đến đội ngũ giảng viên Khoa Công nghệ thông tin, trường Đại học Công nghệ thông tin và Truyền thông, những người đã truyền đạt không chỉ kiến thức chuyên ngành mà còn cách tư duy, cách tiếp cận vấn đề một cách có hệ thống. Nền tảng học thuật mà em tích lũy được trong suốt những năm ngồi trên ghế nhà trường chính là điểm tựa vững chắc để em tự tin bước vào thực tế nghề nghiệp.

Dù đã dành nhiều tâm huyết cho đề tài, em hiểu rằng với kinh nghiệm còn hạn chế của một sinh viên, đồ án này khó tránh khỏi những điểm chưa hoàn chỉnh. Em rất mong nhận được những góp ý thẳng thắn từ thầy cô và các bạn, đó là cơ hội quý giá để em tiếp tục trưởng thành.

Xin chân thành cảm ơn Thầy Cô!

Thái Nguyên, ngày ... tháng ... năm 2026

Sinh viên thực hiện

Lưu Sỹ Trường

MỤC LỤC

LỜI CAM ĐOAN	i
LỜI CẢM ƠN	ii
DANH SÁCH BẢNG	vi
DANH SÁCH HÌNH VẼ	vii
DANH MỤC TỪ VIẾT TẮT	ix
MỞ ĐẦU	1
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	2
1.1. Tổng quan về AI và Mô hình Ngôn ngữ Lớn (LLM)	2
1.1.1. Trí tuệ nhân tạo (Artificial Intelligence)	2
1.1.2. Mô hình ngôn ngữ lớn (Large Language Model)	2
1.1.3. GPT-4o	4
1.2. LLM trong giáo dục lập trình	4
1.2.1. Vai trò kép của LLM trong giáo dục	4
1.2.2. Cá nhân hóa học tập với LLM	5
1.2.3. Nguyên tắc thiết kế hệ thống PPAT từ cơ sở lý thuyết	5
1.3. Công nghệ Frontend	6
1.3.1. Next.js và React 19	6
1.3.2. TypeScript	6
1.3.3. TailwindCSS và hệ thống UI Component	7
1.3.4. Monaco Editor	7
1.3.5. Quản lý state và dữ liệu	7
1.3.6. Xử lý nội dung Markdown và Streaming	8
1.4. Công nghệ Backend	8
1.4.1. Fastify & TypeScript	8
1.4.2. Zod & Drizzle ORM	8
1.4.3. Code Runner Sandbox (Dockerode & tar-stream)	9
1.5. Cơ sở dữ liệu và Cache	9
1.5.1. PostgreSQL 15	9
1.5.2. Redis 7	10
1.6. Kiến trúc SOA và Docker	10
1.6.1. Service-Oriented Architecture (SOA)	10
1.6.2. Docker & Docker Compose	11

CHƯƠNG 2: KHẢO SÁT VÀ PHÂN TÍCH HỆ THỐNG	12
2.1. Khảo sát thực tế	12
2.1.1. Quy trình học lập trình hiện nay	12
2.1.2. Kết quả khảo sát	12
2.1.3. Sơ đồ BPMN quy trình học tập	18
2.1.4. Phân tích hệ thống hiện có	19
2.2. Xác định Actor	20
2.3. Yêu cầu chức năng	20
2.4. Yêu cầu phi chức năng	22
2.5. Mô hình nghiệp vụ	23
2.5.1. Biểu đồ Use Case	23
2.5.2. Đặc tả Use Case	26
CHƯƠNG 3: THIẾT KẾ HỆ THỐNG	40
3.1. Kiến trúc tổng thể	40
3.1.1. Tổng quan kiến trúc	40
3.1.2. Luồng xử lý theo loại request	41
3.2. Thiết kế API	42
3.3. Thiết kế cơ chế Cá nhân hóa AI	43
3.3.1. Pipeline xử lý tin nhắn (Message Lifecycle)	43
3.3.2. Dynamic System Prompt	44
3.3.3. Adaptive Learning Path	45
3.3.4. Streaming Architecture (SSE)	47
3.4. Thiết kế chi tiết	48
3.4.1. Đăng ký	48
3.4.2. Đăng nhập	51
3.4.3. Dashboard kết hợp Theo dõi tiến độ	54
3.4.4. Danh sách Bài học (Xem và Lọc bài học)	56
3.4.5. Chi tiết Bài học	57
3.4.6. Code Editor (Làm bài tập)	59
3.4.7. Chat với AI Tutor	60
3.5. Thiết kế Cơ sở dữ liệu	63
3.5.1. Sơ đồ lớp thực thể (ERD)	63
3.5.2. Mô tả chi tiết các bảng	64
CHƯƠNG 4: TRIỂN KHAI	66
4.1. Môi trường và Stack công nghệ	66
4.2. Cài đặt và triển khai Docker	68
4.2.1. Cấu trúc Docker Compose	68
4.2.2. Cấu trúc Dockerfile	68
4.2.3. Hướng dẫn cài đặt	69

4.3.	Bảo vệ hệ thống	70
4.3.1.	Xác thực JWT	70
4.3.2.	Mã hóa mật khẩu	70
4.3.3.	Cô lập môi trường chạy code	70
4.3.4.	Giới hạn tần suất request	71
4.4.	Tính năng AI Chatbot cá nhân hóa	71
4.4.1.	Ghi nhớ lịch sử và cá nhân hóa theo hồ sơ học tập	71
4.4.2.	Phân tích lỗi code và gợi ý debug theo phương pháp Socratic	72
4.4.3.	Điều chỉnh độ khó và lộ trình học theo performance	73
4.5.	Dữ liệu thử nghiệm	76
4.6.	Kiểm thử hệ thống	77
4.6.1.	Unit Test	77
4.6.2.	Integration Test	79
4.6.3.	Performance Test	80
4.6.4.	Tổng kết kiểm thử	82
4.7.	Tài liệu hướng dẫn sử dụng	82
4.7.1.	Hướng dẫn cài đặt và khởi động hệ thống	82
4.7.2.	Hướng dẫn dành cho sinh viên	82
4.7.3.	Hướng dẫn dành cho Admin	83
4.8.	Kết quả và Đánh giá	83
4.8.1.	Chức năng hoàn thành	83
4.8.2.	Ưu điểm	84
4.8.3.	Hạn chế	85
4.8.4.	Hướng phát triển	85
	KẾT LUẬN	86
	TÀI LIỆU THAM KHẢO	87
	PHỤ LỤC	89
	NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN	94

DANH SÁCH BẢNG

Bảng 2.1.	Danh sách yêu cầu chức năng	21
Bảng 2.2.	Bảng đặc tả chức năng Đăng ký (Sinh viên)	26
Bảng 2.3.	Bảng đặc tả chức năng Đăng nhập (Sinh viên, Admin)	27
Bảng 2.4.	Bảng đặc tả chức năng Quản lý thông tin cá nhân	28
Bảng 2.5.	Bảng đặc tả chức năng Quản lý người dùng	29
Bảng 2.6.	Bảng đặc tả chức năng Quản lý bài học	30
Bảng 2.7.	Bảng đặc tả chức năng Quản lý bài tập	32
Bảng 2.8.	Bảng đặc tả chức năng Quản lý test cases	33
Bảng 2.9.	Bảng đặc tả chức năng Xem & Lọc bài học	35
Bảng 2.10.	Bảng đặc tả chức năng Làm bài tập	35
Bảng 2.11.	Bảng đặc tả chức năng Theo dõi tiến độ	37
Bảng 2.12.	Bảng đặc tả chức năng Chat với AI Tutor (Sinh viên)	37
Bảng 2.13.	Bảng đặc tả chức năng Cập nhật Knowledge log	38
Bảng 3.1.	Ví dụ luồng xử lý theo loại request	41
Bảng 3.2.	Danh sách endpoint REST API	42
Bảng 3.3.	Các bảng thực thể trong hệ thống	65
Bảng 4.1.	Bảng tổng hợp công nghệ của thống	66
Bảng 4.2.	Cấu trúc file Docker Compose	68
Bảng 4.3.	Tổng hợp kết quả dữ liệu thử nghiệm	76
Bảng 4.4.	Phân bố bài học theo ngôn ngữ	77
Bảng 4.5.	Kết quả Unit Test	78
Bảng 4.6.	Kết quả Integration Test	79
Bảng 4.7.	Kết quả Performance Test (50 VUs, 30 giây)	80
Bảng 4.8.	Tổng hợp kết quả kiểm thử	82
Bảng 4.9.	Checklist thi công các Use Case theo yêu cầu PI	83

DANH SÁCH HÌNH VẼ

Hình 2.1.	Khảo sát chuyên ngành sinh viên	13
Hình 2.2.	Khảo sát kinh nghiệm	13
Hình 2.3.	Khảo sát ngôn ngữ học	14
Hình 2.4.	Khảo sát nguồn học	14
Hình 2.5.	Khảo sát thói quen sau lý thuyết	15
Hình 2.6.	Khảo sát khó khăn trong quá trình học	15
Hình 2.7.	Khảo sát hành vi khi gặp lỗi code	16
Hình 2.8.	Khảo sát mức độ sử dụng AI trong học tập	16
Hình 2.9.	Khảo sát kỳ vọng chức năng của AI Tutor	17
Hình 2.10.	Khảo sát sự sẵn sàng sử dụng	17
Hình 2.11.	Quy trình học lập trình hiện tại	18
Hình 2.12.	Quy trình học lập trình với hệ thống PPAT	18
Hình 2.13.	Biểu đồ UC tổng quát	24
Hình 2.14.	Biểu đồ phân rã cho tác nhân Admin	25
Hình 2.15.	Biểu đồ phân rã cho tác nhân Sinh viên	26
Hình 3.1.	Biểu đồ thành phần kiến trúc SOA	40
Hình 3.2.	Pipeline xử lý tin nhắn (Message Lifecycle)	44
Hình 3.3.	Adaptive Learning Path	46
Hình 3.4.	Giao diện chức năng đăng ký	48
Hình 3.5.	Sơ đồ hoạt động chức năng Đăng ký	49
Hình 3.6.	Sơ đồ trình tự chức năng Đăng ký	50
Hình 3.7.	Giao diện chức năng Đăng nhập	51
Hình 3.8.	Sơ đồ hoạt động chức năng Đăng nhập	52
Hình 3.9.	Sơ đồ trình tự chức năng Đăng nhập	53
Hình 3.10.	Giao diện Dashboard kết hợp theo dõi tiến độ	54
Hình 3.11.	Sơ đồ hoạt động chức năng xem Dashboard kết hợp theo dõi tiến độ	55
Hình 3.12.	Sơ đồ trình tự chức năng xem Dashboard kết hợp theo dõi tiến độ	55
Hình 3.13.	Giao diện Danh sách bài học	56
Hình 3.14.	Sơ đồ hoạt động chức năng xem Danh sách bài học	56
Hình 3.15.	Sơ đồ trình tự chức năng xem Danh sách bài học	57
Hình 3.16.	Giao diện Chi tiết bài học	57
Hình 3.17.	Sơ đồ hoạt động chức năng xem Chi tiết bài học	58
Hình 3.18.	Sơ đồ trình tự chức năng xem Chi tiết bài học	58
Hình 3.19.	Giao diện Code Editor (Làm bài tập)	59
Hình 3.20.	Sơ đồ hoạt động chức năng Code editor (Làm bài tập)	59
Hình 3.21.	Sơ đồ trình tự chức năng Code editor (Làm bài tập)	60

Hình 3.22.	Giao diện Chat với AI Tutor	60
Hình 3.23.	Giao diện Chat AI với Tutor trong trình Editor	61
Hình 3.24.	Sơ đồ hoạt động chức năng Chat AI với Tutor	61
Hình 3.25.	Sơ đồ trình tự chức năng Chat AI với Tutor	62
Hình 3.26.	Sơ đồ lớp thực thể hệ thống	63
Hình 4.1.	AI Tutor tóm tắt hồ sơ năng lực cá nhân của sinh viên	72
Hình 4.2.	AI Tutor phân tích lỗi và dẫn dắt sinh viên sửa từng bước	73
Hình 4.3.	Hệ thống gợi ý bài tiếp theo sau khi hoàn thành 4/4 bài tập	74
Hình 4.4.	AI dẫn dắt Socratic ở lần fail đầu tiên	74
Hình 4.5.	AI cung cấp hướng dẫn chi tiết và code mẫu ở lần fail thứ hai	75
Hình 4.6.	Hệ thống yêu cầu ôn lại bài học sau nhiều lần fail liên tiếp	76
Hình 4.7.	Kết quả chạy Unit Test trên terminal	78
Hình 4.8.	Kết quả chạy Integration Test trên terminal	80
Hình 4.9.	Kết quả chạy Performance Test với k6 trên terminal	81

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa đầy đủ
AI	Artificial Intelligence
LLM	Large Language Model
API	Application Programming Interface
JWT	JSON Web Token
SOA	Service-Oriented Architecture
SSE	Server-Sent Events
PPAT	Personalized Programming AI Tutor
UI	User Interface
UX	User Experience
ORM	Object Relational Mapping
CRUD	Create, Read, Update, Delete
GPT	Generative Pre-trained Transformer
SPA	Single Page Application
HMR	Hot Module Replacement
BPMN	Business Process Model and Notation
ERD	Entity Relationship Diagram
UC	Use Case
NFR	Non-Functional Requirement

MỞ ĐẦU

Trí tuệ nhân tạo đang thay đổi căn bản cách thức tiếp cận tri thức trong thế kỷ 21. Các mô hình ngôn ngữ lớn như GPT-4o không còn chỉ là công cụ tìm kiếm thông tin mà đã trở thành trợ lý có khả năng lý giải, phân tích và phản hồi theo từng ngữ cảnh cụ thể. Trong bối cảnh đó, ứng dụng AI vào giáo dục lập trình mở ra những khả năng mà mô hình dạy học truyền thống khó đáp ứng được.

Thực tế giảng dạy lập trình tại các trường đại học cho thấy một nghịch lý quen thuộc: lớp học có một giảng viên nhưng sinh viên thì học với tốc độ và xuất phát điểm hoàn toàn khác nhau. Người nắm vững kiến thức nền bước tiếp nhanh, trong khi người còn lỗ hổng khái niệm cơ bản dần bị bỏ lại phía sau. Ngoài giờ học chính thức, sinh viên chủ yếu tìm đến ChatGPT hay các công cụ tìm kiếm khi gặp vướng mắc. Những công cụ này cho câu trả lời tức thì nhưng hoàn toàn mù quáng trước bối cảnh của người hỏi, không biết sinh viên đang ở giai đoạn nào, đã nắm được những gì và thường xuyên vấp phải lỗi ở điểm nào.

Đề tài này xuất phát từ kỳ vọng xây dựng được một hệ thống có khả năng cải thiện những bất cập hiện nay. Thay vì hoạt động theo kiểu trả lời từng câu hỏi rời rạc, hệ thống PPAT ghi nhận toàn bộ quá trình học tập của mỗi sinh viên, từ trình độ ban đầu, các lỗi lặp lại đến tốc độ tiến bộ theo thời gian, và lấy đó làm cơ sở để điều chỉnh mọi phản hồi cho phù hợp. Thay vì đưa ra đáp án sẵn, hệ thống đặt câu hỏi gợi mở để sinh viên tự tư duy và hình thành kỹ năng giải quyết vấn đề một cách chủ động.

Về mặt kỹ thuật, hệ thống được xây dựng trên nền tảng Next.js kết hợp Monaco Editor cho phép viết và chạy code trực tiếp trên trình duyệt mà không cần cài đặt thêm công cụ. Backend sử dụng Fastify viết bằng TypeScript với PostgreSQL lưu trữ hồ sơ học tập và Redis xử lý cache. GPT-4o được tích hợp thông qua cơ chế system prompt sinh động, mỗi lần gọi API đều mang theo đầy đủ ngữ cảnh về người học. Toàn bộ hệ thống được đóng gói và vận hành qua Docker Compose.

Báo cáo được tổ chức thành 4 chương:

- Chương 1 trình bày nền tảng lý thuyết về AI, LLM và các công nghệ sử dụng.
- Chương 2 trình bày kết quả khảo sát thực tế, xác định yêu cầu chức năng và phi chức năng, đặc tả Use Case.
- Chương 3 trình bày thiết kế kiến trúc SOA, cơ sở dữ liệu, AI Orchestrator và giao diện.
- Chương 4 trình bày quá trình triển khai, kiểm thử và tài liệu hướng dẫn sử dụng.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về AI và Mô hình Ngôn ngữ Lớn (LLM)

1.1.1. Trí tuệ nhân tạo (*Artificial Intelligence*)

Trí tuệ nhân tạo (AI) là nhánh nghiên cứu trong khoa học máy tính nhằm tạo ra các hệ thống có thể thực hiện các tác vụ thường đòi hỏi nhận thức của con người từ nhận dạng ngôn ngữ, suy luận logic cho đến ra quyết định trong môi trường phức tạp. Trong thập kỷ gần đây, sự bùng nổ về dữ liệu và năng lực tính toán đã đưa AI từ phòng thí nghiệm vào ứng dụng thực tiễn ở hầu hết mọi lĩnh vực, từ y tế, tài chính đến giáo dục và kỹ thuật phần mềm.

Xét về phạm vi năng lực, AI hiện nay được phân thành hai nhóm. Nhóm thứ nhất thường gọi là Narrow AI hay AI chuyên biệt bao gồm các mô hình được tối ưu hóa cho một miền bài toán xác định, chẳng hạn phân loại văn bản, nhận dạng giọng nói hay sinh code. GPT-4o thuộc nhóm này: dù hiệu năng rất cao trong xử lý ngôn ngữ tự nhiên và lập luận, mô hình không có nhận thức tổng quát hay ý thức. Nhóm thứ hai Artificial General Intelligence (AGI) là dạng AI lý thuyết có thể linh hoạt giải quyết bất kỳ vấn đề nào tương tự con người. AGI chưa được hiện thực hóa và vẫn là chủ đề nghiên cứu mở trong cộng đồng khoa học.

Trong bối cảnh giáo dục, điều AI làm được mà các phương pháp truyền thống khó đạt là phục vụ từng học viên một cách riêng lẻ. Thay vì một giáo trình cố định áp dụng đồng loạt, AI có thể quan sát hành vi học tập của từng người chủ đề nào được nắm vững, lỗi nào lặp đi lặp lại, nhịp độ tiến bộ ra sao và điều chỉnh nội dung tương ứng. Grassucci và cộng sự (2025) phân tích vai trò kép của LLM trong học tập: một mặt đóng vai gia sư kiên nhẫn, dẫn dắt học viên qua từng bước của bài tập; mặt khác đóng vai cộng tác viên, giúp học viên tiếp cận các bài toán mở phức tạp hơn mức chương trình quy định.

Với lĩnh vực lập trình, AI có thể hỗ trợ ở nhiều tầng: giải thích khái niệm, chỉ ra lỗi trong code, gợi ý hướng debug và phân tích nguyên nhân sai lầm. Tuy nhiên, các công cụ phổ biến hiện nay như ChatGPT vận hành theo từng phiên độc lập không duy trì bộ nhớ về người dùng giữa các lần truy cập, không biết học viên đang ở bước nào trong lộ trình, và không điều chỉnh chiến lược giảng dạy theo trình độ thực tế. Đây là khoảng trống mà hệ thống PPAT trong đề án này hướng đến lấp đầy thông qua cơ chế knowledge log và dynamic system prompt.

1.1.2. Mô hình ngôn ngữ lớn (*Large Language Model*)

Mô hình ngôn ngữ lớn (LLM) là lớp mô hình AI được huấn luyện trên tập văn bản quy mô cực lớn với mục tiêu học cách hiểu và sinh ngôn ngữ tự nhiên. Tuy các nghiên cứu về xử lý ngôn ngữ tự nhiên đã có từ thập niên 1950, bước ngoặt kỹ thuật thực sự xảy ra năm 2017 khi Vaswani và cộng sự giới thiệu kiến trúc Transformer nền tảng được hầu

hết LLM hiện đại kế thừa.

Điểm đột phá của Transformer nằm ở cơ chế self-attention: thay vì đọc văn bản tuần tự từng từ như các kiến trúc RNN hay LSTM trước đó, mô hình có thể tính toán song song mức độ liên quan giữa tất cả các từ trong một chuỗi. Nhờ vậy, các phụ thuộc ngữ nghĩa tầm xa ví dụ đại từ ở cuối câu tham chiếu đến danh từ ở đầu đoạn được nắm bắt hiệu quả hơn nhiều, đồng thời tốc độ huấn luyện cũng cải thiện đáng kể so với các phương pháp trước.

Thuật ngữ "large" phản ánh quy mô tham số ngày càng tăng của các mô hình thế hệ mới. BERT (Google, 2019) có khoảng 110 triệu tham số; PaLM 2 (2023) đạt tới 340 tỷ. Các mô hình này được huấn luyện theo kiểu unsupervised học trực tiếp từ văn bản thô mà không cần nhãn thủ công thông qua mục tiêu dự đoán token tiếp theo trong chuỗi.

Quá trình sinh văn bản của LLM dựa trên autoregressive generation: mỗi token được chọn dựa trên phân phối xác suất tính từ toàn bộ ngữ cảnh phía trước. Đặc điểm này lý giải tại sao cùng một câu hỏi có thể nhận được các câu trả lời khác nhau qua từng lần chạy, và cũng là nguồn gốc của hiện tượng hallucination khi mô hình sinh ra nội dung có vẻ hợp lý về mặt hình thức nhưng không chính xác về nội dung thực tế.

Hallucination là hạn chế kỹ thuật cần đặc biệt lưu ý khi triển khai LLM trong môi trường giáo dục. Khác với tra cứu dữ kiện từ cơ sở dữ liệu, mô hình không "nhớ" thông tin theo nghĩa tường minh mà suy diễn câu trả lời từ các quy luật thống kê học được. Do đó, trong hệ thống gia sư lập trình, sinh viên cần được hướng dẫn luôn kiểm thử code do AI đề xuất thay vì chấp nhận mà không xác minh.

Dưới đây là một số khái niệm kỹ thuật thường gặp khi làm việc với LLM:

- Token là đơn vị cơ bản mà mô hình dùng để xử lý ngôn ngữ. Một token có thể là một từ, một phần của từ hoặc một ký tự đặc biệt. Chi phí sử dụng API của các LLM thường được tính theo số lượng token đầu vào và đầu ra.
- Context window là giới hạn số token mà mô hình có thể ghi nhớ trong một lần xử lý. Context càng lớn, mô hình càng có thể hiểu được những đoạn hội thoại dài hoặc tài liệu phức tạp. Trong dự án này, GPT-4o được sử dụng với context window lên tới 128.000 token, đủ để chứa toàn bộ knowledge log, lịch sử hội thoại và dữ liệu bài học của một sinh viên trong một request duy nhất.
- Prompt Engineering là kỹ thuật thiết kế câu lệnh đầu vào để hướng mô hình tạo ra phản hồi chính xác và phù hợp với mục tiêu. Một prompt có thể bao gồm system prompt, user prompt, ví dụ minh họa và dữ liệu ngữ cảnh. Trong hệ thống PPAT, backend xây dựng dynamic system prompt tức câu lệnh hệ thống được tạo tự động dựa trên knowledge log của từng sinh viên, bao gồm trình độ hiện tại, chủ đề đã học, lỗi thường gặp và tốc độ tiến bộ trước khi gửi đến GPT-4o.
- Temperature là tham số điều chỉnh mức độ ngẫu nhiên trong phản hồi của mô hình.

Giá trị thấp (0.1-0.3) cho phản hồi ổn định và nhất quán, phù hợp khi cần hướng dẫn lập trình chính xác. Giá trị cao (0.7-1.0) cho phản hồi đa dạng và sáng tạo hơn. Trong hệ thống AI Tutor, temperature được đặt ở mức thấp đến trung bình để đảm bảo chất lượng hướng dẫn.

1.1.3. GPT-4o

GPT-4o là phiên bản được đánh giá rất cao trong dòng mô hình GPT của OpenAI, ra mắt tháng 13/5/2024. So với các phiên bản trước, GPT-4o được thiết kế để xử lý hiệu quả hơn các tác vụ đòi hỏi lập luận nhiều bước, hiểu ngữ cảnh dài và sinh code có chất lượng cao những đặc tính trực tiếp liên quan đến bài toán gia sư lập trình.

GPT-4o có một số điểm mạnh nổi bật so với các mô hình thế hệ trước. Về khả năng xử lý code, mô hình có thể giải thích thuật toán, phân tích lỗi cú pháp và logic, gợi ý cách sửa và giải thích từng bước giải quyết bài toán những tính năng phù hợp trực tiếp với yêu cầu của một hệ thống gia sư lập trình. Về context window, GPT-4o hỗ trợ tới 128.000 token, cho phép hệ thống duy trì toàn bộ lịch sử học tập của sinh viên trong suốt phiên làm việc mà không bị mất ngữ cảnh. Ngoài ra, GPT-4o hỗ trợ streaming API, tức trả về từng token theo thời gian thực thay vì chờ sinh xong toàn bộ câu trả lời giúp giao diện chat phản hồi mượt mà và tạo trải nghiệm tương tác tự nhiên hơn cho sinh viên.

1.2. LLM trong giáo dục lập trình

1.2.1. Vai trò kép của LLM trong giáo dục

Grassucci và cộng sự (2025) phân tích LLM trong môi trường học tập qua hai góc nhìn bổ sung cho nhau, không thay thế nhau.

Góc nhìn thứ nhất đặt LLM ở vị trí gia sư kiên nhẫn. Khác với giáo viên trong lớp đông phải điều tiết theo tốc độ trung bình, một LLM có thể phục vụ riêng từng người học vào bất kỳ thời điểm nào giải thích lại khái niệm theo cách khác khi sinh viên chưa hiểu, thu gọn giải thích khi sinh viên đã nắm, và chỉ ra đúng điểm sai thay vì đưa ra nhận xét chung chung. Khả năng điều chỉnh tức thì trong quá trình hội thoại là thứ sách giáo khoa tĩnh không thể làm được.

Góc nhìn thứ hai đặt LLM ở vị trí đối tác khám phá. Thay vì giới hạn trong các bài tập có sẵn, LLM có thể dẫn dắt sinh viên tiếp cận bài toán mở, thử nghiệm nhiều hướng tiếp cận và tự đánh giá kết quả. Đây là kỹ năng quan trọng trong thực tế nghề nghiệp nơi vấn đề không bao giờ được đóng gói sẵn kèm hướng dẫn giải.

Điều cả hai vai trò đều yêu cầu, theo các tác giả, là LLM không được đóng vai người cung cấp đáp án. Giá trị thực nằm ở việc hỗ trợ sinh viên tự xây dựng chiến lược giải quyết vấn đề chuyển người học từ trạng thái thụ động sang chủ động. Đây là nền tảng lý thuyết trực tiếp cho phương pháp Socratic được triển khai trong PPAT: AI đặt câu hỏi dẫn dắt trước, chỉ cung cấp gợi ý nhỏ khi sinh viên thực sự bí sau nhiều lần thử.

1.2.2. Cá nhân hóa học tập với LLM

Một lớp học điển hình tập hợp sinh viên ở nhiều trình độ khác nhau, trong khi giáo viên chỉ có thể giảng theo một tốc độ duy nhất. Kết quả là sinh viên yếu bị bỏ lại phía sau trước khi lỗ hổng được phát hiện, còn sinh viên giỏi thiếu thách thức để tiến xa hơn. LLM mở ra khả năng can thiệp cá nhân mà quy mô lớp học truyền thống không cho phép.

Hình thức cá nhân hóa cơ bản nhất là điều chỉnh theo năng lực hiện tại: nếu sinh viên chưa hiểu, mô hình giải thích lại từ góc độ khác; nếu đã nắm vững, mô hình rút gọn và đẩy sang nội dung tiếp theo; nếu ở trình độ cao, mô hình mở rộng sang các khía cạnh nâng cao hơn. Sự điều chỉnh này xảy ra ngay trong luồng hội thoại, không cần cài đặt thủ công.

Mức cá nhân hóa sâu hơn đòi hỏi tích hợp hồ sơ học tập cá nhân bao gồm lịch sử làm bài, các lỗi lặp đi lặp lại và tốc độ tiến bộ để mô hình có ngữ cảnh đủ phong phú khi xây dựng phản hồi. Ví dụ cụ thể: một sinh viên hay quên điều kiện dừng trong đệ quy sẽ được AI chủ động nhắc về điểm này ngay khi bắt đầu bài liên quan, thay vì chờ sinh viên mắc lỗi rồi mới phản ứng. Đây chính xác là cơ chế knowledge log được xây dựng trong hệ thống PPAT.

Một hướng nghiên cứu xa hơn mà Grassucci và cộng sự (2025) đề xuất là tích hợp tín hiệu sinh lý nhịp tim, biểu cảm khuôn mặt để nhận diện trạng thái cảm xúc và điều chỉnh cách dạy khi phát hiện người học đang căng thẳng hoặc mất tập trung. Hướng này hiện còn trong giai đoạn nghiên cứu và nằm ngoài phạm vi của đề án.

1.2.3. Nguyên tắc thiết kế hệ thống PPAT từ cơ sở lý thuyết

Từ nền tảng lý thuyết trình bày ở trên, PPAT được thiết kế phát triển theo bốn nguyên tắc cốt lõi.

Thứ nhất, mỗi khi sinh viên gửi câu hỏi, backend không gửi ngay đến GPT-4o mà trước tiên truy vấn knowledge log của sinh viên đó từ cơ sở dữ liệu, sau đó xây dựng một system prompt động chứa đầy đủ thông tin về trình độ, chủ đề đã học và lỗi thường gặp. Nhờ vậy, cùng một câu hỏi sẽ nhận được phản hồi hoàn toàn khác nhau tùy vào profile của từng sinh viên.

Thứ hai, AI áp dụng phương pháp Socratic đặt câu hỏi gợi mở thay vì đưa ra đáp án để buộc sinh viên chủ động suy nghĩ. Nếu sinh viên đã thử hai lần mà vẫn chưa ra, AI mới cung cấp gợi ý nhỏ, không phải lời giải hoàn chỉnh.

Thứ ba, khi phát hiện sinh viên chưa có kiến thức nền cần thiết cho chủ đề đang hỏi, AI sẽ nhẹ nhàng chuyển hướng về phần nền tảng trước, thay vì cố gắng giải thích một khái niệm mà sinh viên chưa đủ điều kiện tiếp thu.

Thứ tư, sau mỗi phiên tương tác, hệ thống tự động cập nhật knowledge log dựa trên hành vi của sinh viên bài đã hoàn thành, lỗi đã gặp, câu hỏi đã đặt giúp system prompt

ngày càng chính xác và phù hợp hơn theo thời gian.

1.3. Công nghệ Frontend

1.3.1. *Next.js và React 19*

Next.js là framework phát triển web do Vercel duy trì, xây dựng trên nền React và tập trung giải quyết các vấn đề mà React thuần không xử lý sẵn: server-side rendering, routing theo cấu trúc thư mục, tối ưu hóa hình ảnh và phân tách code tự động. Đây là một trong những lựa chọn phổ biến nhất cho ứng dụng web fullstack hiện đại.

PPAT sử dụng Next.js 15 kết hợp React 19 hai phiên bản mới nhất tại thời điểm thực hiện đồ án. React 19 bổ sung Server Components và Actions, cho phép một phần logic chạy trực tiếp trên server mà không cần round-trip API riêng, giúp giảm độ phức tạp của frontend đồng thời cải thiện thời gian tải trang đầu tiên.

Điểm khác biệt lớn nhất so với React thuần là Next.js tích hợp sẵn nhiều tính năng mà React thuần không có. Hệ thống routing dựa trên cấu trúc thư mục theo App Router giúp tổ chức code rõ ràng mà không cần cài thêm thư viện routing riêng. Quan trọng hơn, Next.js hỗ trợ Server Actions cơ chế cho phép gọi logic phía server trực tiếp từ component React mà không cần định nghĩa một API endpoint riêng. Điều này giúp đơn giản hóa luồng dữ liệu cho các tác vụ như submit form, cập nhật tiến độ học tập hay lưu lịch sử hội thoại, vì toàn bộ được xử lý trong cùng một codebase thay vì phải tách ra thành REST API độc lập.

Dự án cũng bật Turbopack trình bundler thế hệ mới được tích hợp vào Next.js để thay thế Webpack trong môi trường phát triển. Turbopack sử dụng kiến trúc xử lý song song, giúp tốc độ khởi động và hot reload nhanh hơn đáng kể so với Webpack, đặc biệt khi dự án có nhiều thư viện UI component.

Lý do chọn Next.js thay vì React thuần: ngoài việc tích hợp sẵn Server Actions và routing, Next.js còn hỗ trợ tốt cả Server Components lẫn Client Components trong cùng một ứng dụng, giúp tối ưu hóa hiệu năng tải trang mà không cần cấu hình thêm.

1.3.2. *TypeScript*

Toàn bộ frontend được viết bằng TypeScript 5, ngôn ngữ mở rộng của JavaScript với hệ thống kiểu tĩnh. TypeScript giúp phát hiện lỗi ngay tại thời điểm viết code thay vì chỉ khi chạy, đặc biệt hữu ích trong dự án có nhiều interface phức tạp như knowledge log, user analytics hay các kiểu dữ liệu trao đổi giữa frontend và backend. Dự án cấu hình tsc –noEmit như một bước kiểm tra kiểu riêng biệt trong CI, đảm bảo không có lỗi kiểu nào lọt vào bản build.

Việc dùng TypeScript đồng nhất ở cả frontend và backend còn cho phép chia sẻ kiểu dữ liệu qua package @workspace/types một package nội bộ trong monorepo chứa các interface dùng chung. Nhờ đó, khi định nghĩa kiểu dữ liệu của một Message hay UserAnalytics, cả hai phía đều tham chiếu đến cùng một nguồn sự thật, tránh được tình

trạng mismatch giữa request và response.

1.3.3. TailwindCSS và hệ thống UI Component

Về styling, dự án sử dụng TailwindCSS 4 theo hướng tiếp cận utility-first thay vì viết file CSS riêng, các lớp tiện ích được áp dụng trực tiếp vào JSX. TailwindCSS 4 chuyển sang cấu hình hoàn toàn bằng CSS thay vì file tailwind.config.js như các phiên bản trước, tích hợp qua PostCSS plugin @tailwindcss/postcss. Dự án cũng cài thêm @tailwindcss/typography để render nội dung Markdown từ bài học và tin nhắn AI với typography chuẩn mà không cần tự định nghĩa style.

Hệ thống UI component được xây dựng trên Radix UI kết hợp với shadcn/ui. Radix UI cung cấp các primitive component đã xử lý sẵn accessibility (ARIA), keyboard navigation và focus management những yếu tố quan trọng nhưng tốn công tự làm nếu xây từ đầu. shadcn/ui đóng vai trò là lớp styling trên Radix UI, cung cấp các component hoàn chỉnh như Dialog, Dropdown, Toast, Tabs, Select, Accordion mà có thể tùy chỉnh tự do vì code được copy thẳng vào dự án thay vì import từ thư viện đóng gói. Bên cạnh đó, Base UI cũng được sử dụng cho một số component cần kiểm soát hành vi chi tiết hơn.

Các thư viện hỗ trợ UI đáng chú ý khác bao gồm Lucide React cung cấp bộ icon nhất quán, Sonner cho toast notification dạng stack, Vault cho drawer component, next-themes xử lý dark/light mode, và react-resizable-panels cho giao diện split-view trong màn hình code editor nơi đề bài và editor hiển thị song song.

1.3.4. Monaco Editor

Monaco Editor là engine trình soạn thảo code được dùng trong Visual Studio Code, được tích hợp vào dự án qua package @monaco-editor/react. Monaco mang lại trải nghiệm code editor chuyên nghiệp ngay trên trình duyệt với đầy đủ các tính năng như syntax highlighting cho nhiều ngôn ngữ (Python, JavaScript, C++, Java), tự động hoàn thành, đánh dấu lỗi cú pháp theo thời gian thực và minimap. Trong màn hình làm bài tập, Monaco được đặt trong layout split-view cùng với phần đề bài, cho phép sinh viên đọc yêu cầu và viết code cùng lúc mà không phải cuộn trang.

1.3.5. Quản lý state và dữ liệu

Dự án sử dụng hai lớp quản lý state với mục đích khác nhau. TanStack Query (React Query) v5 xử lý toàn bộ server state tức dữ liệu được fetch từ API như danh sách bài học, tiến độ học tập, lịch sử hội thoại. TanStack Query cung cấp cơ chế caching, refetching tự động và đồng bộ hóa dữ liệu, giúp tránh việc fetch lại không cần thiết và giữ UI luôn phản ánh trạng thái mới nhất từ server. Zustand v5 đảm nhận client state tức các trạng thái cục bộ trên giao diện như trạng thái sidebar, theme, hay context của phiên làm bài hiện tại.

Để xử lý form, dự án dùng React Hook Form v7 kết hợp với Zod v4 cho validation. React Hook Form quản lý trạng thái và validation của form theo hướng uncontrolled, giúp giảm số lần re-render. Zod định nghĩa schema validation bằng TypeScript với type inference tự động cùng một schema vừa được dùng để validate dữ liệu form ở frontend,

và sử dụng trên server để kiểm tra dữ liệu và ép kiểu đầu vào trước khi thực hiện thao tác dữ liệu.

TanStack Table v8 được dùng cho các màn hình quản lý dữ liệu dạng bảng ở phía admin, hỗ trợ sẵn sorting, filtering và pagination ở phía client.

1.3.6. Xử lý nội dung Markdown và Streaming

Vì nội dung bài học và phản hồi từ AI Tutor đều được lưu và truyền dưới dạng Markdown, dự án dùng react-markdown kết hợp với plugin remark-gfm để render các phần tử như bảng, danh sách, checkbox và fenced code block đúng theo chuẩn GitHub Flavored Markdown. Các đoạn code trong phản hồi AI được highlight cú pháp bằng react-syntax-highlighter, giúp sinh viên dễ đọc code mẫu hơn khi AI đưa ra ví dụ minh họa.

Để hiển thị biểu đồ trong trang theo dõi tiến độ bao gồm line chart tiến độ theo thời gian, radar chart kỹ năng và heatmap hoạt động dự án sử dụng Recharts, thư viện charting xây dựng trên D3 kết hợp API React-native.

1.4. Công nghệ Backend

1.4.1. Fastify & TypeScript

Fastify là một web framework cho Node.js được thiết kế với triết lý ưu tiên hiệu năng và tính mở rộng. So với Express framework phổ biến nhất trong hệ sinh thái Node.js Fastify xử lý request nhanh hơn xấp xỉ hai lần nhờ cơ chế serialization JSON tối ưu và kiến trúc plugin được thiết kế lại từ đầu. Mỗi tính năng trong Fastify đều được đóng gói dưới dạng plugin độc lập, cho phép hệ thống chỉ tải những thành phần thực sự cần thiết thay vì khởi tạo toàn bộ middleware như cách truyền thống.

Điểm đắt giá của Fastify là khả năng kết hợp schema validation ngay trong khai báo route thông qua JSON Schema hoặc thư viện bên ngoài như Zod. Khi kết hợp với TypeScript, toàn bộ luồng dữ liệu từ request đầu vào đến response đầu ra đều được kiểm tra kiểu tĩnh tại thời điểm biên dịch, loại bỏ một lớp lỗi runtime phổ biến trong các dự án JavaScript thuần. Việc đồng nhất TypeScript trên cả frontend lẫn backend còn mang lại lợi ích quan trọng: các kiểu dữ liệu dùng chung như entity, request body, response schema có thể được định nghĩa một lần trong package chung và chia sẻ giữa hai đầu ứng dụng mà không cần viết lại.

Trong hệ thống PPAT, Fastify đảm nhận vai trò API Gateway tiếp nhận toàn bộ request từ React frontend, xử lý xác thực JWT, điều phối logic nghiệp vụ và gọi xuống các tầng dữ liệu hoặc dịch vụ AI tương ứng.

1.4.2. Zod & Drizzle ORM

Zod là thư viện validation schema cho TypeScript với đặc điểm quan trọng là tự động suy diễn kiểu TypeScript từ định nghĩa schema, loại bỏ hoàn toàn sự trùng lặp giữa khai báo kiểu và logic kiểm tra dữ liệu. Thay vì viết interface TypeScript riêng rồi viết

thêm hàm validate riêng, lập trình viên chỉ cần định nghĩa một schema Zod duy nhất và từ đó suy ra cả kiểu lẫn validation. Trong PPAT, Zod được dùng để kiểm tra toàn bộ request body trước khi đưa vào xử lý từ payload đăng ký tài khoản, nội dung tin nhắn chat, đến code submission của sinh viên đảm bảo dữ liệu đầu vào luôn hợp lệ trước khi chạm vào logic nghiệp vụ.

Drizzle ORM là một Object Relational Mapper thế hệ mới cho TypeScript, theo hướng "type-safe by default". Khác với các ORM truyền thống như Sequelize hay TypeORM vốn sinh ra query dựa trên decorator và reflection, Drizzle định nghĩa schema database trực tiếp bằng TypeScript và tự động suy diễn kiểu cho kết quả truy vấn. Điều này có nghĩa là khi thực hiện JOIN giữa bảng users và user_analytics, TypeScript biết chính xác kết quả trả về có những trường nào mà không cần cast thủ công. Drizzle cũng hỗ trợ migration tự động theo dõi thay đổi của schema và tạo ra SQL migration script tương ứng giúp quá trình phát triển và triển khai trở nên nhất quán hơn.

1.4.3. Code Runner Sandbox (Dockerode & tar-stream)

Một trong những yêu cầu kỹ thuật đặc thù của hệ thống gia sư lập trình là khả năng thực thi code do người dùng gửi lên một cách an toàn. Code tùy ý từ người dùng có thể chứa vòng lặp vô tận, truy cập hệ thống file, hoặc gọi network tất cả đều là mối nguy nếu chạy trực tiếp trên server. Giải pháp được áp dụng trong PPAT là cô lập mỗi lần thực thi trong một Docker container tạm thời với tài nguyên bị giới hạn nghiêm ngặt: không có quyền truy cập mạng, không có quyền ghi vào filesystem, và tự động bị huỷ sau tối đa mười giây.

Dockerode là thư viện Node.js cho phép giao tiếp với Docker Engine API từ bên trong ứng dụng backend mà không cần gọi CLI. Thông qua Dockerode, Fastify API có thể tạo container mới, stream stdout và stderr về server, rồi xoá container ngay sau khi thực thi xong tất cả trong một request/response cycle. tar-stream đóng vai trò hỗ trợ: vì Docker yêu cầu file được đưa vào container dưới định dạng tar archive, tar-stream được dùng để đóng gói source code của sinh viên thành buffer trước khi copy vào container, tránh việc phải ghi tạm ra đĩa.

Cách tiếp cận này cho phép Code Runner nằm hoàn toàn bên trong Fastify service thay vì phải tách thành một microservice độc lập, giữ kiến trúc gọn gàng ở giai đoạn hiện tại mà vẫn đảm bảo mức độ cô lập cần thiết cho môi trường thực thi không tin cậy.

1.5. Cơ sở dữ liệu và Cache

1.5.1. PostgreSQL 15

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở với lịch sử phát triển hơn ba mươi năm, nổi tiếng với sự tuân thủ nghiêm ngặt chuẩn SQL và khả năng mở rộng linh hoạt. Trong PPAT, PostgreSQL được chọn vì hai lý do kỹ thuật chính. Thứ nhất, đây là yêu cầu công nghệ của đề án. Thứ hai, PostgreSQL hỗ trợ kiểu dữ liệu JSONB lưu trữ JSON nhị phân có thể index và truy vấn rất phù hợp để lưu knowledge log của từng sinh

viên. Các trường như `known_topics`, `error_patterns`, hay `confidence_scores` có cấu trúc thay đổi theo từng người dùng và theo thời gian; nếu chuẩn hóa hoàn toàn thành bảng quan hệ sẽ phức tạp không cần thiết, trong khi JSONB cho phép lưu linh hoạt và vẫn truy vấn được bằng SQL thông thường.

Tính nhất quán ACID của PostgreSQL đảm bảo rằng khi một sinh viên nộp bài tập và hệ thống cùng lúc cập nhật `user_progress` lẫn `user_analytics`, hai thao tác này luôn thành công hoặc cùng thất bại không có trạng thái nửa vời khiến dữ liệu học tập bị sai lệch.

1.5.2. *Redis 7*

Redis là cơ sở dữ liệu in-memory chạy trên server, hỗ trợ đa dạng cấu trúc dữ liệu, hoạt động chủ yếu trong RAM nên tốc độ đọc/ghi nhanh hơn vài bậc so với cơ sở dữ liệu truyền thống. Trong PPAT, Redis đảm nhận ba vai trò riêng biệt.

Vai trò thứ nhất là quản lý session và refresh token: sau khi người dùng đăng nhập, refresh token được lưu vào Redis với TTL bảy ngày thay vì lưu vào PostgreSQL, giúp quá trình kiểm tra và vô hiệu hóa token diễn ra cực nhanh mà không tạo thêm tải cho database chính.

Vai trò thứ hai là cache danh sách bài học theo mô hình cache-aside: khi có request `GET /api/lessons`, hệ thống kiểm tra Redis trước; nếu có cache thì trả về ngay (TTL một giờ), nếu miss mới truy vấn PostgreSQL và lưu kết quả vào Redis cho lần tiếp theo. Dữ liệu bài học ít thay đổi nhưng được đọc rất nhiều, nên cache-aside ở đây mang lại hiệu quả rõ rệt.

Vai trò thứ ba là rate limiting: Redis được dùng để đếm số lượng request từ mỗi IP trong khoảng thời gian nhất định, kết hợp với plugin `@fastify/rate-limit` để bảo vệ các endpoint nhạy cảm như đăng nhập hay gọi AI. Redis bật chế độ AOF (Append Only File) để đảm bảo dữ liệu không mất khi container khởi động lại.

1.6. Kiến trúc SOA và Docker

1.6.1. *Service-Oriented Architecture (SOA)*

SOA là phong cách kiến trúc phần mềm tổ chức hệ thống thành các đơn vị dịch vụ độc lập, mỗi đơn vị phụ trách một tập hợp chức năng xác định và trao đổi với nhau qua giao diện API được định nghĩa trước. Khác với kiến trúc monolithic nơi toàn bộ logic nghiệp vụ gộp chung trong một codebase SOA cho phép phát triển, kiểm thử và triển khai từng service riêng biệt mà không kéo theo rủi ro cho phần còn lại.

Trong PPAT, kiến trúc SOA được hiện thực hóa qua bốn tầng tách biệt về trách nhiệm. Frontend React đảm nhận toàn bộ phần giao diện và tương tác người dùng. Backend Fastify API tiếp nhận request, thực thi logic nghiệp vụ và điều phối luồng xử lý. AI Orchestrator chịu trách nhiệm xây dựng dynamic system prompt từ knowledge log rồi chuyển tiếp đến OpenAI, đồng thời pipe stream response về phía client. Data Layer gồm

PostgreSQL lưu trữ dữ liệu bền vững và Redis xử lý cache cùng session.

Ranh giới rõ ràng giữa các tầng tạo ra lợi thế bảo trì thực tế: Code Runner có thể được tách thành service độc lập khi cần mở rộng sang nhiều ngôn ngữ, hoặc Analytics Service có thể được bổ sung sau mà không cần chỉnh sửa code của các service hiện có.

1.6.2. Docker & Docker Compose

Docker là nền tảng container hóa cho phép đóng gói ứng dụng cùng toàn bộ dependencies vào một đơn vị có thể chạy nhất quán trên bất kỳ môi trường nào từ máy phát triển của lập trình viên đến server production mà không gặp vấn đề "chạy được trên máy tôi". Mỗi container chia sẻ kernel của hệ điều hành host nhưng được cô lập về filesystem, process và network, nhẹ hơn đáng kể so với máy ảo truyền thống.

Docker Compose là công cụ định nghĩa và quản lý nhiều container cùng lúc thông qua một file cấu hình YAML duy nhất. Trong PPAT, Docker Compose khởi động bốn service song song: api chứa Fastify backend, web chứa React frontend được serve qua Nginx, db chứa PostgreSQL với volume persistent để dữ liệu không mất khi restart, và cache chứa Redis. Tất cả kết nối với nhau qua internal Docker network ppat-network các service giao tiếp bằng tên service thay vì IP cứng, và không service nào tiếp xúc trực tiếp với internet ngoại trừ web qua port 80. Ngoài bốn container cố định này, backend còn tạo thêm container tạm thời khi cần thực thi code của sinh viên qua Dockerode các container này được hủy ngay sau khi thực thi xong và không được khai báo trong Compose file.

CHƯƠNG 2: KHẢO SÁT VÀ PHÂN TÍCH HỆ THỐNG

2.1. Khảo sát thực tế

2.1.1. Quy trình học lập trình hiện nay

Theo báo cáo thị trường nhân lực công nghệ thông tin của TopDev, mỗi năm Việt Nam có hơn 57.000 sinh viên CNTT tốt nghiệp nhưng chỉ khoảng 30% trong số đó đáp ứng được yêu cầu chuyên môn thực tế từ phía doanh nghiệp. Phần lớn sinh viên ra trường nắm lý thuyết tương đối vững nhưng hạn chế về khả năng thực hành và tư duy xử lý các tình huống phát sinh trong bối cảnh làm việc thực tế. Khoảng cách này bắt nguồn từ nhiều nguyên nhân, trong đó một yếu tố quan trọng là cách thức sinh viên tiếp cận lập trình trong giai đoạn học tập tại trường.

Quy trình học lập trình phổ biến hiện nay thường diễn ra theo một vòng lặp quen thuộc: tiếp nhận lý thuyết trên lớp hoặc qua video, tự ngồi làm bài tập, gặp lỗi, rồi tìm kiếm giải pháp qua Google hoặc các công cụ AI như ChatGPT. Vấn đề nằm ở chỗ mỗi lần tìm kiếm như vậy đều bắt đầu lại từ đầu, không có ngữ cảnh về những gì người học đã biết, đang gặp khó khăn ở đâu, hay đã mắc lỗi tương tự bao nhiêu lần trước đó. Các công cụ AI thông thường trả lời đúng câu hỏi được đặt ra nhưng hoàn toàn không lưu lại thông tin về người đặt câu hỏi. Điều này khiến sinh viên dù có thể tìm ra lời giải cho bài toán hiện tại nhưng lại chưa thực sự rèn luyện được khả năng tự phân tích và xử lý vấn đề. Vì vậy, các lỗi tương tự vẫn tiếp tục lặp lại ở những bài tập sau.

Ngoài ra, sĩ số lớp học lớn cũng tạo ra không ít khó khăn trong quá trình giảng dạy. Giảng viên khó theo dõi sát tiến độ của từng sinh viên, đồng thời khó điều chỉnh phương pháp hoặc kịp thời nhận ra khi người học bắt đầu mất kiến thức nền tảng. Sinh viên tiếp thu nhanh phải ngồi chờ, trong khi sinh viên tiếp thu chậm không dám hỏi vì sợ làm chậm cả lớp. Đây là bài toán cấu trúc không thể giải quyết chỉ bằng cách tăng số lượng giảng viên hay điều chỉnh giáo trình.

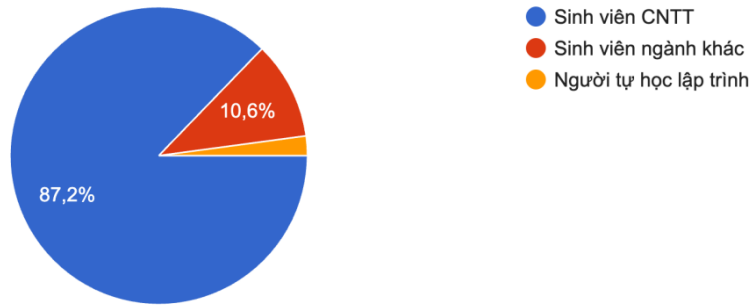
Từ thực trạng đó, xuất hiện nhu cầu xây dựng một hệ thống có thể hỗ trợ người học xuyên suốt quá trình học tập. Hệ thống không chỉ dừng lại ở việc trả lời câu hỏi mà còn cần theo dõi tiến độ học, phát hiện điểm còn hạn chế và đưa ra cách hỗ trợ phù hợp với từng sinh viên. Nội dung khảo sát ở phần tiếp theo sẽ phản ánh rõ vấn đề này thông qua ý kiến từ chính các sinh viên đang theo học lập trình.

2.1.2. Kết quả khảo sát

Nhằm tạo cơ sở thực tiễn cho quá trình phát triển hệ thống, em đã thực hiện một cuộc khảo sát trực tuyến vào tháng 3/2026. Kết quả thu được gồm 47 phản hồi hợp lệ từ sinh viên học lập trình đến từ 15 trường đại học khác nhau. Đối tượng tham gia chủ yếu là sinh viên chuyên ngành Công nghệ Thông tin, chiếm hơn 87%, phần còn lại là sinh viên ngành khác có học lập trình như môn tự chọn hoặc tự học thêm ngoài chương trình.

Bạn hiện đang là?

47 câu trả lời



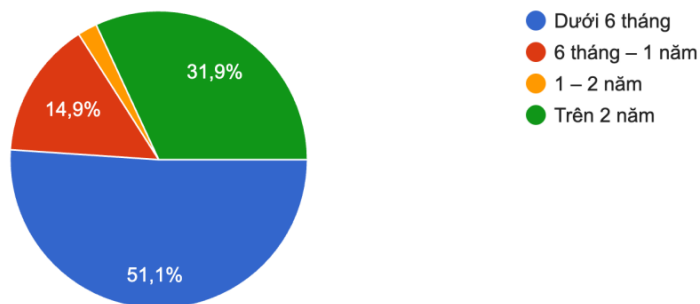
Hình 2.1. Khảo sát chuyên ngành sinh viên

2.1.2.1. Về kinh nghiệm và ngôn ngữ học

Gần 51% người tham gia có dưới sáu tháng kinh nghiệm lập trình, đây là nhóm mới bắt đầu, chưa hình thành thói quen tự học có hệ thống. Nhóm có trên hai năm kinh nghiệm chiếm 32%, chủ yếu là sinh viên năm ba, năm tư đã qua các môn lập trình cơ sở. Ngôn ngữ phổ biến nhất là C++ với gần một nửa số phản hồi, tiếp theo là Python, Java và JavaScript, phản ánh đúng xu hướng đào tạo lập trình cơ sở tại các trường kỹ thuật Việt Nam hiện nay.

Bạn đã học lập trình trong bao lâu?

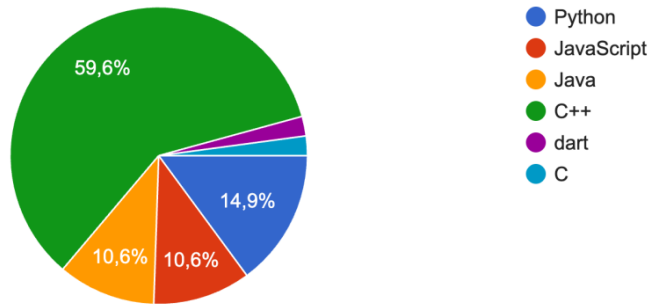
47 câu trả lời



Hình 2.2. Khảo sát kinh nghiệm

Bạn đang học ngôn ngữ lập trình nào?

47 câu trả lời



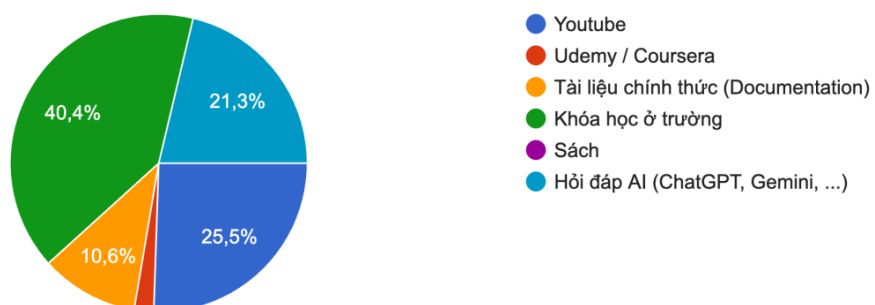
Hình 2.3. Khảo sát ngôn ngữ học

2.1.2.2. Về nguồn học và thói quen sau lý thuyết

Khóa học trên lớp và YouTube là hai kênh được sử dụng nhiều nhất. Đáng chú ý là hỏi đáp AI đã trở thành nguồn học thứ ba, vượt qua cả tài liệu chính thức và nền tảng học trực tuyến trả phí. Điều này cho thấy sinh viên đã chủ động tích hợp AI vào quy trình học tập, song đây là AI tổng quát, không được tối ưu cho bối cảnh học lập trình cá nhân. Sau khi tiếp nhận lý thuyết, phần lớn người được hỏi chọn làm bài tập thực hành ngay, cho thấy nhận thức về tầm quan trọng của thực hành khá tốt. Vấn đề thực sự nằm ở giai đoạn sau khi gặp lỗi, không phải giai đoạn bắt đầu.

Bạn thường học lập trình thông qua nguồn nào?

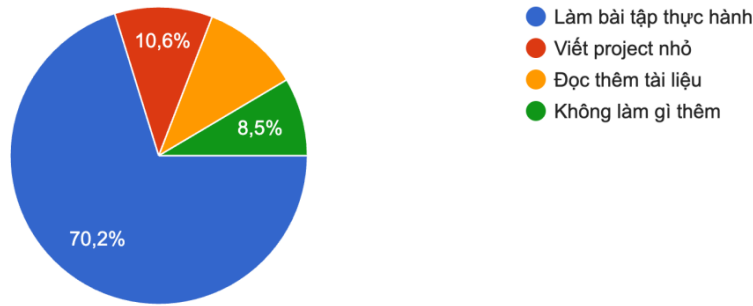
47 câu trả lời



Hình 2.4. Khảo sát nguồn học

Bạn thường làm gì sau khi học lý thuyết lập trình?

47 câu trả lời



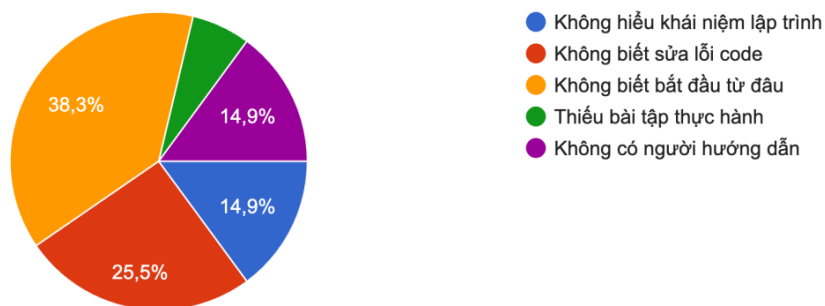
Hình 2.5. Khảo sát thói quen sau lý thuyết

2.1.2.3. Về khó khăn trong quá trình học

Đây là nhóm câu hỏi phản ánh rõ nhất khoảng trống mà hệ thống PPAT hướng đến lấp đầy. Gần 38% người tham gia cho biết khó khăn lớn nhất là không biết bắt đầu từ đâu, tức là thiếu lộ trình học tập cụ thể và không có người dẫn dắt định hướng. Nhóm thứ hai chiếm 26% gặp khó khăn khi không biết cách sửa lỗi code, đây là kỹ năng debug chỉ có thể hình thành qua thực hành lặp lại với phản hồi đúng thời điểm. Khoảng 15% không hiểu được khái niệm lập trình dù đã đọc tài liệu, và 15% thiếu người hướng dẫn bên cạnh khi cần. Tổng hợp lại, hơn 90% người tham gia đang gặp ít nhất một trong những vấn đề mà một gia sư cá nhân có thể trực tiếp giải quyết.

Khó khăn lớn nhất của bạn khi học lập trình là gì?

47 câu trả lời



Hình 2.6. Khảo sát khó khăn trong quá trình học

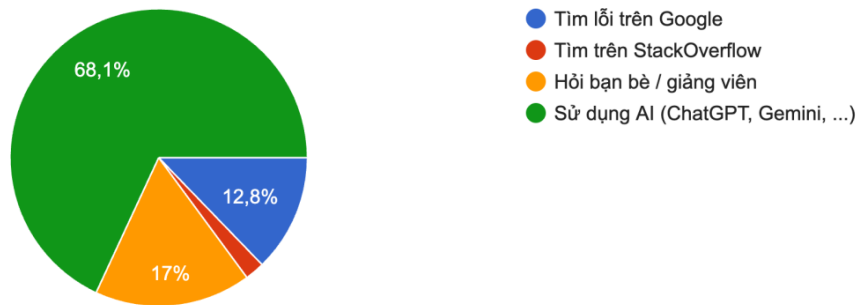
2.1.2.4. Về hành vi khi gặp lỗi code

Hơn một nửa số người được khảo sát, chiếm 68%, chọn công cụ AI làm điểm dừng đầu tiên khi gặp lỗi, vượt xa so với tìm kiếm trên Google ở mức 13% và hỏi bạn bè hoặc giảng viên ở mức 17%. Con số này một mặt khẳng định AI đã trở thành công cụ debug

mặc định trong cộng đồng sinh viên, mặt khác đặt ra câu hỏi về chất lượng hỗ trợ. AI tổng quát có thể xử lý lỗi cụ thể trong một phiên làm việc, nhưng không giải thích được tại sao sinh viên đó hay mắc lỗi đó, cũng không theo dõi xem lỗi có tái diễn ở những bài sau hay không.

Khi gặp lỗi code, bạn thường làm gì?

47 câu trả lời



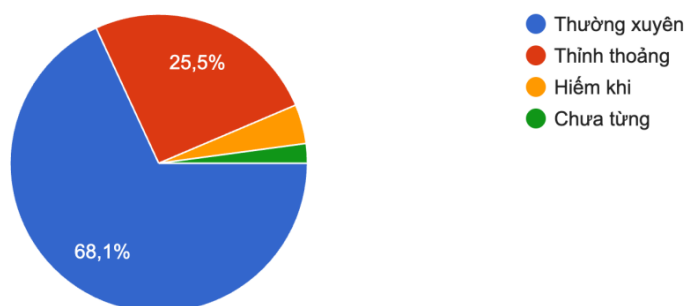
Hình 2.7. Khảo sát hành vi khi gặp lỗi code

2.1.2.5. Về mức độ sử dụng AI trong học tập

68% sử dụng AI thường xuyên và 26% sử dụng thỉnh thoảng, tổng cộng hơn 94% đã có thói quen tương tác với AI trong quá trình học. Đây là tín hiệu quan trọng cho thấy người dùng mục tiêu đã quen với giao diện chat-based và không có rào cản tâm lý khi sử dụng AI. Điều cần cải thiện không phải là thay đổi hành vi người dùng mà là nâng cấp chất lượng AI họ đang dùng, từ AI trả lời đơn lẻ sang AI hiểu toàn bộ hành trình học tập.

Bạn có sử dụng AI để hỗ trợ học lập trình không?

47 câu trả lời



Hình 2.8. Khảo sát mức độ sử dụng AI trong học tập

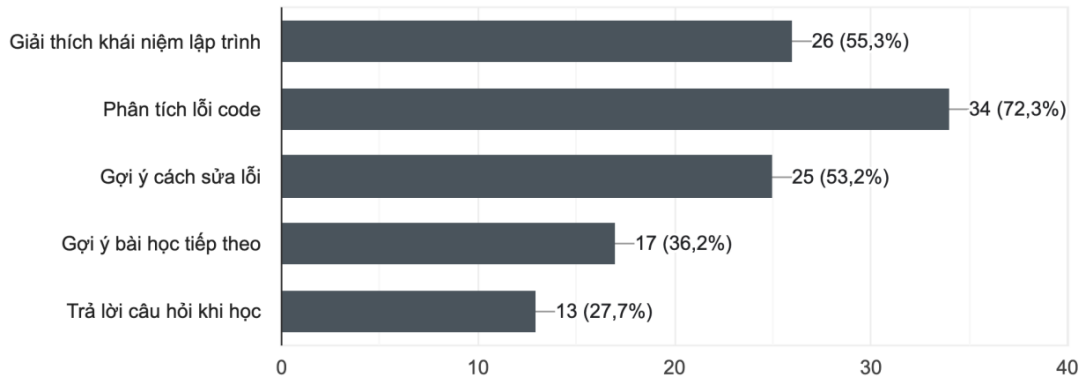
2.1.2.6. Về kỳ vọng chức năng của AI Tutor

Khi được hỏi về các chức năng mong muốn, phân tích lỗi code và gợi ý sửa lỗi là hai tính năng được nhắc đến nhiều nhất, tiếp theo là giải thích khái niệm lập trình và gợi

ý bài học tiếp theo. Một số phản hồi mở đề cập đến nhu cầu về lộ trình học tập chi tiết, nhắc nhở học tập định kỳ và theo dõi tiến độ theo thời gian, đây đều là những tính năng mà các công cụ AI thông thường không cung cấp. Tập hợp này chính là cơ sở để xác định các chức năng cốt lõi của hệ thống PPAT.

Bạn mong muốn hệ thống AI tutor (Gia sư AI) hỗ trợ những chức năng nào?

47 câu trả lời



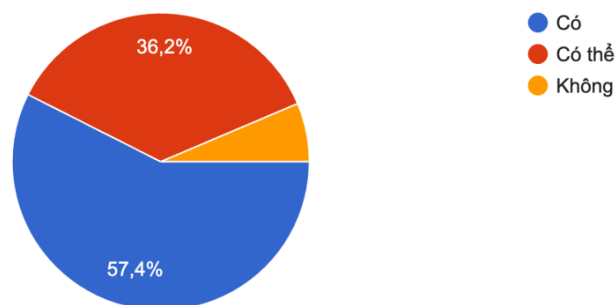
Hình 2.9. Khảo sát kỳ vọng chức năng của AI Tutor

2.1.2.7. Về sự sẵn sàng sử dụng

Gần 57% người được hỏi cho biết sẵn sàng sử dụng một hệ thống AI Tutor chuyên biệt, và thêm 36% trả lời có thể cân nhắc, chỉ có 8% từ chối. Tỷ lệ muốn tham gia thử nghiệm hệ thống lên đến 93%. Những con số này cho thấy nhu cầu thực sự tồn tại và người dùng mục tiêu có thái độ cởi mở, tạo điều kiện thuận lợi cho việc triển khai và thu thập phản hồi trong giai đoạn thử nghiệm.

Bạn có sẵn sàng sử dụng một hệ thống AI tutor để hỗ trợ học lập trình không?

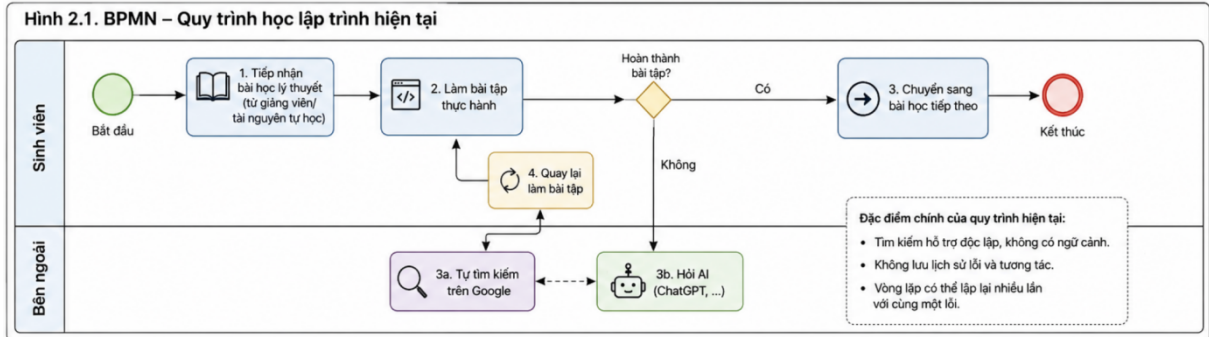
47 câu trả lời



Hình 2.10. Khảo sát sự sẵn sàng sử dụng

2.1.3. Sơ đồ BPMN quy trình học tập

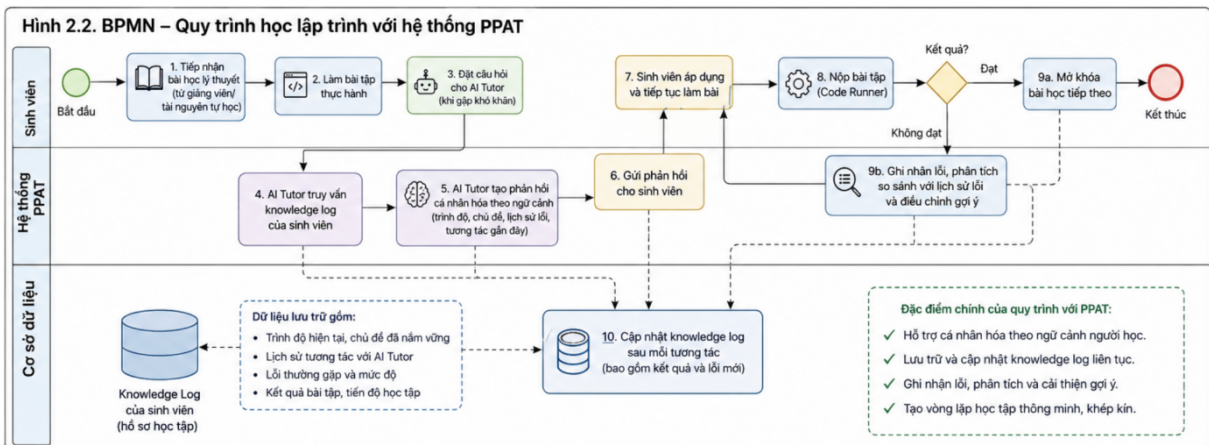
Để làm rõ sự khác biệt giữa cách học lập trình hiện tại và cách học có sự hỗ trợ của hệ thống PPAT, tác giả xây dựng hai sơ đồ quy trình nghiệp vụ theo chuẩn BPMN 2.0, lần lượt mô tả hai luồng hoạt động tương ứng.



Hình 2.11. Quy trình học lập trình hiện tại

Sơ đồ BPMN thứ nhất (Hình 2.11) thể hiện quy trình học lập trình phổ biến mà sinh viên đang thực hiện trước khi có hệ thống hỗ trợ chuyên biệt. Quy trình bắt đầu khi sinh viên tiếp nhận bài học lý thuyết từ giảng viên hoặc tài nguyên tự học như video, tài liệu. Sau đó sinh viên chuyển sang giai đoạn làm bài tập thực hành. Tại đây xuất hiện điểm rẽ nhánh quan trọng: nếu bài tập hoàn thành thành công, sinh viên tiếp tục sang bài học tiếp theo; nếu gặp lỗi hoặc không hiểu, sinh viên phải tự tìm kiếm giải pháp thông qua Google hoặc đặt câu hỏi cho các công cụ AI thông thường như ChatGPT.

Điểm yếu cốt lõi của luồng này nằm ở bước tìm kiếm hỗ trợ. Mỗi phiên tìm kiếm diễn ra hoàn toàn độc lập, không có bất kỳ thông tin nào về lịch sử học tập, trình độ hiện tại hay các lỗi mà sinh viên đó đã từng mắc phải. Sau khi nhận được câu trả lời, sinh viên quay lại làm bài tập nhưng không có cơ chế nào ghi nhận liệu vấn đề đã thực sự được giải quyết triệt để hay chưa. Vòng lặp này có thể lặp đi lặp lại nhiều lần với cùng một dạng lỗi mà không tạo ra sự cải thiện bền vững trong khả năng của người học.



Hình 2.12. Quy trình học lập trình với hệ thống PPAT

Sơ đồ BPMN thứ hai (Hình 2.12) mô tả quy trình học tập khi sinh viên sử dụng hệ thống PPAT. Điểm khởi đầu tương tự: sinh viên tiếp nhận bài học và chuyển sang làm bài tập. Tuy nhiên từ đây, luồng xử lý có sự khác biệt căn bản.

Khi sinh viên gặp khó khăn và đặt câu hỏi cho AI Tutor, hệ thống không xử lý yêu cầu như một câu hỏi đơn lẻ. Trước khi sinh ra phản hồi, hệ thống truy vấn knowledge log của sinh viên đó từ cơ sở dữ liệu, bao gồm trình độ hiện tại, các chủ đề đã nắm vững, các lỗi thường gặp và lịch sử tương tác gần đây. Dựa trên tập thông tin này, AI Tutor xây dựng phản hồi được cá nhân hóa theo đúng ngữ cảnh của người học. Sau mỗi lần tương tác, hệ thống tự động cập nhật lại knowledge log, ghi nhận thêm thông tin mới về hành vi và tiến độ của sinh viên.

Khi sinh viên nộp bài tập, hệ thống chấm qua Code Runner và phân nhánh theo kết quả. Nếu vượt qua, bài học tiếp theo được mở khóa và knowledge log được cập nhật theo hướng tích cực. Nếu thất bại, hệ thống ghi nhận loại lỗi, so sánh với lịch sử lỗi đã có và điều chỉnh gợi ý phù hợp cho lần tiếp theo. Toàn bộ vòng lặp này tạo ra một cơ chế học tập khép kín, trong đó mỗi tương tác đều đóng góp vào việc làm phong phú thêm hồ sơ học tập của sinh viên và nâng cao chất lượng hỗ trợ ở những phiên sau.

So sánh hai sơ đồ cho thấy sự khác biệt không chỉ nằm ở công nghệ sử dụng mà còn ở triết lý thiết kế. Quy trình cũ xử lý từng sự kiện học tập một cách rời rạc, trong khi quy trình mới kết nối tất cả các sự kiện đó thành một hành trình học tập liên tục có trí nhớ và khả năng thích nghi.

2.1.4. Phân tích hệ thống hiện có

Trước khi xây dựng hệ thống PPAT, em tiến hành phân tích các công cụ hỗ trợ học lập trình hiện có mà sinh viên đang sử dụng phổ biến, trong đó ChatGPT là đại diện tiêu biểu nhất. Mục tiêu của phân tích này là xác định rõ những giới hạn của công cụ hiện tại, từ đó làm cơ sở lý giải sự cần thiết của một hệ thống chuyên biệt hơn.

ChatGPT và các công cụ AI tổng quát tương tự đang được phần lớn sinh viên sử dụng như một trợ lý học tập không chính thức. Kết quả khảo sát ở mục trước cho thấy hơn 53% sinh viên chọn AI làm điểm dừng đầu tiên khi gặp lỗi code. Điều này phản ánh một thực tế rõ ràng: các công cụ này có khả năng trả lời nhanh, phủ rộng nhiều chủ đề và dễ tiếp cận. Tuy nhiên, chính vì được thiết kế cho mục đích tổng quát nên chúng bộc lộ nhiều hạn chế khi được dùng trong ngữ cảnh học tập có hệ thống.

Hạn chế lớn nhất là không có khả năng ghi nhớ lịch sử học tập xuyên suốt nhiều phiên làm việc. Mỗi cuộc trò chuyện với ChatGPT bắt đầu lại từ đầu, không có bất kỳ thông tin nào về người dùng là ai, đang học đến đâu, hay đã gặp khó khăn gì trước đó. Điều này khiến mỗi phản hồi đều mang tính chung chung, không được điều chỉnh theo trình độ thực tế của người học. Một sinh viên mới bắt đầu học lập trình và một sinh viên năm ba có thể nhận được câu trả lời gần như giống nhau cho cùng một câu hỏi, dù nhu cầu và nền tảng kiến thức của hai người hoàn toàn khác biệt.

Hạn chế thứ hai liên quan đến chiến lược dạy học. ChatGPT có xu hướng cung cấp đáp án trực tiếp thay vì dẫn dắt người học tự tìm ra giải pháp. Cách tiếp cận này tuy nhanh nhưng không giúp sinh viên hình thành tư duy giải quyết vấn đề, vốn là kỹ năng cốt lõi mà bất kỳ lập trình viên nào cũng cần có. Sinh viên nhận được code hoạt động nhưng không hiểu tại sao code cũ của mình sai và lần sau gặp vấn đề tương tự vẫn phải hỏi lại từ đầu.

Hạn chế thứ ba là không có cơ chế theo dõi tiến độ và phát hiện điểm yếu. ChatGPT không biết sinh viên đó đã hoàn thành bao nhiêu bài tập, hay mắc lỗi ở dạng bài nào nhiều nhất. Do đó không thể chủ động đề xuất nội dung ôn tập, điều chỉnh độ khó hay cảnh báo khi phát hiện một lỗ hổng kiến thức đang lặp lại. Người học hoàn toàn phải tự điều hướng quá trình học của mình mà không có bất kỳ phản hồi có cấu trúc nào từ hệ thống.

Ngoài ra, ChatGPT không tích hợp môi trường thực thi code. Sinh viên phải chạy code trên môi trường riêng rồi sao chép kết quả hoặc thông báo lỗi vào chat, tạo ra một bước thủ công làm gián đoạn luồng học tập. Trong khi đó, việc thực thi và kiểm tra code ngay trong cùng một giao diện là yếu tố quan trọng để tạo ra vòng phản hồi nhanh, giúp sinh viên học qua thực hành hiệu quả hơn.

2.2. Xác định Actor

Hệ thống PPAT được thiết kế để phục vụ hai nhóm người dùng với vai trò và nhu cầu khác nhau. Việc xác định rõ các actor tham gia vào hệ thống là bước quan trọng để đảm bảo các chức năng được xây dựng đúng đối tượng và đáp ứng đúng nhu cầu thực tế.

Actor thứ nhất là Sinh viên, người dùng trung tâm và quan trọng nhất của hệ thống. Sinh viên tương tác trực tiếp với toàn bộ các chức năng cốt lõi bao gồm xem bài học, làm bài tập, chat với AI Tutor và theo dõi tiến độ học tập của bản thân. Để truy cập bất kỳ chức năng nào, sinh viên bắt buộc phải đăng ký tài khoản và đăng nhập vào hệ thống. Mọi cơ chế cá nhân hóa trong hệ thống đều hướng đến việc phục vụ tốt nhất cho nhóm actor này.

Actor thứ hai là Admin đóng vai trò quản trị toàn bộ hệ thống. Admin có quyền quản lý tài khoản người dùng, tạo mới, chỉnh sửa và xóa bài học, bài tập cùng các test case tương ứng. Khác với sinh viên chỉ có quyền đọc và tương tác với nội dung, Admin có toàn quyền kiểm soát kho nội dung học tập. Admin cũng cần đăng nhập vào hệ thống trước khi thực hiện bất kỳ thao tác quản trị nào.

2.3. Yêu cầu chức năng

Dựa trên kết quả khảo sát thực tế và phân tích các actor tham gia hệ thống, em xác định các nhóm yêu cầu chức năng chính:

2.3.0.1. Quản lý người dùng

- Đăng ký tài khoản bằng email và mật khẩu.
- Đăng nhập, xác thực bằng JWT với thời hạn 24 giờ và refresh token 7 ngày.

- Cập nhật thông tin cá nhân, mật khẩu, trình độ ban đầu.

2.3.0.2. *Quản lý bài học*

- Xem danh sách bài học, lọc theo ngôn ngữ lập trình, chủ đề, mức độ khó.
- Xem chi tiết bài học với nội dung lý thuyết và ví dụ code có syntax highlighting.
- Theo dõi trạng thái hoàn thành của từng bài học.

2.3.0.3. *Quản lý bài tập*

- Xem đề bài kèm starter code tương ứng.
- Viết và chỉnh sửa code trực tiếp trong Monaco Editor tích hợp.
- Nộp bài và kiểm tra kết quả tự động qua test cases.
- Nhận gợi ý sửa lỗi theo mức độ tương ứng với số lần thất bại liên tiếp.

2.3.0.4. *Tương tác chat với AI Tutor*

- Gửi câu hỏi và nhận phản hồi cá nhân hóa từ GPT-4o theo trình độ và lịch sử học tập.
- Nhận phản hồi theo cơ chế streaming token-by-token, không chờ toàn bộ nội dung.
- Ghi nhớ Context, lưu lịch sử hội thoại vĩnh viễn, có thể xem lại bất kỳ lúc nào.
- Tạo nhiều phiên hội thoại độc lập cho các chủ đề khác nhau.

2.3.0.5. *Theo dõi tiến độ học tập*

- Xem tổng quan tiến độ: số bài đã hoàn thành, chuỗi ngày học liên tiếp.
- Xem biểu đồ radar thể hiện điểm mạnh và điểm yếu theo từng chủ đề.
- Xem danh sách các lỗi thường gặp được phân tích từ lịch sử nộp bài.
- Xem đề xuất bài học tiếp theo dựa trên knowledge log hiện tại.

2.3.0.6. *Quản lý nội dung (Admin)*

- Tạo mới, chỉnh sửa và xóa bài học, bài tập và test cases.
- Phân loại nội dung theo ngôn ngữ, chủ đề và mức độ khó.

2.3.0.7. *Cập nhật Knowledge Log (Hệ thống tự động)*

- Tự động cập nhật hồ sơ học tập sau mỗi lần tương tác chat hoặc nộp bài.
- Ghi nhận loại lỗi, tần suất mắc lỗi và tốc độ tiến bộ của từng sinh viên.
- Điều chỉnh lộ trình học tập gợi ý dựa trên dữ liệu tích lũy theo thời gian.

Bảng 2.1. Danh sách yêu cầu chức năng

Mã	Tên yêu cầu	Mô tả ngắn
FR01	Quản lý người dùng	Đăng ký, đăng nhập JWT, cập nhật profile
FR02	Quản lý bài học	Danh sách bài học, lọc theo ngôn ngữ/chủ đề/độ khó
FR03	Quản lý bài tập	Xem đề, code editor, submit, chạy test case, xem gợi ý

FR04	Chat với AI Tutor	Chat streaming SSE, phản hồi cá nhân hóa, lưu lịch sử
FR05	Theo dõi tiến độ	Dashboard: streak, bài hoàn thành, điểm mạnh/yếu
FR06	Quản lý nội dung	CRUD bài học, bài tập, test cases, import/export
FR07	Cập nhật Knowledge Log	Tự động sau mỗi interaction, trigger điều chỉnh lộ trình

2.4. Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống PPAT còn phải đáp ứng một tập hợp các yêu cầu phi chức năng liên quan đến hiệu năng, bảo mật, khả dụng và trải nghiệm người dùng.

2.4.0.1. Hiệu năng

- Thời gian phản hồi của các API không gọi LLM không vượt quá 200 mili giây trong điều kiện tải bình thường.
- Hệ thống xử lý đồng thời tối thiểu 50 người dùng trong môi trường triển khai phát triển.
- Chức năng chat AI sử dụng cơ chế streaming, người dùng nhận được token đầu tiên ngay lập tức thay vì chờ toàn bộ phản hồi.

2.4.0.2. Bảo mật

- Xác thực người dùng bằng JWT với thời hạn 24 giờ, kết hợp refresh token có hiệu lực 7 ngày.
- Mật khẩu được mã hóa bằng bcrypt trước khi lưu vào cơ sở dữ liệu.
- Code của sinh viên được thực thi trong Docker container cô lập, giới hạn thời gian chạy 10 giây, không có quyền truy cập mạng và không thể ghi ra filesystem của máy chủ.
- Các endpoint nhạy cảm được bảo vệ bằng rate limiting để ngăn chặn tấn công brute force.

2.4.0.3. Tính ổn định và tin cậy

- Hệ thống duy trì uptime 99% trong khung giờ học tập từ 7 giờ sáng đến 10 giờ tối.
- Dữ liệu PostgreSQL được sao lưu tự động hàng ngày.
- Redis được cấu hình chế độ AOF để đảm bảo không mất dữ liệu session khi khởi động lại.

2.4.0.4. Khả năng mở rộng

- Kiến trúc SOA cho phép thêm service mới như analytics hoặc notification độc lập mà không ảnh hưởng đến các thành phần hiện tại.

- Code Runner có thể được tách thành microservice riêng biệt hoặc mở rộng hỗ trợ thêm ngôn ngữ lập trình khi cần.

2.4.0.5. *Khả năng sử dụng*

- Giao diện trực quan, thao tác nhất quán giữa các màn hình.
- Monaco Editor tích hợp sẵn syntax highlighting và hỗ trợ nhiều ngôn ngữ lập trình.
- Phản hồi streaming tạo trải nghiệm tự nhiên, giảm cảm giác chờ đợi khi tương tác với AI.

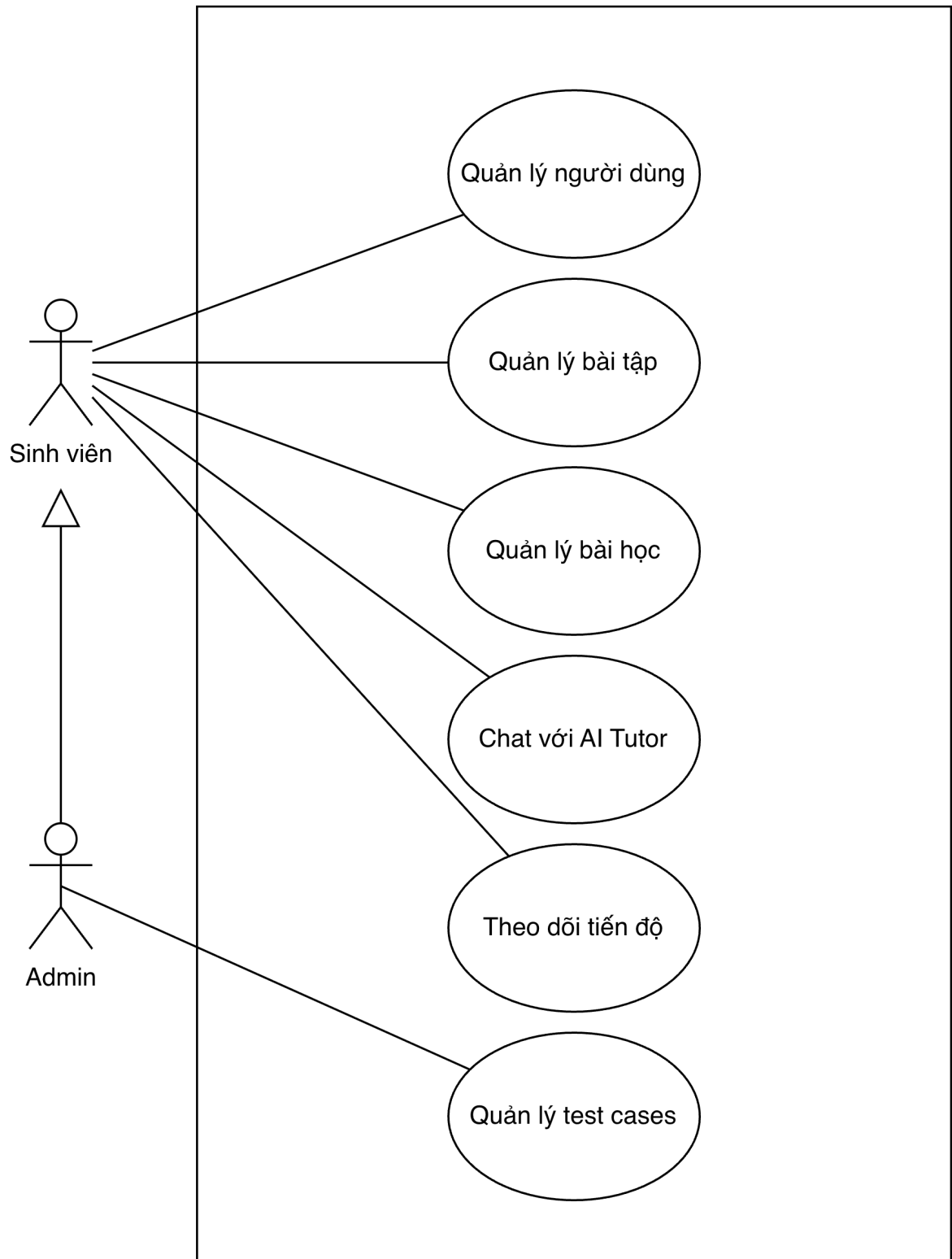
2.4.0.6. *Khả năng tương thích*

- Hoạt động tốt trên các trình duyệt phổ biến bao gồm Chrome, Edge và Firefox.
- Giao diện responsive, hỗ trợ sử dụng trên cả máy tính và thiết bị di động.

2.5. Mô hình nghiệp vụ

2.5.1. *Biểu đồ Use Case*

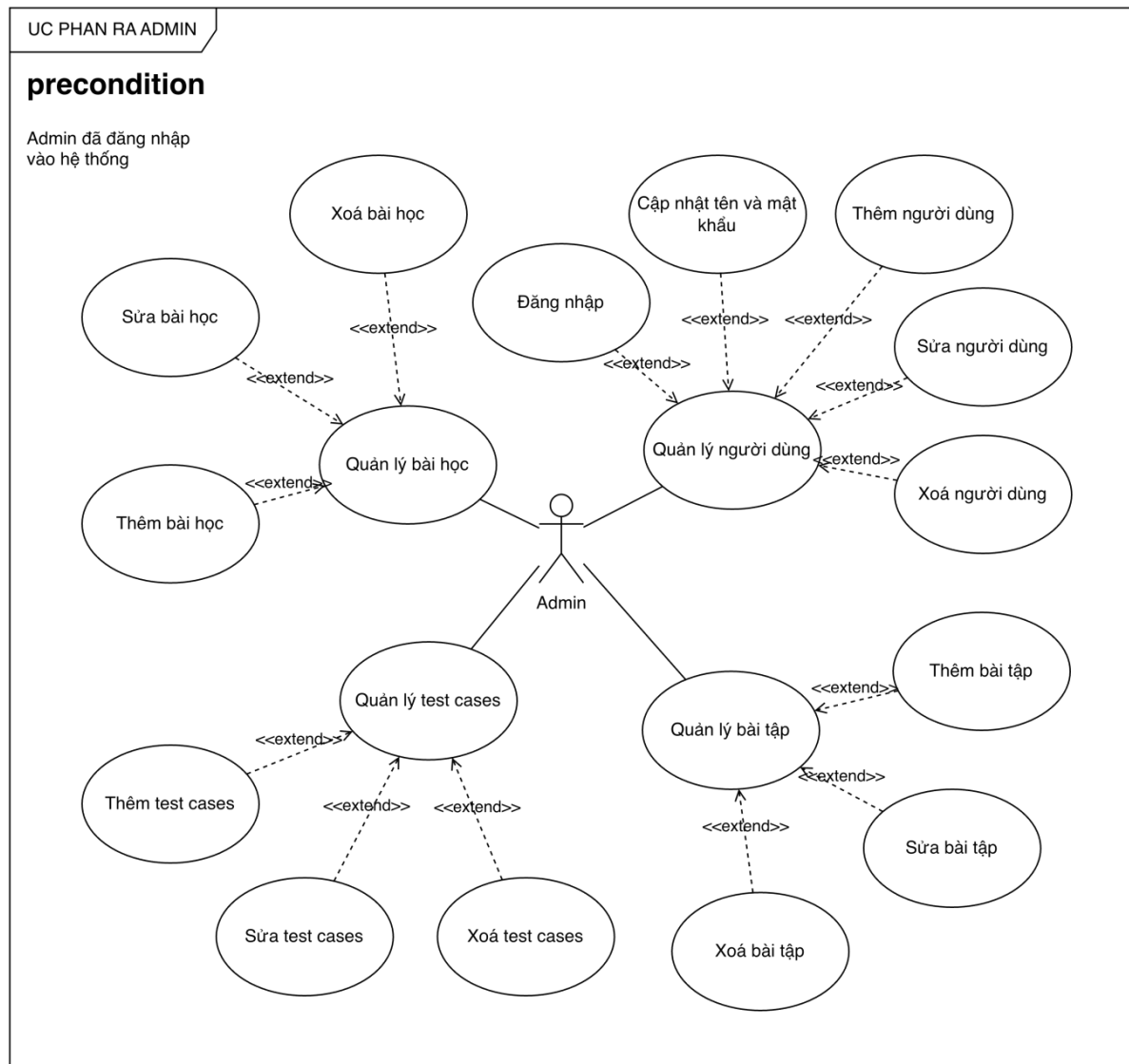
- a. Biểu đồ UC tổng quát



Hình 2.13. Biểu đồ UC tổng quát

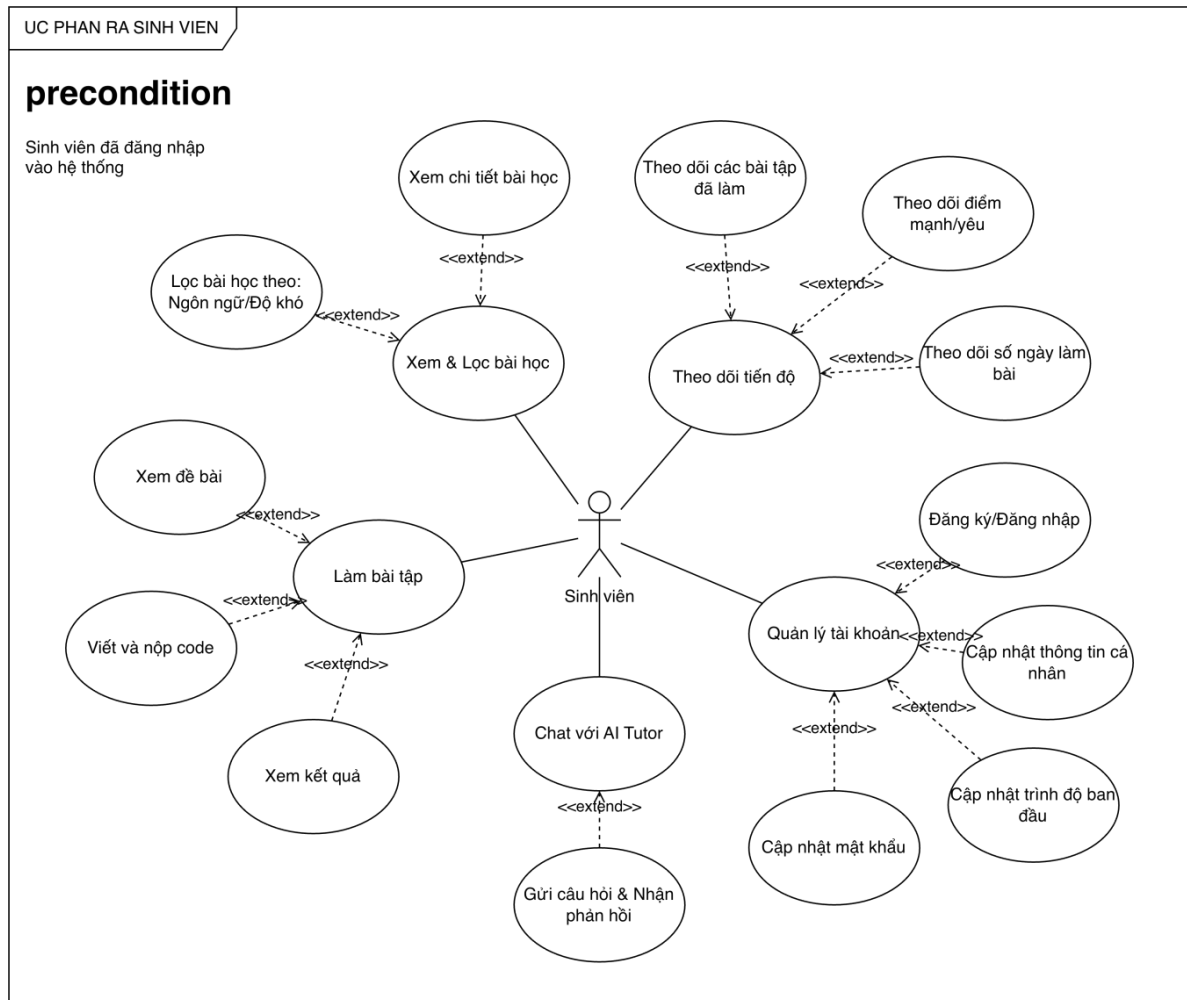
b. Biểu đồ UC phân rã

- Biểu đồ phân rã cho tác nhân Admin



Hình 2.14. Biểu đồ phân rã cho tác nhân Admin

- Biểu đồ phân rã cho tác nhân Sinh viên



Hình 2.15. Biểu đồ phân rã cho tác nhân Sinh viên

2.5.2. Đặc tả Use Case

2.5.2.1. Đặc tả chức năng Đăng ký (Sinh viên)

Bảng 2.2. Bảng đặc tả chức năng Đăng ký (Sinh viên)

Mã Use Case	UC01
Tên Use Case	Đăng ký tài khoản
Tác nhân chính	Sinh viên
Mô tả	Người dùng (Sinh viên) thực hiện tạo tài khoản mới trên hệ thống để bắt đầu quá trình học tập.
Tiền điều kiện	- Người dùng chưa có tài khoản trong hệ thống. - Hệ thống kết nối ổn định với CSDL.

Hậu điều kiện	- Một bản ghi người dùng mới được tạo trong CSDL với vai trò là student. - Hệ thống cấp mã thông báo (Access Token) và tự động đăng nhập cho người dùng.
Luồng chính	1. Người dùng chọn chức năng "Đăng ký" từ giao diện. 2. Hệ thống hiển thị form đăng ký bao gồm các trường: Email, Mật khẩu, Tên hiển thị, Trình độ (Beginner/Intermediate/Advanced). 3. Người dùng nhập đầy đủ và chính xác các thông tin yêu cầu. 4. Người dùng nhấn nút "Đăng ký". 5. Hệ thống thực hiện kiểm tra dữ liệu: 5.1. Kiểm tra các trường bắt buộc không được để trống. 5.2. Kiểm tra định dạng Email hợp lệ. 5.3. Kiểm tra tính duy nhất của Email trong CSDL. 6. Hệ thống thực hiện mã hóa mật khẩu (bằng bcrypt). 7. Hệ thống lưu thông tin người dùng mới vào CSDL. 8. Hệ thống tạo Access Token và Refresh Token. 9. Hệ thống hiển thị thông báo "Đăng ký thành công" và chuyển hướng người dùng vào trang chủ.
Luồng phụ	- 5.1.a. Nếu thông tin bị thiếu: Hệ thống hiển thị thông báo "Vui lòng điền đầy đủ các trường bắt buộc". - 5.2.a. Nếu Email sai định dạng: Hệ thống hiển thị cảnh báo "Email không hợp lệ". - 5.3.a. Nếu Email đã tồn tại: Hệ thống hiển thị lỗi "Email đã được sử dụng".
Ngoại lệ	- 7.a. Nếu quá trình lưu vào CSDL thất bại (lỗi server): Hệ thống thông báo "Đăng ký thất bại, vui lòng thử lại sau". - 7.b. Mất kết nối CSDL: Hệ thống hiển thị lỗi kết nối máy chủ.

2.5.2.2. Đặc tả chức năng Đăng nhập

Bảng 2.3. Bảng đặc tả chức năng Đăng nhập (Sinh viên, Admin)

Mã Use Case	UC01
Tên Use Case	Đăng nhập hệ thống
Tác nhân chính	Sinh viên, Admin

Mô tả	Người dùng thực hiện đăng nhập vào hệ thống để truy cập vào các tính năng theo phân quyền của mình.
Tiền điều kiện	- Người dùng đã có tài khoản hợp lệ trong hệ thống. - Hệ thống đang kết nối ổn định với CSDL.
Hậu điều kiện	- Người dùng truy cập thành công vào hệ thống. - Hệ thống trả về thông tin hồ sơ và mã xác thực (Access Token, Refresh Token).
Luồng chính	1. Người dùng truy cập vào trang Đăng nhập. 2. Hệ thống hiển thị form đăng nhập bao gồm: Email và Mật khẩu. 3. Người dùng nhập Email và Mật khẩu sau đó nhấn nút "Đăng nhập". 4. Hệ thống kiểm tra thông tin đăng nhập: 4.1. Truy vấn thông tin người dùng theo Email. 4.2. So sánh mật khẩu đã nhập với bản mã hóa trong CSDL (sử dụng bcrypt.compare). 5. Nếu thông tin khớp: 5.1. Hệ thống tạo phiên làm việc mới (Access Token 24h, Refresh Token 7d). 5.2. Hệ thống xác định vai trò (Role: admin/student) và các quyền hạn tương ứng. 5.3. Hệ thống trả về thông báo "Đăng nhập thành công". 6. Hệ thống chuyển hướng người dùng về trang chủ hoặc trang Dashboard theo vai trò.
Luồng phụ	- 5.a. Nếu Email hoặc Mật khẩu không chính xác: Hệ thống hiển thị thông báo "Sai email hoặc mật khẩu" và yêu cầu nhập lại. - 5.b. Nếu phát hiện Refresh Token cũ còn hiệu lực: Hệ thống thực hiện thu hồi và cấp mã mới để đảm bảo bảo mật.
Ngoại lệ	- 4.1.a. Nếu CSDL bị lỗi: Hệ thống hiển thị thông báo lỗi hệ thống chung. - 4.1.b. Mất kết nối mạng: Hiển thị lỗi kết nối máy chủ.

2.5.2.3. Đặc tả chức năng Quản lý thông tin cá nhân

Bảng 2.4. Bảng đặc tả chức năng Quản lý thông tin cá nhân

Mã Use Case	UC01
--------------------	------

Tên Use Case	Quản lý thông tin cá nhân
Tác nhân chính	Sinh viên, Admin
Mô tả	Cho phép người dùng tự cập nhật thông tin cá nhân của mình bao gồm tên hiển thị, mật khẩu và trình độ học tập hiện tại.
Tiền điều kiện	Người dùng đã đăng nhập thành công vào hệ thống.
Hậu điều kiện	Thông tin cá nhân của sinh viên, Admin được cập nhật thành công trong CSDL.
Luồng chính	1. Người dùng chọn chức năng "Thông tin tài khoản". 2. Hệ thống hiển thị thông tin hiện tại: Tên hiển thị, Trình độ hiện tại. 3. Người dùng thực hiện các thay đổi mong muốn: 3.1. Thay đổi tên hiển thị. 3.2. Thay đổi trình độ học tập (Beginner/Intermediate/Advanced). 3.3. Thay đổi mật khẩu (yêu cầu nhập mật khẩu cũ và mật khẩu mới). 4. Người dùng nhấn "Lưu thay đổi". 5. Hệ thống kiểm tra tính hợp lệ của dữ liệu: 5.1. Kiểm tra mật khẩu cũ có khớp với dữ liệu trong CSDL hay không. 5.2. Kiểm tra độ mạnh của mật khẩu mới (nếu có thay đổi). 6. Hệ thống cập nhật thông tin mới vào CSDL. 7. Hệ thống hiển thị thông báo "Cập nhật thông tin thành công".
Luồng phụ	- 5.1.a. Nếu mật khẩu cũ không chính xác: Hệ thống báo lỗi "Mật khẩu cũ không chính xác" và yêu cầu nhập lại. - 5.2.a. Nếu thông tin nhập vào không hợp lệ (ví dụ tên quá ngắn): Hệ thống hiển thị cảnh báo tương ứng.
Ngoại lệ	- 6.a. Nếu quá trình cập nhật gặp lỗi CSDL: Hệ thống thông báo lỗi hệ thống và yêu cầu thử lại sau. - 6.b. Mất kết nối mạng: Hiển thị lỗi kết nối máy chủ.

2.5.2.4. Đặc tả chức năng Quản lý người dùng

Bảng 2.5. Bảng đặc tả chức năng Quản lý người dùng

Mã Use Case	UC06
--------------------	------

Tên Use Case	Quản lý người dùng
Tác nhân chính	Admin
Mô tả	Admin thực hiện quản lý toàn diện (thêm, sửa, xóa) các tài khoản người dùng khác trong hệ thống.
Tiền điều kiện	Admin đã đăng nhập thành công với quyền quản trị viên.
Hậu điều kiện	Các thay đổi về thông tin người dùng khác được lưu thành công vào CSDL.
Luồng chính	- Thêm người dùng mới: - Admin chọn "Thêm người dùng". - Nhập Email, Tên, Vai trò (Admin/Student), Mật khẩu và Trình độ. - Hệ thống kiểm tra tính duy nhất của Email và lưu vào CSDL. - Sửa thông tin người dùng khác: - Admin chọn người dùng cần sửa từ danh sách quản lý. - Hệ thống hiển thị form chỉnh sửa. - Admin thay đổi thông tin (trừ Email) và nhấn "Cập nhật". - Hệ thống ghi đè thông tin mới vào CSDL. - Xóa tài khoản người dùng: - Admin chọn tài khoản cần xóa và bấm "Xóa". - Hệ thống hiển thị hộp thoại xác nhận xóa. - Admin xác nhận "Đồng ý". - Hệ thống xóa vĩnh viễn dữ liệu người dùng và các thông tin liên quan, báo "Xóa thành công".
Luồng phụ	- 1.2.1. Email trùng: Báo lỗi "Email đã được đăng ký cho tài khoản khác". - 3.3.1. Admin chọn "Hủy": Đóng hộp thoại, không thay đổi dữ liệu.
Ngoại lệ	- 2.4.a. Lỗi hệ thống khi cập nhật: Thông báo "Cập nhật thất bại, vui lòng thử lại". - 3.4.a. Mất kết nối CSDL: Hiển thị lỗi kết nối máy chủ.

2.5.2.5. Đặc tả chức năng Quản lý bài học

Bảng 2.6. Bảng đặc tả chức năng Quản lý bài học

Mã Use Case	UC06
Tên Use Case	Quản lý bài học
Tác nhân chính	Admin
Mô tả	Usecase này cho phép Admin thực hiện các thao tác thêm, sửa, xóa các bài học trong hệ thống để cung cấp nội dung học tập cho người dùng.
Tiền điều kiện	Admin đã đăng nhập thành công vào hệ thống với quyền quản trị viên.
Hậu điều kiện	Thông tin bài học được cập nhật thành công trong CSDL.
Luồng chính	1. Thêm Bài học: 1.1. Admin chọn chức năng "Thêm bài học". 1.2. Hệ thống hiển thị biểu mẫu bao gồm: Tiêu đề, Ngôn ngữ, Chủ đề, Độ khó (Dễ/Trung bình/Khó) và Nội dung (Markdown). 1.3. Admin nhập đầy đủ thông tin và chọn "Lưu". 1.4. Hệ thống kiểm tra dữ liệu: 1.4.1. Nếu hợp lệ: Lưu thông tin bài học mới vào CSDL và báo "Thêm bài học thành công". 1.4.2. Nếu không hợp lệ: Hiển thị thông báo lỗi. 2. Sửa Bài học: 2.1. Admin chọn bài học cần chỉnh sửa từ danh sách. 2.2. Hệ thống hiển thị thông tin bài học hiện tại. 2.3. Admin thực hiện thay đổi và chọn "Cập nhật". 2.4. Hệ thống kiểm tra và ghi đè dữ liệu mới vào CSDL, báo "Cập nhật thành công". 3. Xóa Bài học: 3.1. Admin chọn bài học và yêu cầu xóa. 3.2. Hệ thống hiển thị hộp thoại xác nhận. 3.3. Admin chọn "Đồng ý": 3.3.1. Hệ thống xóa bài học và các dữ liệu liên quan (tập bài tập, tiến độ) khỏi hệ thống. 3.3.2. Hiển thị thông báo "Xóa bài học thành công".
Luồng phụ	- 1.3.1. Thiếu thông tin bắt buộc: Hệ thống yêu cầu người dùng điền đầy đủ. - 3.3.2. Admin chọn "Hủy": Đóng hộp thoại, không thay đổi dữ liệu.

Ngoại lệ	- 1.4.a. Lỗi kết nối CSDL: Hệ thống hiển thị thông báo lỗi kết nối. - 3.3.1.a. Xóa thất bại do lỗi ràng buộc: Thông báo lỗi hệ thống.
-----------------	--

2.5.2.6. *Đặc tả chức năng Quản lý bài tập*

Bảng 2.7. *Bảng đặc tả chức năng Quản lý bài tập*

Mã Use Case	UC06
Tên Use Case	Quản lý bài tập
Tác nhân chính	Admin
Mô tả	Usecase này cho phép Admin thiết kế và quản lý các bài tập đi kèm với bài học, bao gồm các thông tin về mã nguồn khởi tạo, gợi ý và kết quả mong muốn.
Tiền điều kiện	Admin đã đăng nhập thành công vào hệ thống.
Hậu điều kiện	Thông tin tập bài tập được lưu trữ hoặc cập nhật thành công trong CSDL.

Luồng chính	1. Thêm Bài Tập: 1.1. Admin chọn bài học muốn thêm bài tập và chọn "Thêm bài tập". 1.2. Hệ thống hiển thị biểu mẫu: Tiêu đề, Mô tả, Mã nguồn khởi tạo, Kết quả mong muốn, Gợi ý, Bài học tiên quyết. 1.3. Admin nhập các thông tin chi tiết và chọn "Lưu". 1.4. Hệ thống kiểm tra và lưu vào CSDL, báo "Thêm bài tập thành công". 2. Sửa Bài Tập: 2.1. Admin chọn bài tập cần chỉnh sửa. 2.2. Hệ thống tải dữ liệu hiện tại lên biểu mẫu. 2.3. Admin thay đổi nội dung (ví dụ gợi ý hoặc mã khởi tạo) và nhấn "Cập nhật". 2.4. Hệ thống lưu các thay đổi và thông báo thành công. 3. Xóa Bài Tập: 3.1. Admin chọn bài tập cần xóa. 3.2. Hệ thống yêu cầu xác nhận. 3.3. Admin xác nhận "Xóa vĩnh viễn": 3.3.1. Hệ thống xóa bài tập và các bản nộp bài (submission) liên quan của người dùng. 3.3.2. Thông báo xóa thành công.
Luồng phụ	- 1.2.1. Không chọn bài học liên kết: Hệ thống yêu cầu chọn bài học để gán bài tập. - 2.2.1. Thông tin nhập sai định dạng: Hệ thống hiển thị cảnh báo.
Ngoại lệ	- 1.4.a. Lỗi hệ thống khi lưu: Hiển thị thông báo thử lại sau. - 3.3.1.a. Mất kết nối CSDL: Hiển thị lỗi kết nối.

2.5.2.7. Đặc tả chức năng Quản lý test cases

Bảng 2.8. Bảng đặc tả chức năng Quản lý test cases

Mã Use Case	UC06
Tên Use Case	Quản lý test cases
Tác nhân chính	Admin

Mô tả	Cho phép Admin thiết lập các bộ dữ liệu kiểm thử (đầu vào và đầu ra mong muốn) cho từng bài tập để hệ thống tự động chấm điểm bài nộp của người dùng.
Tiền điều kiện	Admin đã đăng nhập và bài tập tương ứng đã tồn tại trong hệ thống.
Hậu điều kiện	Danh sách test cases được cập nhật và áp dụng cho quá trình kiểm tra mã nguồn của người dùng.
Luồng chính	- Thêm bộ kiểm thử: - Admin chọn một bài tập cụ thể và mở mục "Cấu hình Test Cases". - Admin nhấn "Thêm bộ kiểm thử mới". - Nhập "Đầu vào (Input)" và "Đầu ra mong muốn (Expected Output)". - Admin chọn "Lưu". - Hệ thống thêm cặp dữ liệu vào danh sách kiểm thử của bài tập đó trong CSDL. - Sửa bộ kiểm thử: - Admin chọn test case trong danh sách của bài tập. - Admin cập nhật giá trị đầu vào hoặc đầu ra. - Hệ thống ghi lại thay đổi. - Xóa bộ kiểm thử: - Admin nhấn biểu tượng xóa tại một dòng test case. - Hệ thống thực hiện xóa ngay lập tức khỏi bộ nhớ bản ghi bài tập. - Hiển thị thông báo "Đã loại bỏ test case".
Luồng phụ	- 1.3.1. Định dạng dữ liệu không khớp: Hệ thống báo lỗi nếu kiểu dữ liệu input/output không hợp lệ cho ngôn ngữ lập trình của bài học. - 1.5.1. Dữ liệu rỗng: Hệ thống yêu cầu nhập đầy đủ trước khi lưu.
Ngoại lệ	- 1.5.a. Lỗi cập nhật CSDL: Thông báo "Lưu cấu hình thất bại". - 3.2.a. Xóa test case cuối cùng: Hệ thống cảnh báo bài tập cần ít nhất một bộ kiểm thử để hoạt động.

2.5.2.8. Đặc tả chức năng Xem & Lọc bài học

Bảng 2.9. Bảng đặc tả chức năng Xem & Lọc bài học

Mã Use Case	UC02
Tên Use Case	Xem & Lọc bài học
Tác nhân chính	Sinh viên
Mô tả	Sinh viên truy cập vào thư viện bài học để xem danh sách các bài học, lọc theo tiêu chí ngôn ngữ lập trình và xem chi tiết nội dung lý thuyết của từng bài.
Tiền điều kiện	Sinh viên đã đăng nhập thành công.
Hậu điều kiện	Sinh viên nắm bắt được lộ trình học tập và nội dung lý thuyết của bài học đã chọn.
Luồng chính	1. Sinh viên chọn mục "Danh sách bài học" trên thanh bên. 2. Hệ thống hiển thị: - Bảng thống kê cá nhân (Số bài đã hoàn thành, số bài tập đã làm, trình độ hiện tại, tỉ lệ hoàn thành). - Danh sách các thẻ bài học. 3. Sinh viên sử dụng bộ lọc ngôn ngữ (Python, JavaScript, C++, Java) hoặc chủ đề/độ khó. 4. Hệ thống cập nhật danh sách bài học khớp với tiêu chí lọc. 5. Sinh viên nhấn vào một thẻ bài học cụ thể. 6. Hệ thống hiển thị trang chi tiết bài học: - Nội dung lý thuyết (định dạng Markdown). - Danh sách các bài tập thực hành liên quan (kèm trạng thái đã hoàn thành hay chưa).
Luồng phụ	- 4.a. Không có bài học nào khớp: Hệ thống hiển thị thông báo "Chưa có bài học nào trong danh mục này". - 6.a. Bài học chưa có bài tập: Hiển thị thông báo "Đang cập nhật bài tập".
Ngoại lệ	- 2.a. Lỗi tải dữ liệu: Hệ thống hiển thị các khung trống (skeleton) và thông báo lỗi kết nối.

2.5.2.9. Đặc tả chức năng Làm bài tập**Bảng 2.10.** Bảng đặc tả chức năng Làm bài tập

Mã Use Case	UC03
--------------------	------

Tên Use Case	Làm bài tập
Tác nhân chính	Sinh viên
Mô tả	Sinh viên thực hiện giải quyết các thử thách lập trình thông qua trình soạn thảo code tích hợp, chạy kiểm thử và nhận phản hồi tức thì từ hệ thống.
Tiền điều kiện	Sinh viên đã đăng nhập và chọn một bài tập cụ thể.
Hậu điều kiện	Kết quả làm bài và tiến trình được lưu lại vào hệ thống.
Luồng chính	1. Sinh viên truy cập vào trang chi tiết một bài tập. 2. Hệ thống hiển thị giao diện gồm: - Cột trái: Tiêu đề, mô tả yêu cầu bài tập, kết quả mong đợi và danh sách bộ kiểm thử (test cases). - Cột phải: Trình soạn thảo mã nguồn (đã có sẵn mã khởi tạo) và nút "Nộp bài", bên dưới là Terminal hiển thị kết quả. 3. Sinh viên viết mã nguồn giải quyết vấn đề vào trình soạn thảo. 4. Sinh viên nhấn nút "Nộp bài". 5. Hệ thống thực hiện: 5.1. Gửi mã nguồn về máy chủ để thực thi. 5.2. Chạy mã nguồn đối chiếu với các bộ kiểm thử. 6. Hệ thống hiển thị kết quả dưới terminal: - Kết quả thực thi code. - Thời gian thực thi. - Chỉ số bộ kiểm thử bị sai (nếu có). - Thông báo lỗi chi tiết từ trình biên dịch/thông dịch (nếu có).
Luồng phụ	- 6.1. Sai nhiều lần (2 lần): Hệ thống hiển thị thêm mục "Gợi ý" để giúp sinh viên định hướng. - 6.2. Thất bại liên tiếp (3 lần trở lên): Hệ thống hiển thị đề xuất sinh viên nên quay lại ôn tập bài học lý thuyết tương ứng. - 6.3. Giải thành công: Hệ thống đánh dấu bài tập đã hoàn thành và cập nhật điểm số/tiến độ.
Ngoại lệ	- 5.a. Lỗi thực thi mã (Timeout/Crash): Hệ thống thông báo lỗi runtime và yêu cầu kiểm tra lại tối ưu mã nguồn. - 5.b. Lỗi kết nối máy chủ chấm bài: Thông báo "Nộp bài thất bại, vui lòng thử lại".

2.5.2.10. Đặc tả chức năng Theo dõi tiến độ

Bảng 2.11. Bảng đặc tả chức năng Theo dõi tiến độ

Mã Use Case	UC05
Tên Use Case	Theo dõi tiến độ
Tác nhân chính	Sinh viên
Mô tả	Sinh viên xem tổng quan quá trình học tập của mình thông qua bảng điều khiển trung tâm (Dashboard), bao gồm các chỉ số thống kê và biểu đồ xu hướng.
Tiền điều kiện	Sinh viên đã đăng nhập thành công.
Hậu điều kiện	Sinh viên nắm rõ tình hình học tập và các mảng kiến thức cần cải thiện.
Luồng chính	1. Sinh viên truy cập vào trang "Bảng điều khiển" (Dashboard). 2. Hệ thống truy vấn dữ liệu từ CSDL và hiển thị các khối thông tin: - Thống kê tổng số bài đạt, điểm số từng bài. - Thống kê về điểm mạnh/điểm yếu theo thời gian. - Thống kê số ngày đã làm bài (Streak). 3. Hệ thống hiển thị biểu đồ xu hướng học tập (Chart) thể hiện hoạt động theo thời gian. 4. Hệ thống hiển thị danh sách các bài học/bài tập gần đây nhất mà sinh viên đã tương tác.
Luồng phụ	- 2.a. Người dùng mới (Chưa có dữ liệu): Hệ thống hiển thị các chỉ số bằng 0 và khuyến khích bắt đầu bài học đầu tiên.
Ngoại lệ	- 1.a. Lỗi đồng bộ dữ liệu: Hệ thống thông báo dữ liệu đang được cập nhật và hiển thị giá trị gần nhất được lưu trong bộ nhớ tạm.

2.5.2.11. Đặc tả chức năng Chat với AI Tutor

Bảng 2.12. Bảng đặc tả chức năng Chat với AI Tutor (Sinh viên)

Mã Use Case	UC04
Tên Use Case	Chat với AI Tutor
Tác nhân chính	Sinh viên

Mô tả	Sinh viên thảo luận, hỏi đáp với trợ lý AI để giải quyết các khó khăn trong quá trình học. AI Tutor cá nhân hóa phản hồi dựa trên trình độ và lịch sử lỗi của sinh viên.
Tiền điều kiện	Sinh viên đã đăng nhập. Hệ thống đã thu thập được một số dữ liệu về trình độ (Level) và lỗi thường gặp của sinh viên.
Hậu điều kiện	Cuộc hội thoại được lưu lại và các mảng kiến thức yếu của sinh viên được cập nhật vào hồ sơ phân tích.
Luồng chính	1. Sinh viên mở cửa sổ chat hoặc chọn một cuộc trò chuyện cũ. 2. Sinh viên nhập câu hỏi hoặc yêu cầu hỗ trợ về một đoạn mã nguồn. 3. Hệ thống (AI Tutor) thực hiện: 3.1. Phân tích hồ sơ sinh viên: Trình độ (Sơ cấp/Trung bình/Cao cấp), các ngôn ngữ đang học, các loại lỗi hay mắc phải. 3.2. Áp dụng chiến lược giảng dạy (Socratic): Đưa ra các câu hỏi gợi mở, dùng phép tương đồng, dẫn dắt từng bước nhỏ thay vì đưa ra đáp án trực tiếp. 3.3. Tạo tiêu đề tự động cho cuộc trò chuyện (nếu là tin nhắn đầu tiên). 4. Hệ thống phản hồi dưới dạng dòng chảy dữ liệu (streaming) để sinh viên theo dõi câu trả lời thời gian thực. 5. Sinh viên tiếp tục đặt câu hỏi dựa trên phản hồi của AI cho đến khi hiểu vấn đề. 6. Hệ thống lưu lại toàn bộ lịch sử tin nhắn và cập nhật các thống kê về từ khóa lỗi vào CSDL phân tích.
Luồng phụ	- 3.1.a. SV Sơ cấp: AI dùng ngôn ngữ đơn giản, ví dụ gần gũi, khích lệ nhiều. - 3.1.b. SV Cao cấp: AI phản hồi súc tích, yêu cầu SV phân tích các trường hợp biên và tối ưu hóa.
Ngoại lệ	- 4.a. Lỗi kết nối dịch vụ AI: Hệ thống thông báo "AI Tutor đang bận, vui lòng thử lại sau ít phút". - 4.b. Tin nhắn quá dài: Hệ thống yêu cầu sinh viên rút ngắn câu hỏi để đảm bảo chất lượng phản hồi.

2.5.2.12. Đặc tả chức năng Cập nhật Knowledge log

Bảng 2.13. Bảng đặc tả chức năng Cập nhật Knowledge log

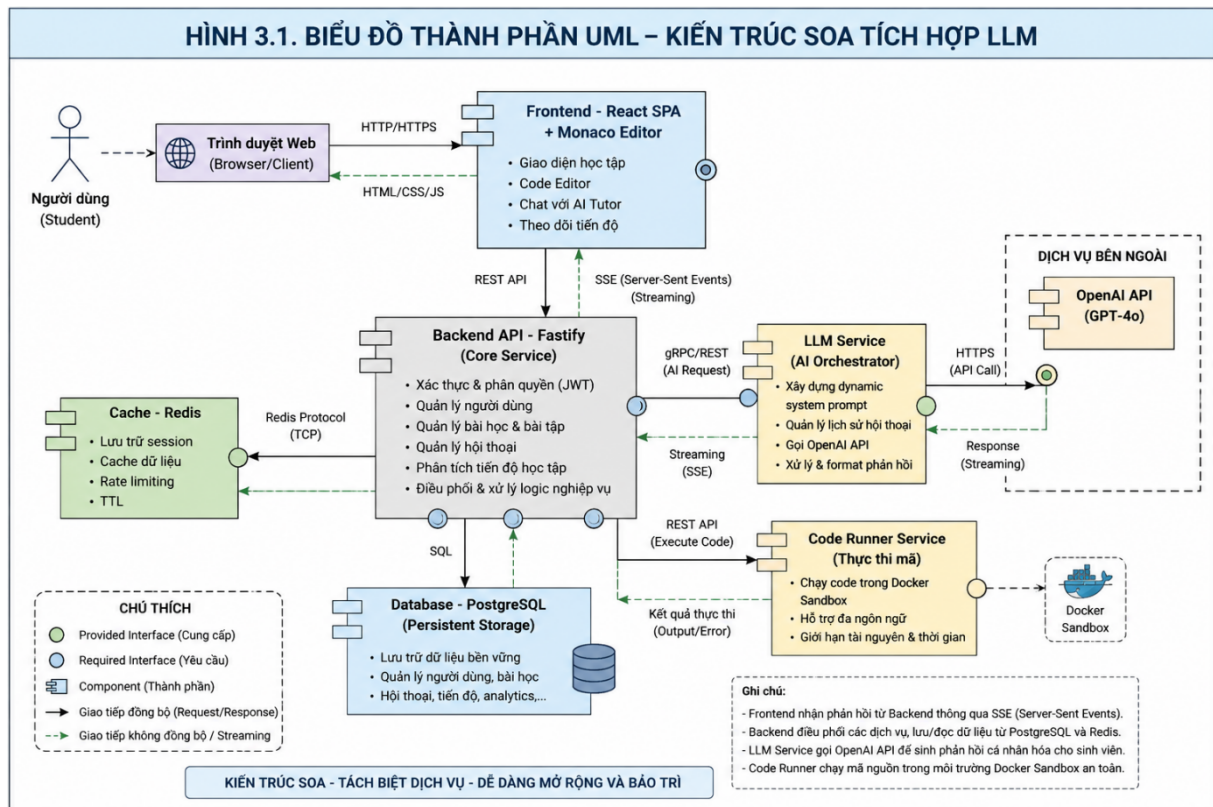
Mã Use Case	UC07
Tên Use Case	Cập nhật Knowledge log
Tác nhân chính	Hệ thống
Mô tả	Tự động ghi lại và phân tích mọi hoạt động học tập của sinh viên để cập nhật hồ sơ năng lực và điều chỉnh trình độ học tập tương ứng.
Tiền điều kiện	Sinh viên thực hiện các tương tác (Nộp bài tập, Chat với AI) có kết quả hợp lệ hoặc lỗi.
Hậu điều kiện	Hồ sơ năng lực (Analytics) của sinh viên được cập nhật dữ liệu mới nhất.
Luồng chính	1. Hệ thống tiếp nhận sự kiện sau mỗi lượt tương tác: - Trường hợp Nộp bài Đạt: Hệ thống ghi nhận chủ đề bài học vừa hoàn thành vào danh sách "Kiến thức đã nắm vững". Đánh dấu trạng thái "Hoàn thành" cho bài tập đó trong tiến trình học tập. - Trường hợp Nộp bài Lỗi/Sai: Hệ thống phân tích loại lỗi (Cú pháp, Logic, Runtime) và tăng bộ đếm tần suất cho loại lỗi đó trong hồ sơ. 2. Hệ thống kiểm tra điều kiện nâng cấp trình độ (Level Up): - Tính toán tỷ lệ vượt qua bài tập (Pass Rate) dựa trên ít nhất 5 bài nộp gần nhất. - Nếu tỷ lệ đạt > 80%, hệ thống tự động cập nhật "Ước tính trình độ" mới (ví dụ từ Sơ cấp lên Trung cấp). 3. Hệ thống đồng bộ hóa các từ khóa quan trọng từ cuộc hội thoại với AI vào hồ sơ phân tích mảng kiến thức yếu/mạnh.
Luồng phụ	- 2.a. Chưa đủ số lượng bài nộp: Hệ thống tiếp tục tích lũy dữ liệu mà không thực hiện thay đổi trình độ.
Ngoại lệ	- 1.a. Lỗi cập nhật CSDL Analytics: Hệ thống thử lại sau hoặc ghi vào nhật ký lỗi chờ xử lý hậu kỳ.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể

3.1.1. Tổng quan kiến trúc

Hệ thống PPAT được xây dựng theo kiến trúc hướng dịch vụ (Service-Oriented Architecture - SOA), trong đó mỗi thành phần đảm nhận một trách nhiệm riêng biệt và giao tiếp với nhau qua các giao thức được định nghĩa rõ ràng. Cách tổ chức này giúp các thành phần có thể phát triển, nâng cấp và mở rộng độc lập mà không ảnh hưởng đến phần còn lại của hệ thống.



Hình 3.1. Biểu đồ thành phần kiến trúc SOA

Như thể hiện trong Hình 3.1, toàn bộ hệ thống gồm năm thành phần chính: Frontend, Backend API, LLM Service, Code Runner Service và tầng dữ liệu (PostgreSQL + Redis).

Frontend được xây dựng bằng Next.js tích hợp Monaco Editor, là điểm tiếp xúc trực tiếp với người dùng. Trình duyệt giao tiếp với Frontend qua HTTP/HTTPS và nhận về HTML/CSS/JS để render giao diện. Frontend không xử lý logic nghiệp vụ mà chỉ đảm nhận việc hiển thị và thu thập tương tác từ sinh viên, bao gồm giao diện học bài, code editor, chat với AI Tutor và trang theo dõi tiến độ.

Backend API sử dụng Fastify đóng vai trò trung tâm điều phối toàn bộ hệ thống.

Đây là nơi xử lý xác thực JWT, quản lý người dùng, bài học, bài tập và hội thoại. Mọi request từ Frontend đều đi qua Backend qua REST API. Backend cũng là nơi duy nhất kết nối trực tiếp với cả PostgreSQL, Redis, LLM Service và Code Runner Service đảm bảo logic nghiệp vụ được tập trung, không phân tán.

LLM Service hoạt động như một AI Orchestrator độc lập. Khi nhận yêu cầu từ Backend, service này truy vấn knowledge log của sinh viên, xây dựng dynamic system prompt phù hợp với từng cá nhân, sau đó gọi OpenAI API (GPT-4o) qua HTTPS. Phản hồi từ GPT-4o được stream ngược về Backend rồi tiếp tục forward về Frontend thông qua Server-Sent Events (SSE) đây là giao tiếp không đồng bộ duy nhất trong hệ thống, được biểu diễn bằng mũi tên nét đứt màu xanh trong Hình 3.1.

Code Runner Service là thành phần xử lý việc thực thi code của sinh viên. Khi sinh viên nhấn chạy code, Backend gửi yêu cầu đến service này qua REST API. Code được chạy bên trong một Docker Sandbox hoàn toàn cô lập không có quyền truy cập mạng, giới hạn tài nguyên CPU/RAM và timeout cứng nhằm đảm bảo an toàn cho server. Kết quả thực thi (output hoặc error) được trả về Backend rồi hiển thị cho sinh viên.

Tầng dữ liệu gồm PostgreSQL đảm nhận lưu trữ bền vững toàn bộ dữ liệu nghiệp vụ (người dùng, bài học, hội thoại, tiến độ, analytics) và Redis đảm nhận cache, session và rate limiting. Backend giao tiếp với PostgreSQL qua SQL và với Redis qua Redis Protocol (TCP).

3.1.2. *Luồng xử lý theo loại request*

Tùy vào loại tương tác của sinh viên, hệ thống xử lý theo các luồng khác nhau:

Bảng 3.1. Ví dụ luồng xử lý theo loại request

Loại request	Luồng xử lý
Đăng nhập	Frontend → Backend → PostgreSQL (xác thực) → Redis (lưu session) → trả JWT
Xem bài học	Frontend → Backend → Redis (cache check) → PostgreSQL (nếu cache miss) → trả dữ liệu
Chat AI	Frontend → Backend → LLM Service → OpenAI API → stream SSE ngược về Frontend
Chạy code	Frontend → Backend → Code Runner → Docker Sandbox → trả output/error
Xem tiến độ	Frontend → Backend → Redis (cache) → PostgreSQL (aggregate query) → trả dashboard

Thiết kế theo luồng này đảm bảo các request thông thường (xem bài học, xem tiến độ) được phục vụ nhanh nhờ Redis cache, trong khi các request nặng hơn (chat AI, chạy code) được tách ra xử lý ở service riêng mà không ảnh hưởng đến hiệu năng tổng thể của Backend.

3.2. Thiết kế API

Hệ thống cung cấp 14 endpoint REST API, được nhóm theo chức năng. Toàn bộ các endpoint yêu cầu xác thực đều kiểm tra JWT token trong header Authorization: Bearer <token>.

Bảng 3.2. Danh sách endpoint REST API

Method	Endpoint	Phân quyền	Mô tả
POST	/api/auth/register	Public	Đăng ký tài khoản mới
POST	/api/auth/login	Public	Đăng nhập, trả về JWT và refresh token
GET	/api/lessons	Student	Danh sách bài học, hỗ trợ filter theo ngôn ngữ, chủ đề, độ khó
GET	/api/lessons/:id	Student	Chi tiết một bài học
GET	/api/exercises/:id	Student	Chi tiết bài tập kèm starter code
POST	/api/exercises/:id/submit	Student	Nộp code, chạy test cases, trả kết quả pass/fail/error
GET	/api/conversations	Student	Danh sách phiên hội thoại của sinh viên
POST	/api/conversations	Student	Tạo phiên hội thoại mới
POST	/api/conversations/:id/messages	Student	Gửi tin nhắn, nhận phản hồi AI qua SSE stream
GET	/api/progress/me	Student	Dashboard tiến độ cá nhân
GET	/api/analytics/me	Student	Knowledge log, điểm mạnh và điểm yếu

GET	/api/admin/lessons	Admin	Danh sách bài học phía quản trị
POST	/api/admin/lessons	Admin	Tạo bài học mới
GET	/api/admin/reports	Admin	Báo cáo tổng hợp toàn bộ sinh viên

Endpoint quan trọng nhất về mặt kỹ thuật là POST /api/conversations/:id/messages. Khác với các endpoint thông thường trả về JSON, endpoint này trả về response theo định dạng text/event-stream, tức là giữ kết nối mở và đẩy từng token từ GPT-4o về client theo thời gian thực thay vì chờ sinh xong toàn bộ câu trả lời. Cơ chế streaming này được mô tả chi tiết hơn ở mục 3.4.4.

3.3. Thiết kế cơ chế Cá nhân hóa AI

3.3.1. Pipeline xử lý tin nhắn (Message Lifecycle)

Đây là phần kỹ thuật cốt lõi tạo ra sự khác biệt so với việc dùng ChatGPT thông thường. Khi sinh viên gửi một tin nhắn, hệ thống không chuyển thẳng nội dung đó đến GPT-4o mà thực hiện một pipeline 5 bước như mô tả trong Hình 3.3.

Bước 1 - Sinh viên gửi tin nhắn: Frontend gửi POST request đến /api/conversations/:id/messages và đồng thời mở kết nối EventSource để lắng nghe SSE stream. Ở bước này, ba sinh viên với ba trình độ khác nhau gửi cùng một câu hỏi sự khác biệt trong phản hồi bắt đầu từ bước tiếp theo.

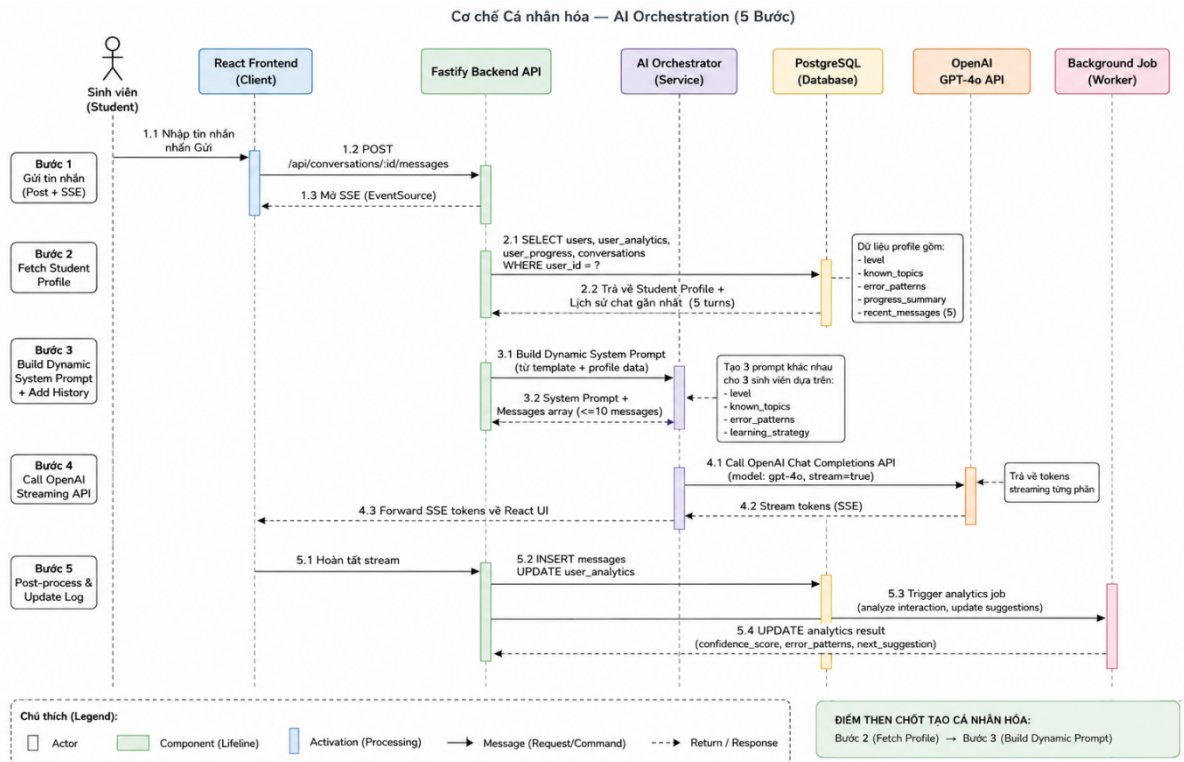
Bước 2 - Fetch Student Profile: Backend truy vấn PostgreSQL để lấy toàn bộ thông tin học tập của sinh viên, bao gồm trình độ hiện tại, danh sách chủ đề đã nắm, các lỗi thường gặp từ user_analytics và lịch sử 10 tin nhắn gần nhất. Đây là bước hoàn toàn không có trong ChatGPT thông thường mỗi sinh viên trả về một tập dữ liệu khác nhau dù cùng gửi một câu hỏi.

Bước 3 - Build Dynamic System Prompt: AI Orchestrator nhận dữ liệu từ bước 2 và điền vào template system prompt có sẵn. Kết quả là ba sinh viên khác nhau tạo ra ba system prompt hoàn toàn khác nhau trước khi gửi đến GPT-4o. Ví dụ với sinh viên trình độ intermediate hay quên base case, system prompt sẽ hướng AI dùng chiến lược bridge từ vòng lặp sang đệ quy và chủ động nhắc về điều kiện dừng.

Bước 4 - Gọi GPT-4o với stream: Backend gọi OpenAI SDK với tham số stream: true. Từng token được GPT-4o sinh ra được forward ngay về Frontend qua Server-Sent Events thay vì chờ đến khi hoàn thành toàn bộ câu trả lời. Sinh viên thấy phản hồi xuất hiện dần dần trên màn hình, tạo cảm giác tương tác tự nhiên như đang chat với người thật.

Bước 5 - Lưu message và cập nhật log: Sau khi stream kết thúc, một background

job được kích hoạt để INSERT tin nhắn vào bảng messages và UPDATE user_analytics cập nhật confidence score, ghi nhận lỗi mới nếu có, và tính toán gợi ý bài học tiếp theo. Bước này chạy bất đồng bộ nên không làm chậm trải nghiệm của sinh viên.



Hình 3.2. Pipeline xử lý tin nhắn (Message Lifecycle)

3.3.2. Dynamic System Prompt

Cơ chế dynamic system prompt là yếu tố kỹ thuật trực tiếp tạo ra sự cá nhân hóa trong phản hồi của AI. Thay vì dùng một system prompt cố định cho mọi người dùng, hệ thống xây dựng prompt riêng cho từng sinh viên trước mỗi lần gọi GPT-4o.

Template prompt được thiết kế gồm hai phần. Phần đầu là profile data các placeholder được điền động từ database:

```

You are a personalized programming tutor AI.
Here is the student's profile:
Name: {student_name}
Current level: {level} (beginner / intermediate / advanced)
Languages learning: {languages}
Topics already mastered: {known_topics}
Common mistakes: {error_patterns}
Lessons completed: {completed_lessons}/{total_lessons}
Recent interaction summary: {recent_summary}
  
```

Các placeholder này được lấy từ các bảng khác nhau trong database: student_name và level từ bảng users, known_topics và error_patterns từ cột JSONB trong userAnalytics,

completed_lessons tổng hợp từ userProgress, còn recent_summary là tóm tắt từ 10 tin nhắn gần nhất trong messages.

Phần thứ hai là teaching guidelines bộ quy tắc cố định áp dụng cho mọi sinh viên, định nghĩa hành vi của AI Tutor:

Teaching guidelines:

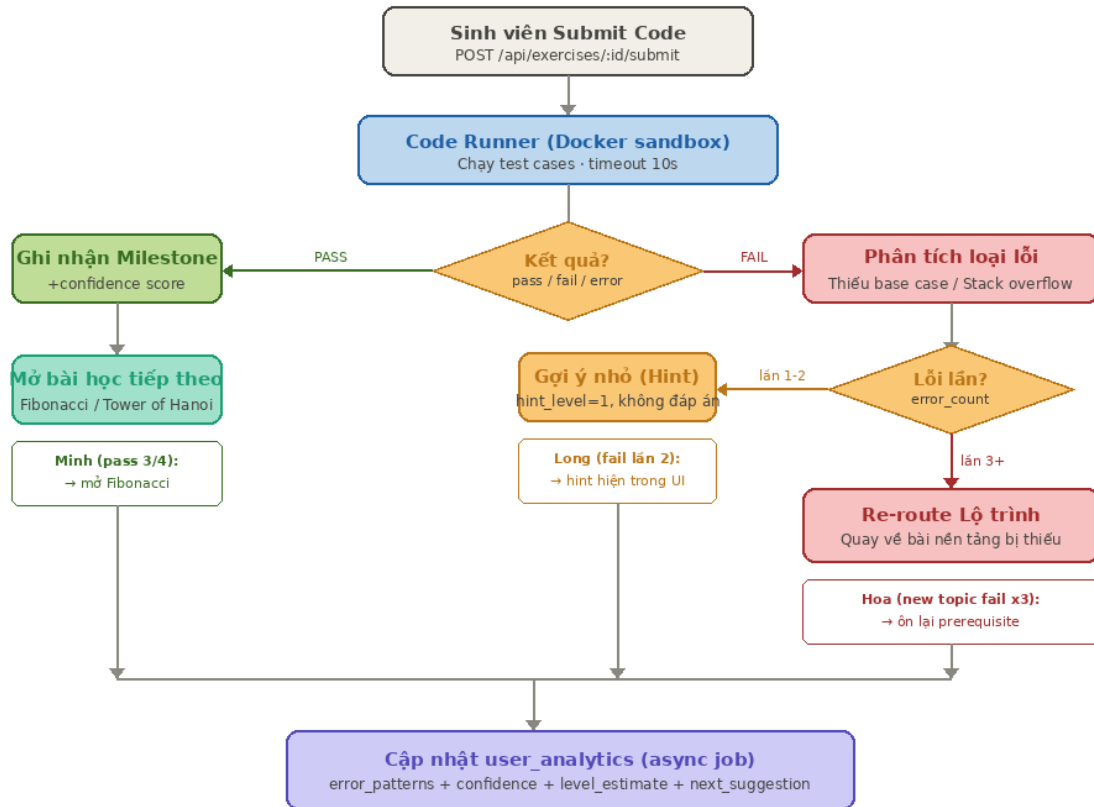
1. NEVER give the full solution directly. Use the Socratic method - ask
→ guiding questions.
2. Adjust explanation depth to the student's level.
3. If the student hasn't learned a prerequisite, gently redirect to that
→ first.
4. For intermediate/advanced students, suggest edge cases, best
→ practices, or comparisons.
5. Keep responses concise. Use code blocks for examples.
6. Acknowledge when the student does something right - build confidence.
7. If the student is stuck after 2 attempts, provide a small hint, not
→ the answer.

Bảy quy tắc này phản ánh trực tiếp các nguyên tắc sư phạm đã trình bày ở Chương 2: không đưa đáp án trực tiếp (quy tắc 1), điều chỉnh theo trình độ (quy tắc 2), phát hiện kiến thức tiên quyết còn thiếu (quy tắc 3), và chỉ gợi ý nhỏ sau hai lần thất bại (quy tắc 7). Nhờ phần profile data thay đổi theo từng sinh viên, cùng một bộ quy tắc này cho ra hành vi hoàn toàn khác nhau sinh viên beginner được giải thích bằng analogy đơn giản, sinh viên advanced được thách thức thêm với edge case và so sánh độ phức tạp thuật toán.

3.3.3. *Adaptive Learning Path*

Hệ thống không chỉ cá nhân hóa ở phần chat mà còn tự động điều chỉnh lộ trình học tập dựa trên kết quả làm bài tập của sinh viên. Mỗi lần nộp bài, thay vì chỉ trả về pass/fail đơn thuần, hệ thống phân nhánh xử lý theo 5 trường hợp.

Flow 2 — Tự động điều chỉnh lộ trình học (Adaptive Learning Path)



* Mọi nhánh đều hội tụ về cập nhật knowledge log — hệ thống học từ từng interaction

Hình 3.3. Adaptive Learning Path

Khi sinh viên pass toàn bộ test case, hệ thống ghi nhận milestone vào userProgress, cập nhật knownTopics trong userAnalytics và mở khóa bài tập tiếp theo. AI Tutor sẽ gửi lời chúc mừng ngắn và gợi ý bài học tiếp theo phù hợp với lộ trình.

Khi fail lần đầu, hệ thống ghi nhận loại lỗi vào errorPatterns nhưng không hiển thị gì thêm trên giao diện. Thông tin này được dùng ở phiên chat tiếp theo AI sẽ chủ động nhắc về lỗi đó mà sinh viên không cần phải hỏi lại.

Khi fail lần hai, một gợi ý nhỏ xuất hiện ngay trong giao diện code editor. Gợi ý này chỉ hướng đến điểm sai cụ thể chứ không đưa ra đáp án ví dụ "kiểm tra lại điều kiện dừng của vòng lặp" thay vì viết sẵn code đúng.

Khi fail từ lần ba trở đi, hệ thống tra cứu prerequisiteLessonId của bài tập hiện tại để xác định kiến thức tiên quyết còn thiếu. Thay vì để sinh viên tiếp tục loay hoay, AI Tutor đề xuất ôn lại bài học nền tảng trước và giải thích nhẹ nhàng lý do cần làm vậy.

Khi xảy ra runtime error, hệ thống lưu lại stack trace và chuyển thông tin này vào context của AI Tutor. Thay vì đưa ra fix trực tiếp, AI hướng dẫn sinh viên cách đọc thông

báo lỗi và tự xác định nguyên nhân đây là kỹ năng debug thực tế quan trọng hơn việc biết đáp án của một bài cụ thể.

Toàn bộ các nhánh xử lý trên đều hội tụ về một điểm chung là cập nhật userAnalytics, đảm bảo knowledge log luôn phản ánh trạng thái học tập mới nhất của sinh viên và system prompt ở lần chat tiếp theo sẽ chính xác hơn.

3.3.4. *Streaming Architecture (SSE)*

Thay vì chờ GPT-4o sinh xong toàn bộ câu trả lời rồi mới trả về, hệ thống sử dụng Server-Sent Events (SSE) để forward từng token về phía client ngay khi có. Cách tiếp cận này giúp sinh viên thấy phản hồi xuất hiện dần dần theo thời gian thực, tạo trải nghiệm tự nhiên hơn so với chờ đợi màn hình trắng.

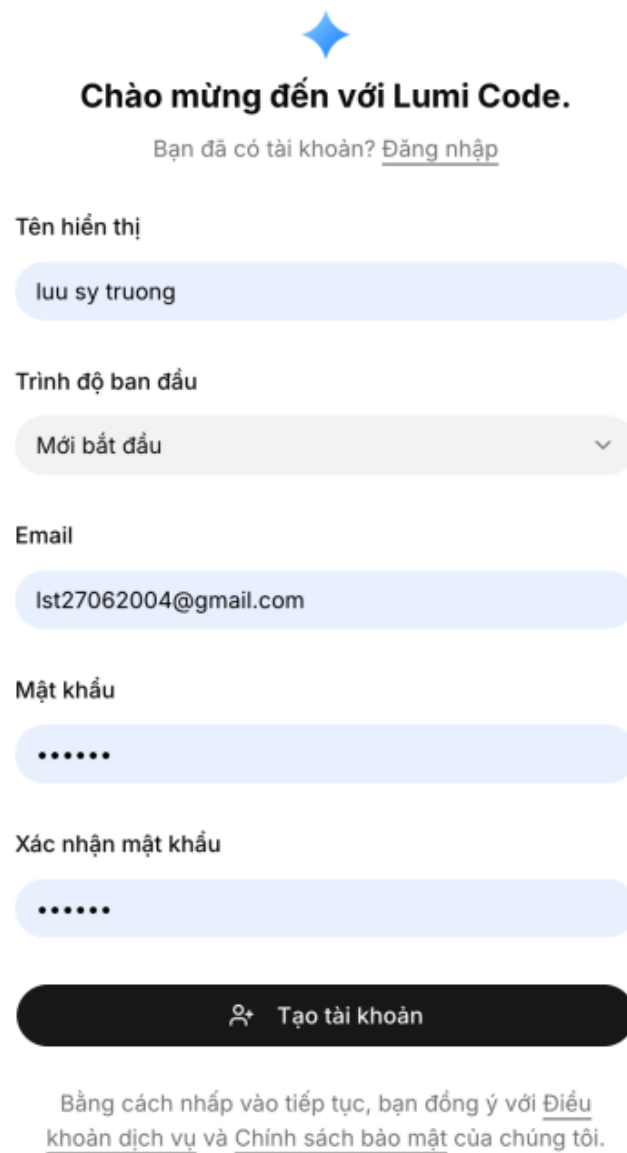
Luồng xử lý diễn ra theo thứ tự sau. Khi sinh viên gửi tin nhắn, Frontend gọi POST request đến endpoint chat và ngay lập tức mở một EventSource connection để lắng nghe stream. Phía Backend, sau khi hoàn thành bước build dynamic system prompt, gọi OpenAI SDK với stream: true và set header response là Content-Type: text/event-stream để giữ kết nối mở. Mỗi token được GPT-4o trả về, Backend forward ngay qua SSE mà không buffer lại. Frontend lắng nghe từng SSE event, lấy nội dung token và append vào chat bubble hiện tại theo từng ký tự.

Khi GPT-4o gửi tín hiệu kết thúc [DONE], Backend đóng SSE connection và kích hoạt bất đồng bộ hai tác vụ: INSERT toàn bộ nội dung tin nhắn vào bảng messages và UPDATE userAnalytics. Việc tách phần lưu dữ liệu ra khỏi luồng streaming là có chủ ý sinh viên nhận được token đầu tiên trong vòng vài trăm millisecond mà không bị block bởi các tác vụ ghi database phía sau.

Một điểm cần lưu ý là SSE là giao thức một chiều từ server về client, khác với WebSocket hai chiều. Điều này phù hợp với bài toán chat AI vì trong khi AI đang trả lời, sinh viên không cần gửi thêm dữ liệu chỉ cần nhận stream. Khi sinh viên muốn gửi tin nhắn mới, một POST request hoàn toàn mới được khởi tạo và một SSE stream mới được mở.

3.4. Thiết kế chi tiết

3.4.1. Đăng ký



The registration form for Lumi Code features a blue star icon at the top. Below the title, there is a link for existing users. The form includes input fields for display name, initial level, email, and password, each with a light blue background. A dark blue button with a user icon is used for submission. At the bottom, a disclaimer states that by registering, the user agrees to the terms of service and privacy policy.

Chào mừng đến với Lumi Code.

Bạn đã có tài khoản? [Đăng nhập](#)

Tên hiển thị

luu sy truong

Trình độ ban đầu

Mới bắt đầu

Email

lst27062004@gmail.com

Mật khẩu

.....

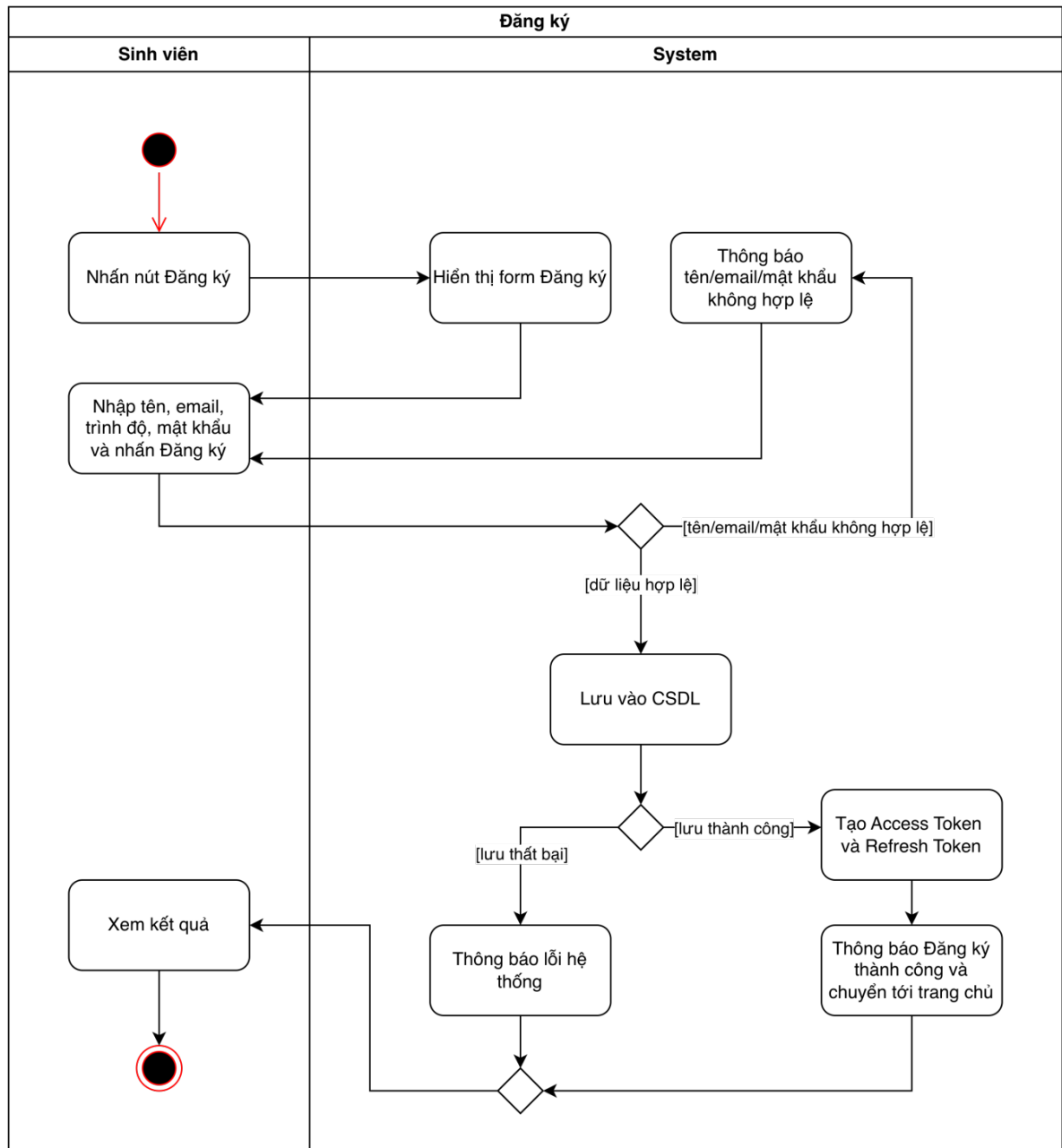
Xác nhận mật khẩu

.....

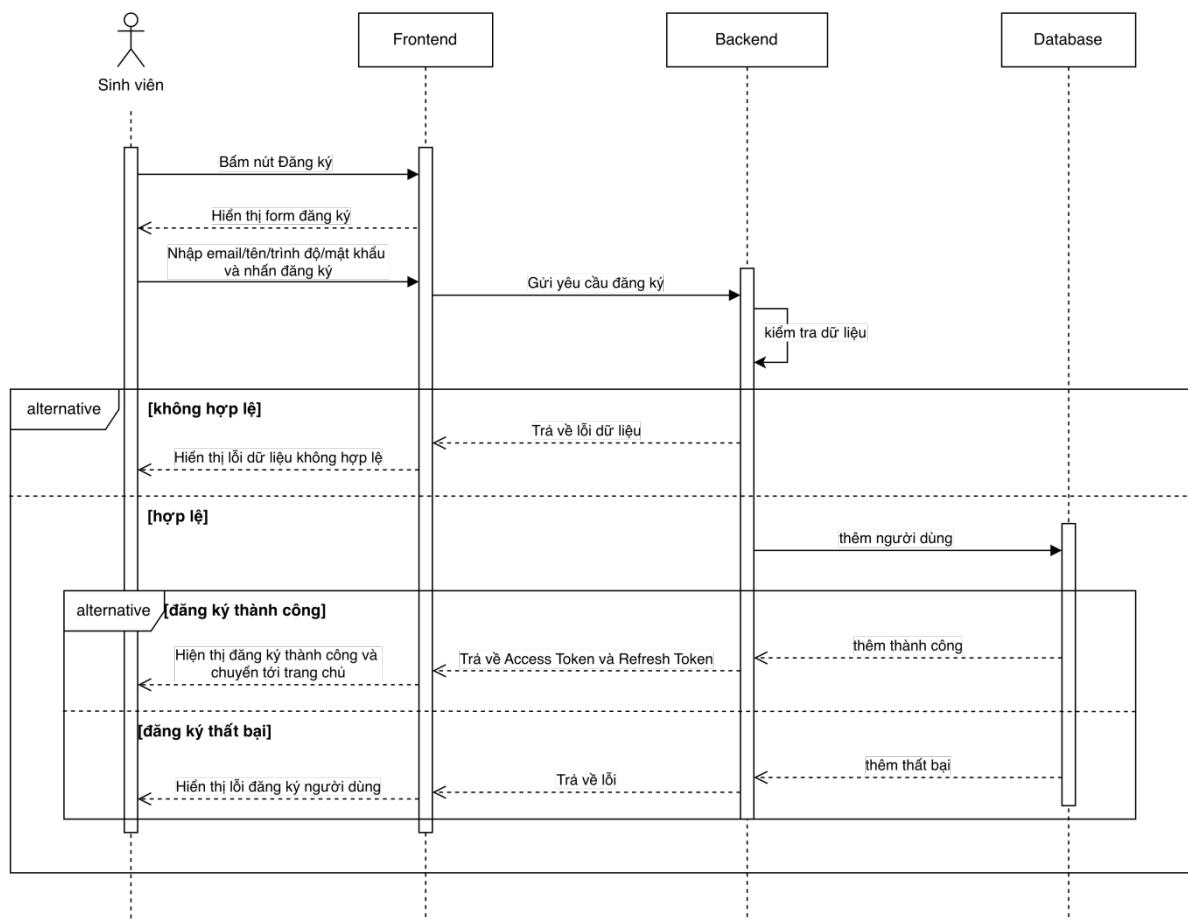
 **Tạo tài khoản**

Bằng cách nhấp vào tiếp tục, bạn đồng ý với [Điều khoản dịch vụ](#) và [Chính sách bảo mật](#) của chúng tôi.

Hình 3.4. Giao diện chức năng đăng ký

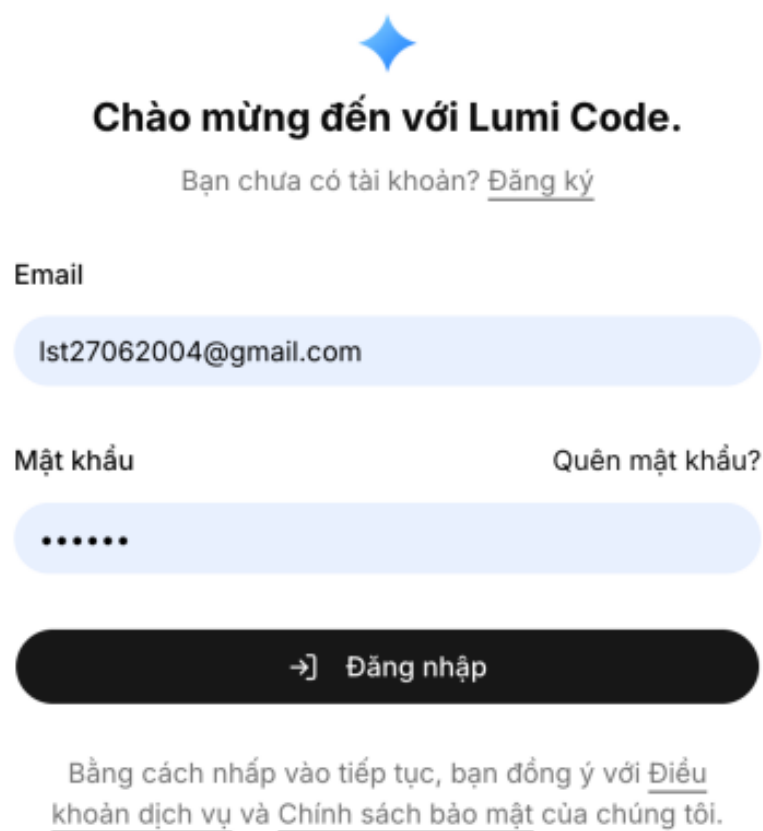


Hình 3.5. Sơ đồ hoạt động chức năng Đăng ký



Hình 3.6. Sơ đồ trình tự chức năng Đăng ký

3.4.2. Đăng nhập



The image shows a login interface for 'Lumi Code'. At the top, there is a blue four-pointed star icon. Below it, the text 'Chào mừng đến với Lumi Code.' is displayed in a bold, black font. Underneath this, a smaller line of text asks 'Bạn chưa có tài khoản?' followed by a link 'Đăng ký'. The form consists of two input fields: 'Email' and 'Mật khẩu'. The 'Email' field contains the text 'lst27062004@gmail.com'. The 'Mật khẩu' field is masked with six dots. To the right of the password field is a link 'Quên mật khẩu?'. Below the input fields is a large, dark blue button with the text '→] Đăng nhập'. At the bottom, there is a line of text: 'Bằng cách nhấp vào tiếp tục, bạn đồng ý với Điều khoản dịch vụ và Chính sách bảo mật của chúng tôi.'

Chào mừng đến với Lumi Code.

Bạn chưa có tài khoản? [Đăng ký](#)

Email

lst27062004@gmail.com

Mật khẩu

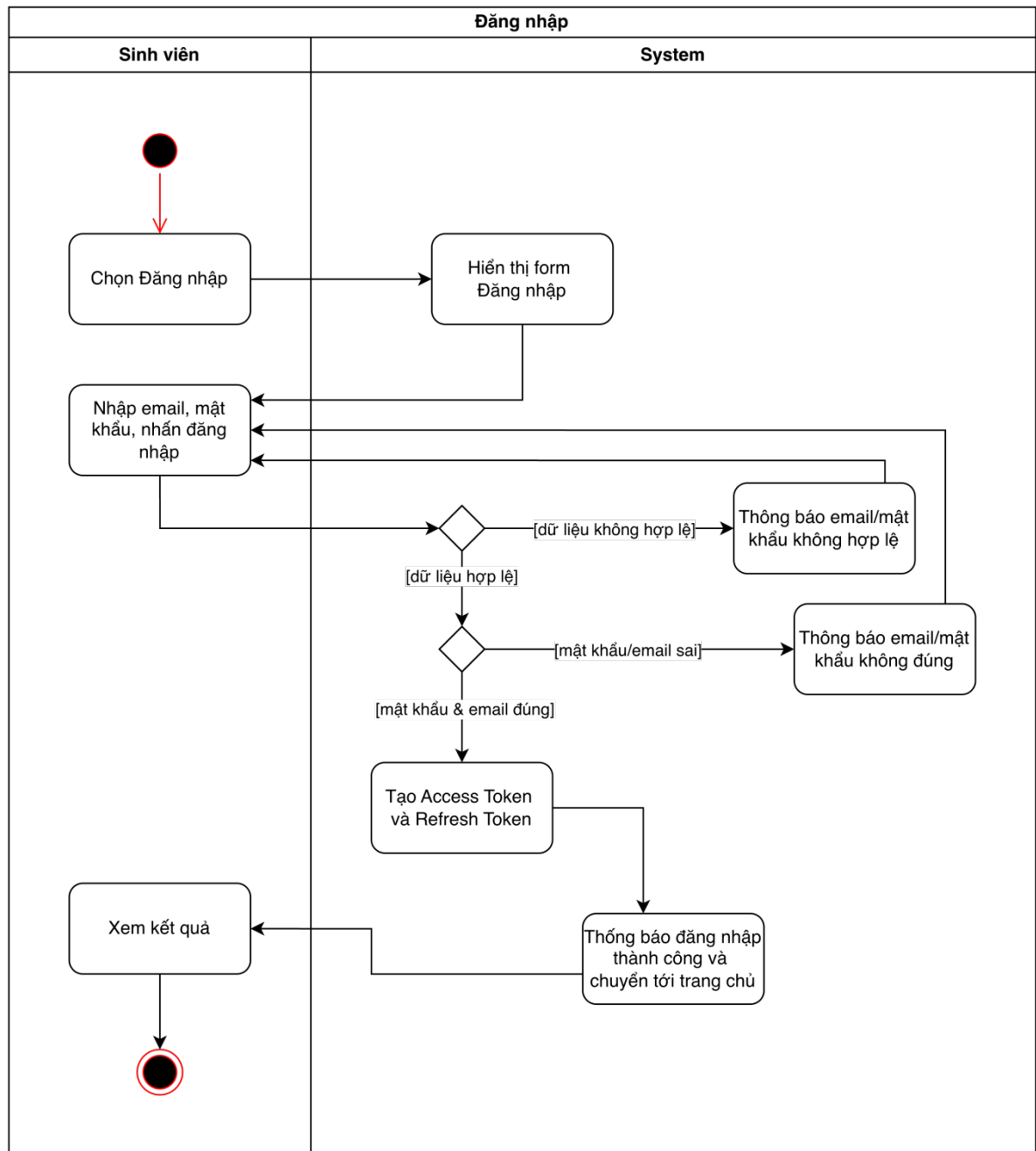
Quên mật khẩu?

.....

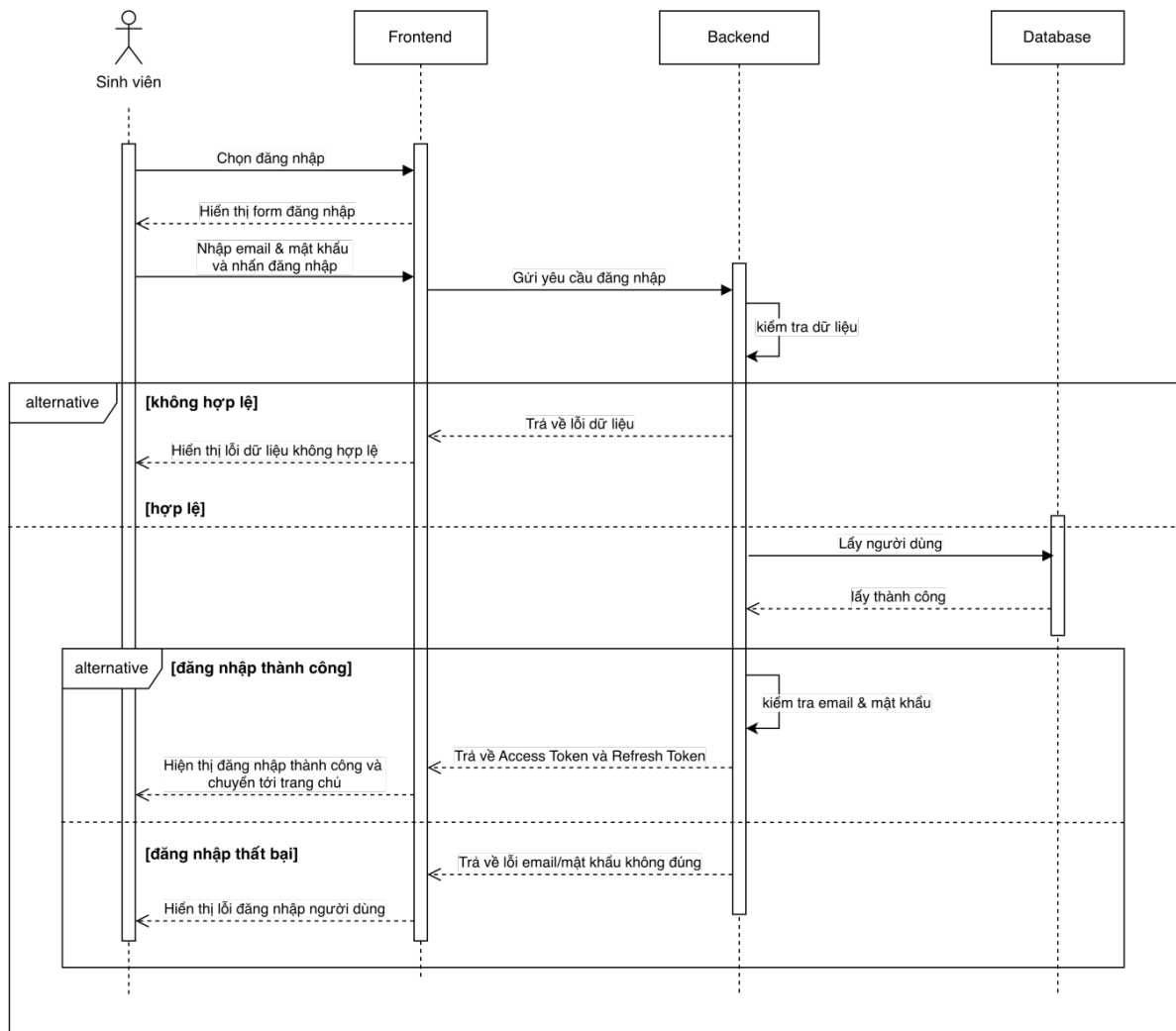
→] Đăng nhập

Bằng cách nhấp vào tiếp tục, bạn đồng ý với [Điều khoản dịch vụ](#) và [Chính sách bảo mật](#) của chúng tôi.

Hình 3.7. Giao diện chức năng Đăng nhập

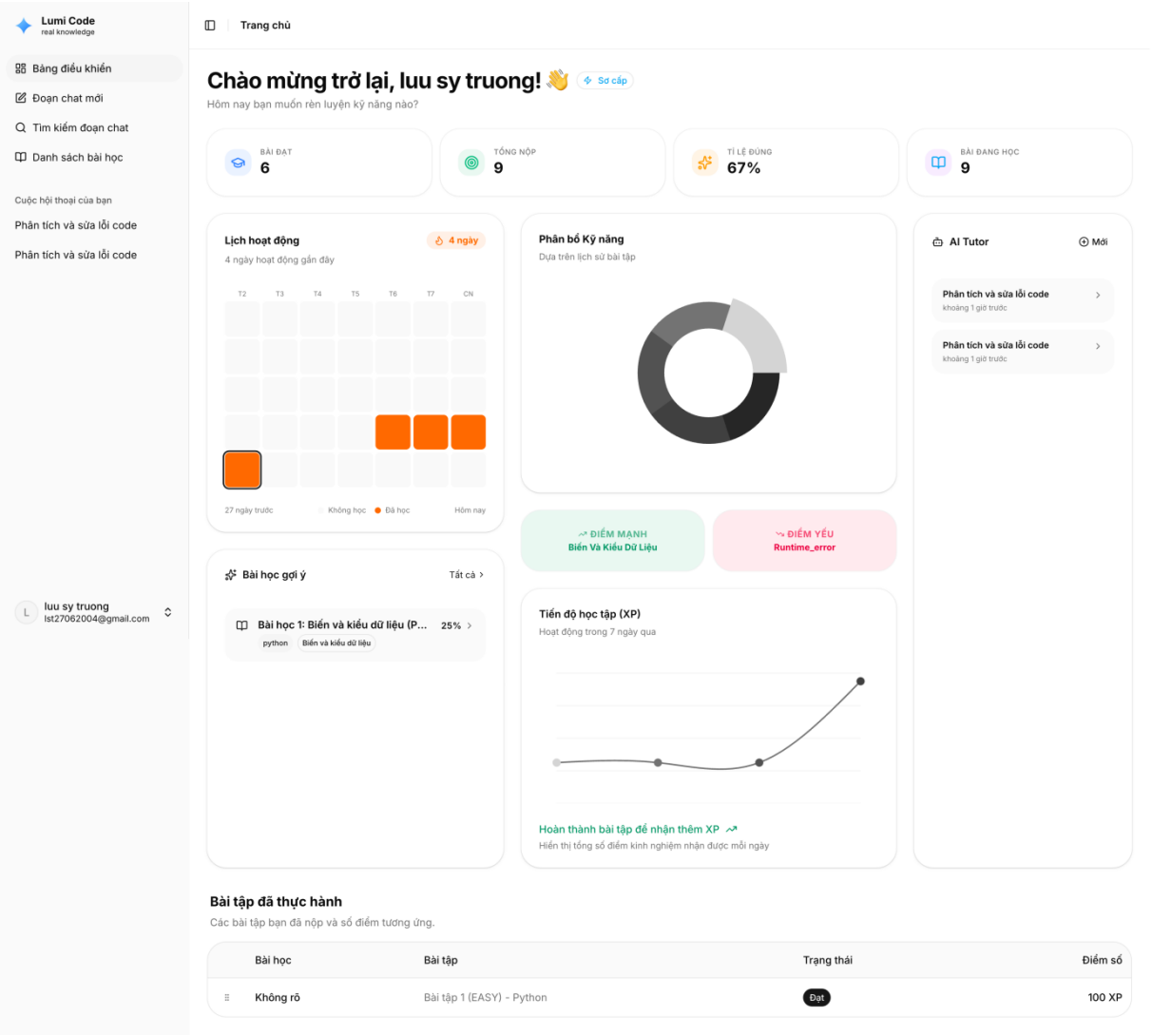


Hình 3.8. Sơ đồ hoạt động chức năng Đăng nhập

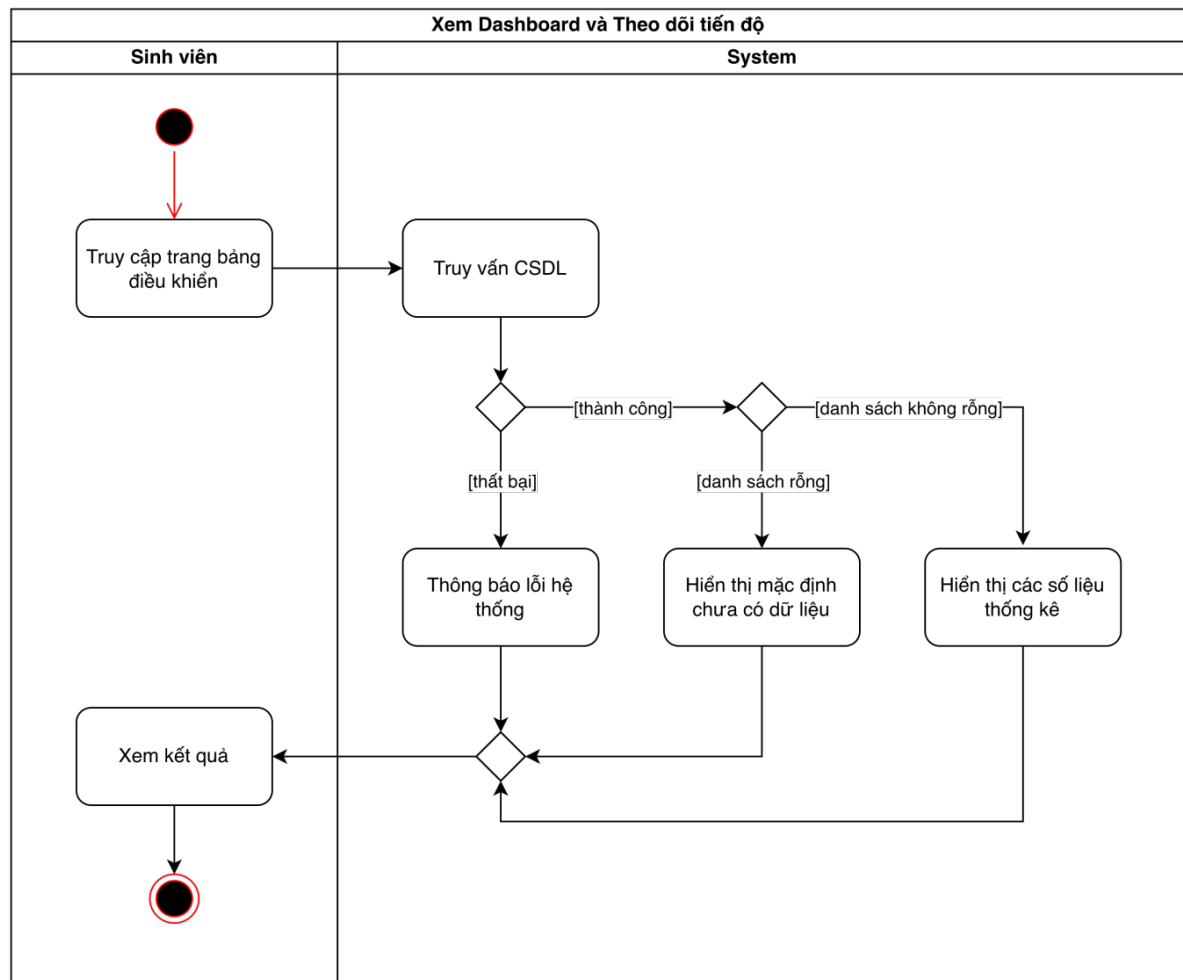


Hình 3.9. Sơ đồ trình tự chức năng Đăng nhập

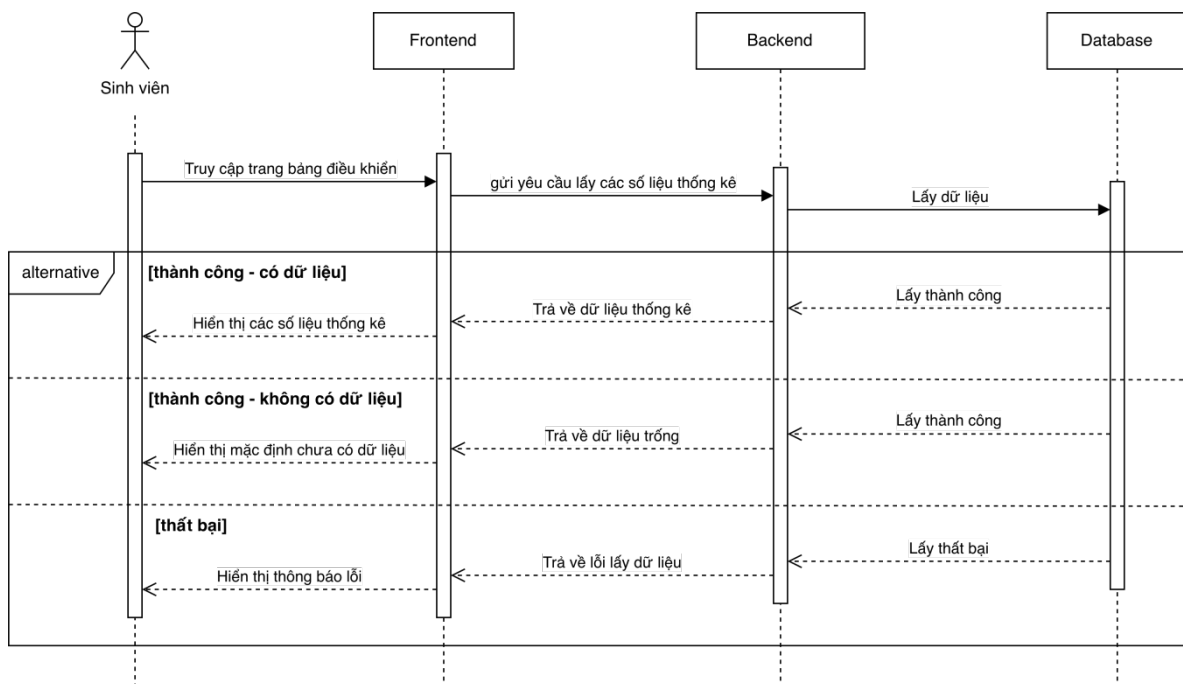
3.4.3. Dashboard kết hợp Theo dõi tiến độ



Hình 3.10. Giao diện Dashboard kết hợp theo dõi tiến độ

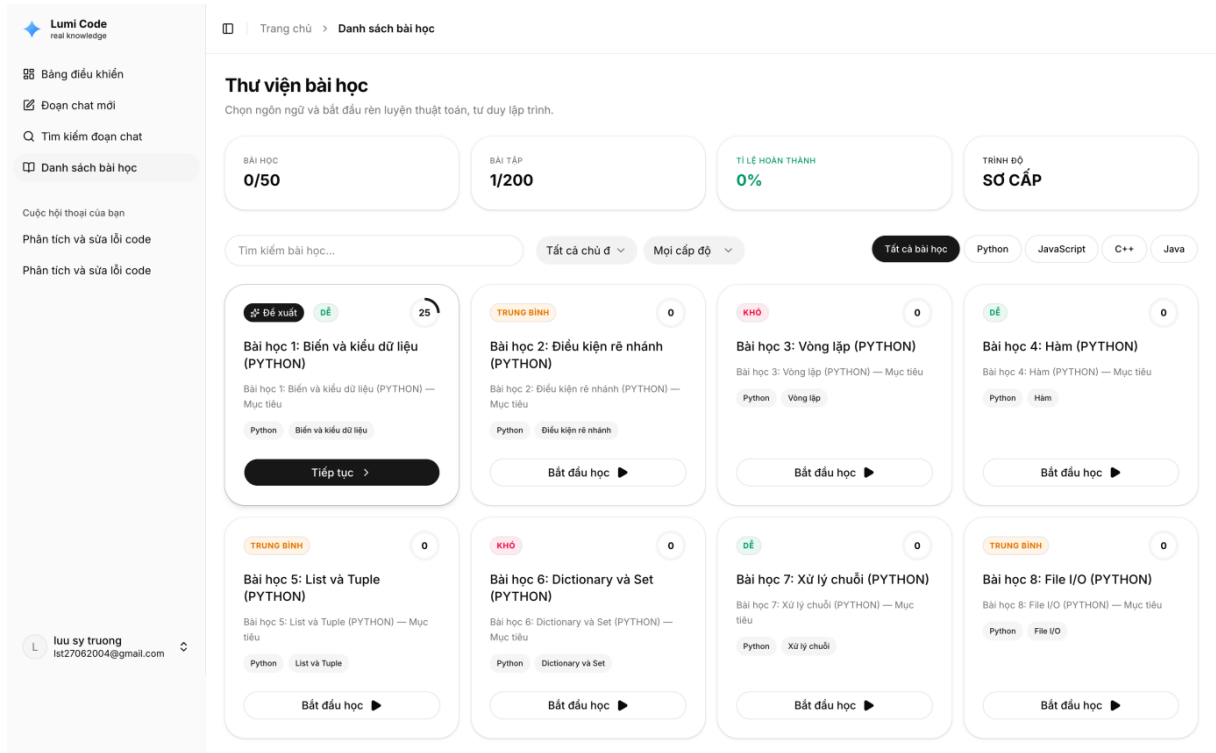


Hình 3.11. Sơ đồ hoạt động chức năng xem Dashboard kết hợp theo dõi tiến độ

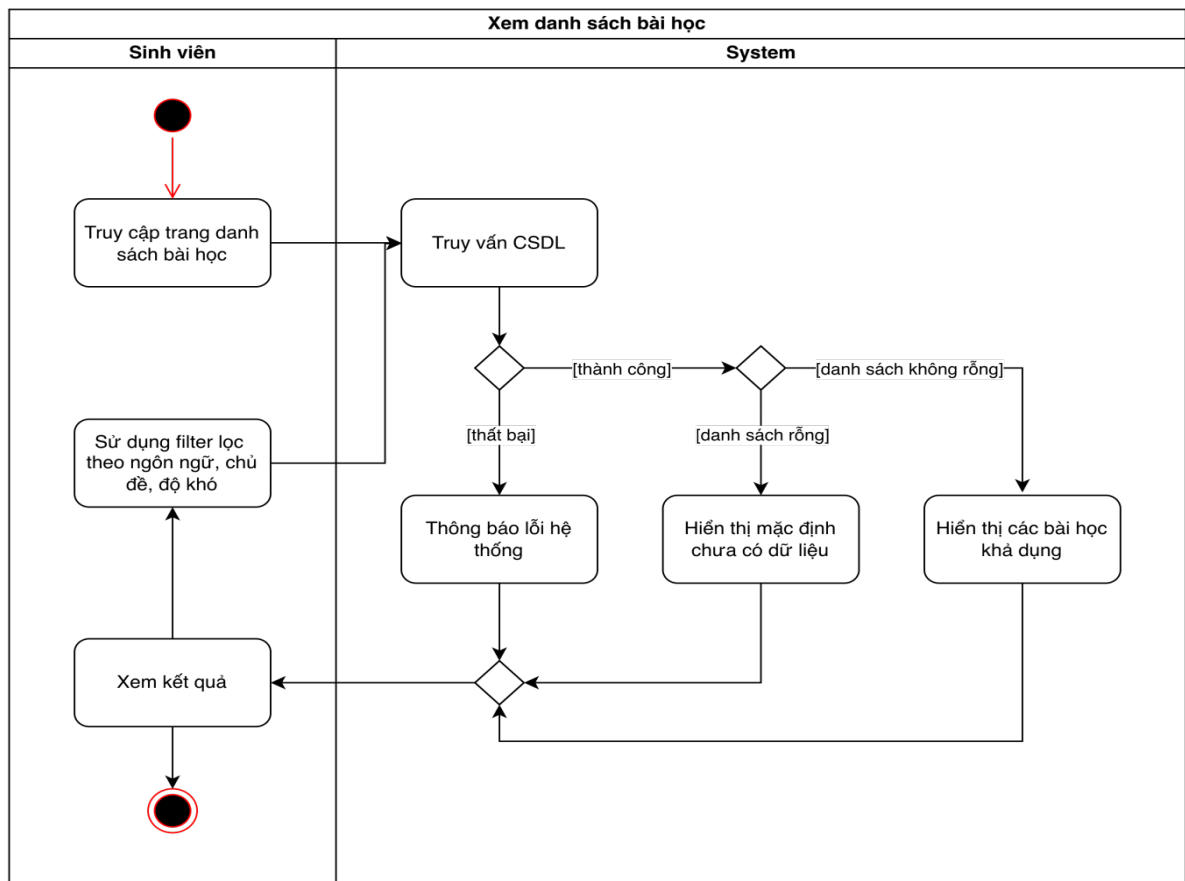


Hình 3.12. Sơ đồ trình tự chức năng xem Dashboard kết hợp theo dõi tiến độ

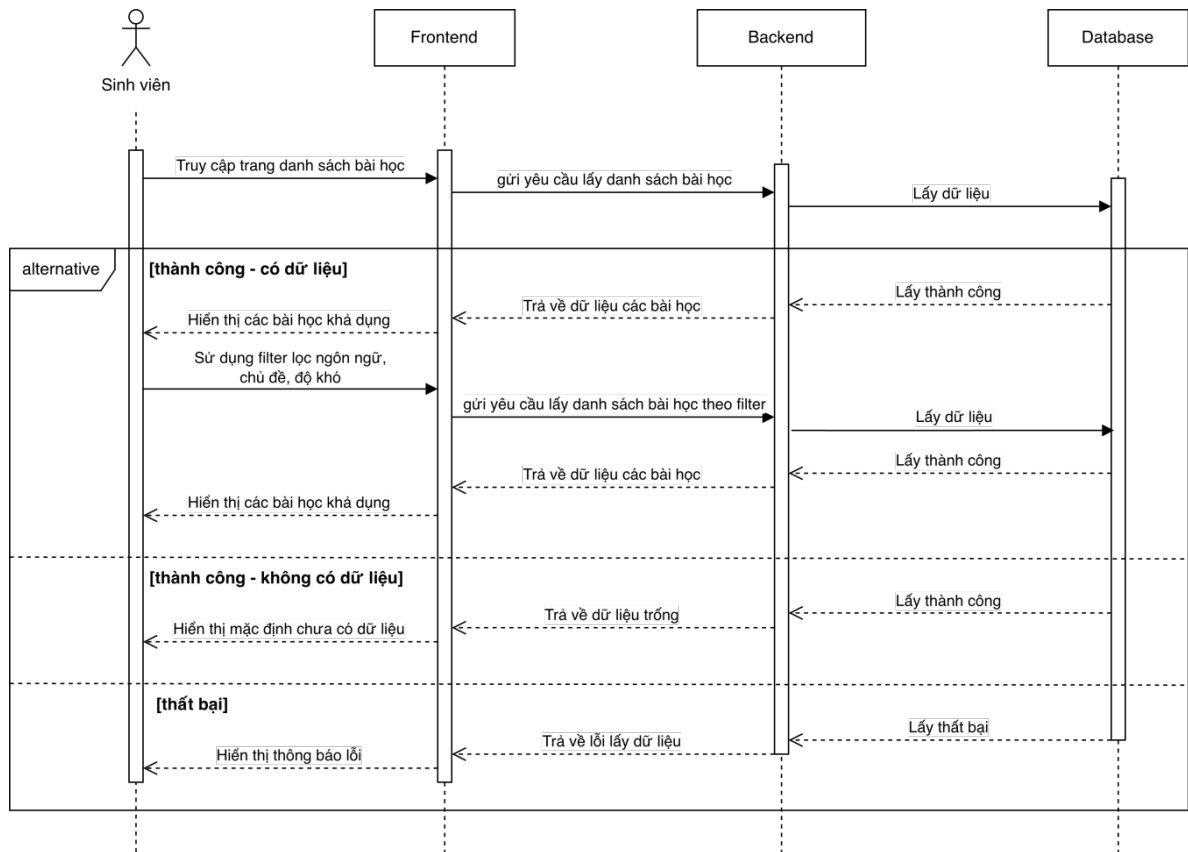
3.4.4. Danh sách Bài học (Xem và Lọc bài học)



Hình 3.13. Giao diện Danh sách bài học



Hình 3.14. Sơ đồ hoạt động chức năng xem Danh sách bài học



Hình 3.15. Sơ đồ trình tự chức năng xem Danh sách bài học

3.4.5. Chi tiết Bài học

Lumicode
real knowledge

Trang chủ > Danh sách bài học > Bài học 1: Biến và kiểu dữ liệu (PYTHON)

Bài học 1: Biến và kiểu dữ liệu (PYTHON)

Mục tiêu

- Nắm vững chủ đề: Biến và kiểu dữ liệu.
- Áp dụng được vào bài tập thực hành trong PYTHON.

Lý thuyết cốt lõi

Bài học này trình bày các khái niệm nền tảng của **Biến và kiểu dữ liệu** trong **PYTHON** cùng ví dụ minh họa ngắn gọn.

Lưu ý triển khai

- Kiểm tra input/output đúng định dạng trước khi nộp bài.
- Ưu tiên lời giải rõ ràng trước tối ưu vì mô.

MỤC LỤC

MỤC LỤC

Bài học 1: Biến và kiểu dữ liệu (PYTHON)

Mục tiêu

Lý thuyết cốt lõi

Lưu ý triển khai

Tiến độ bài học 25%

Hoàn thành 1/4 bài tập.

BÀI TẬP THỰC HÀNH

- Bài tập 1 (EASY) - Python
- Bài tập 2 (EASY) - Python
- Bài tập 3 (EASY) - Python
- Bài tập 4 (EASY) - Python

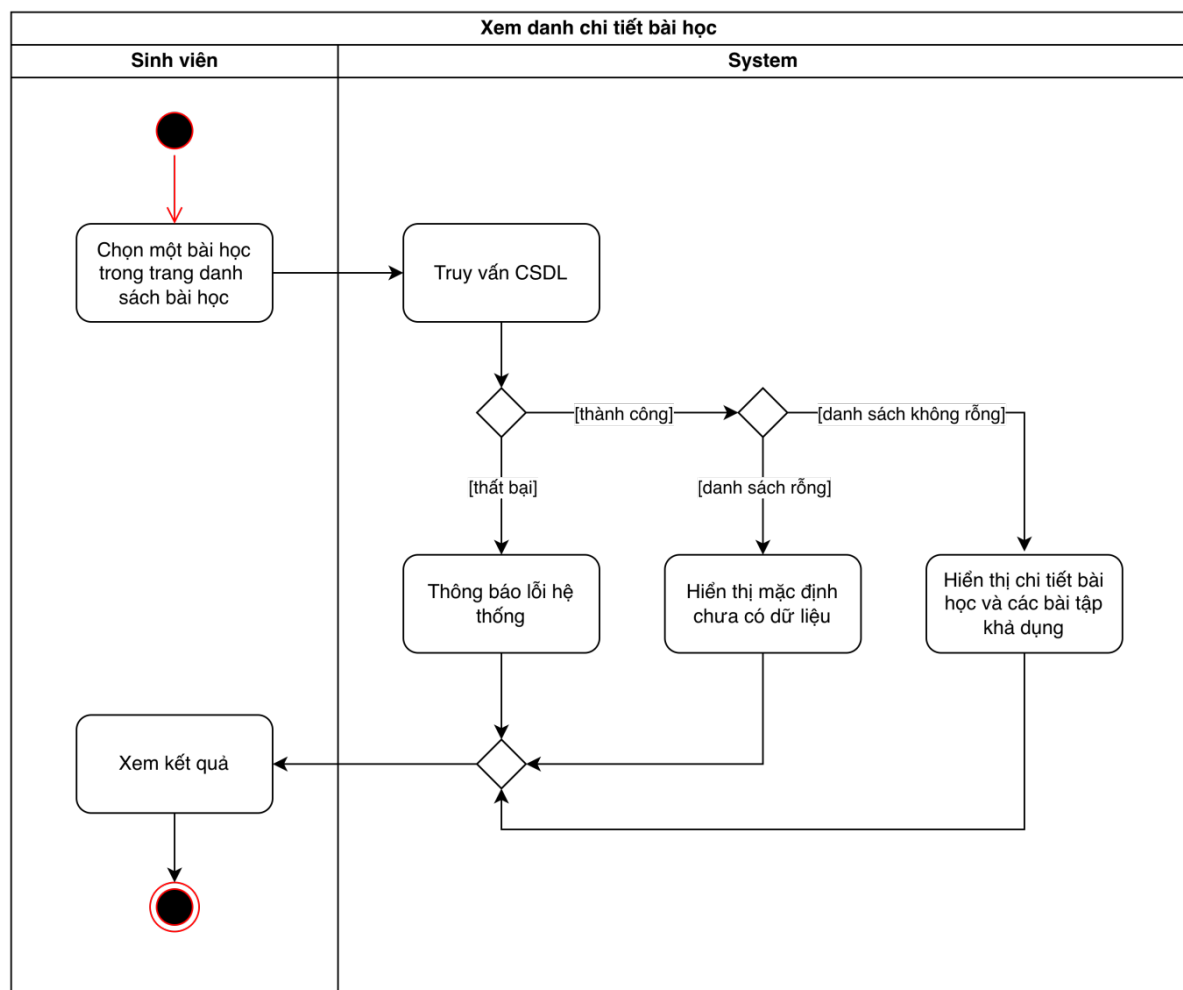
TIP HỌC TẬP

Đừng ngại hỏi AI Tutor nếu bạn gặp khó khăn trong khi làm bài tập!

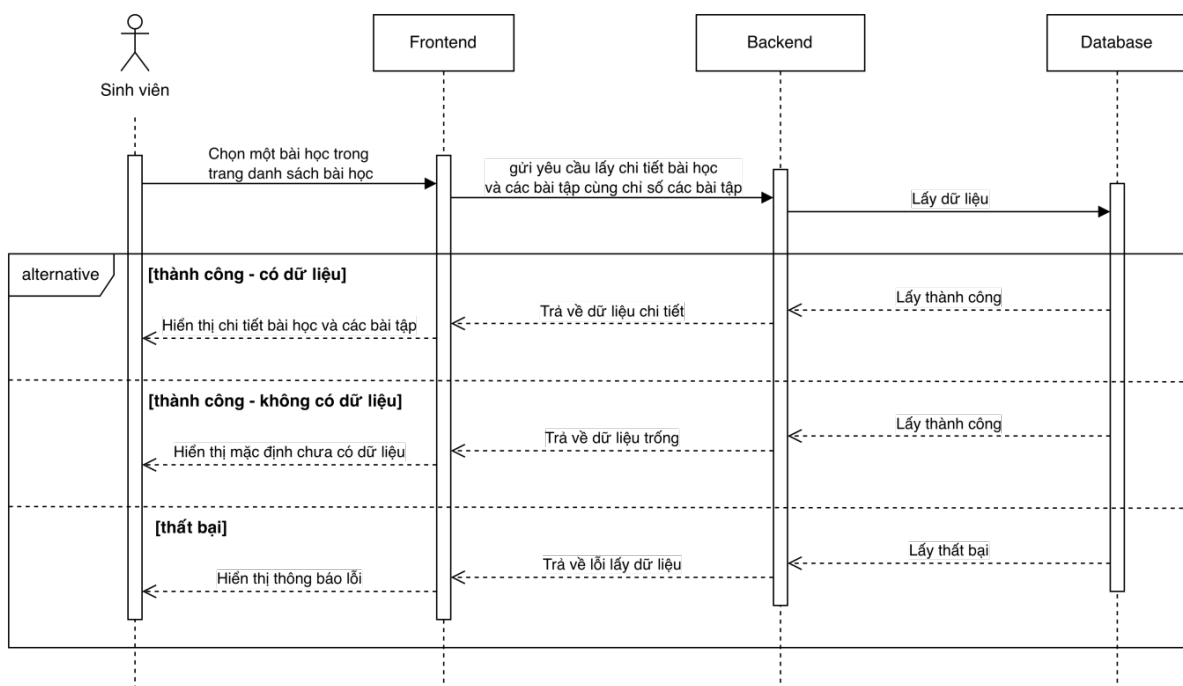
QUAY LẠI Thư viện bài học

TIẾP THEO Bài tập 2 (EASY) - Python

Hình 3.16. Giao diện Chi tiết bài học

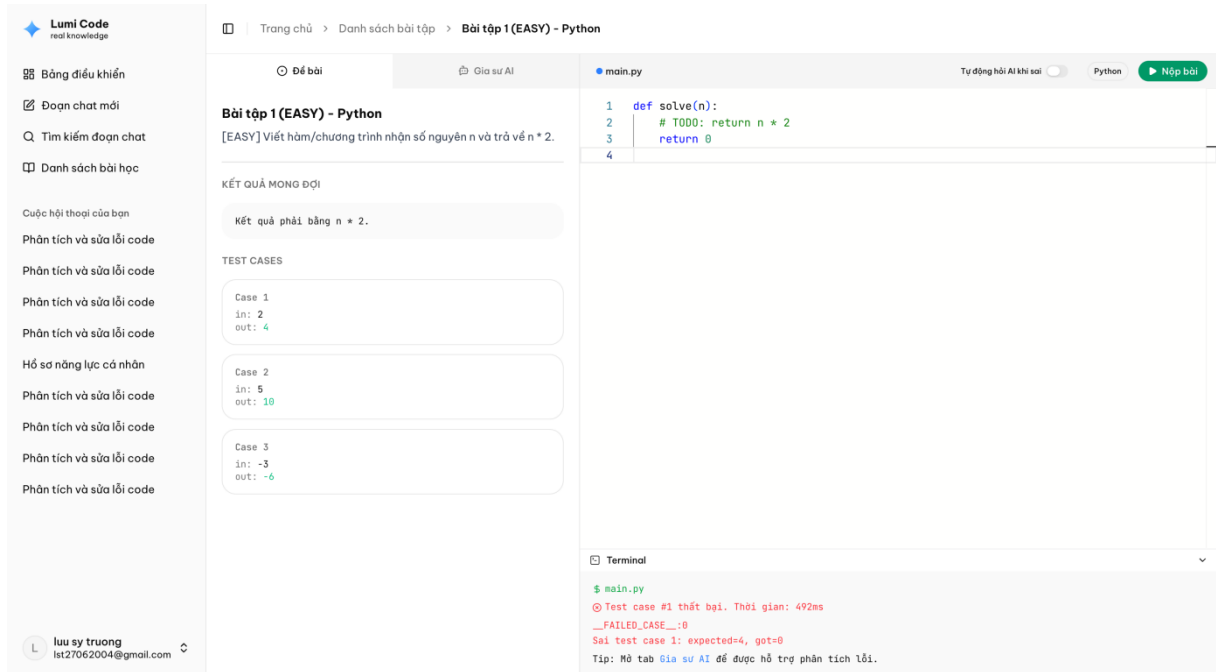


Hình 3.17. Sơ đồ hoạt động chức năng xem Chi tiết bài học

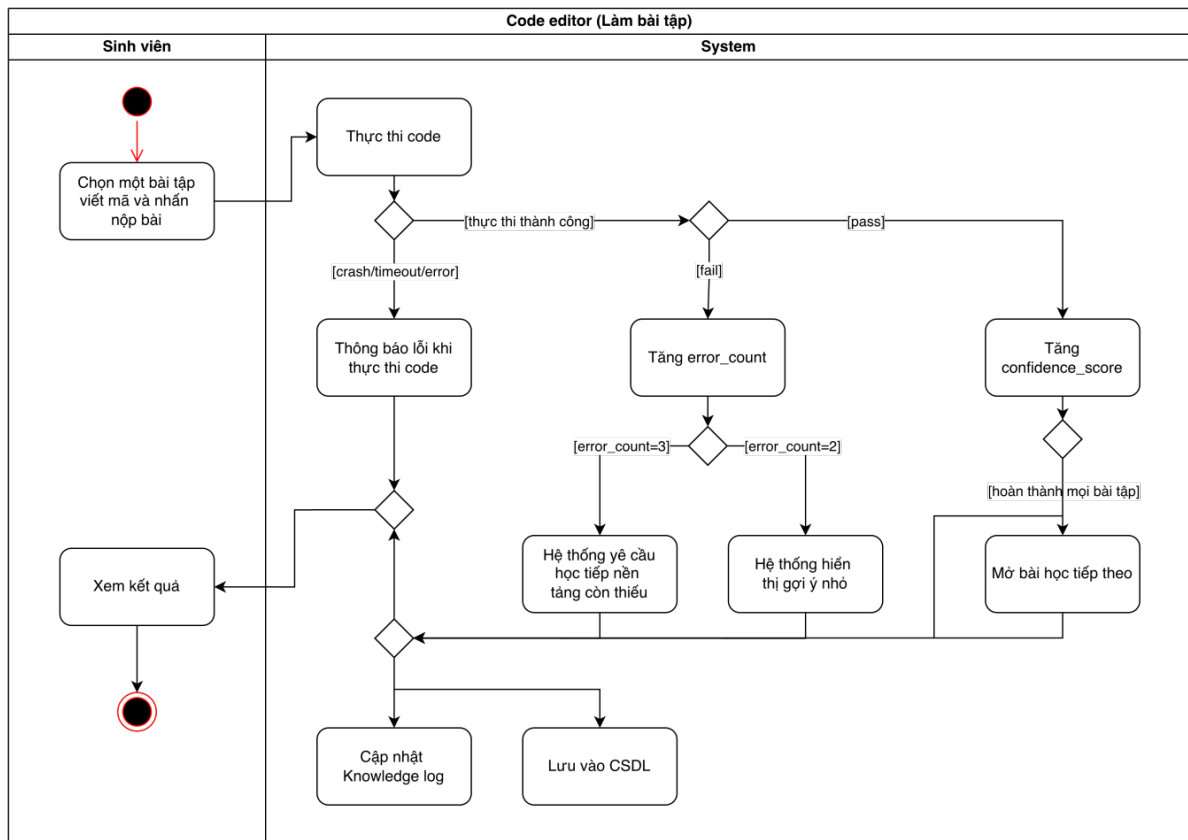


Hình 3.18. Sơ đồ trình tự chức năng xem Chi tiết bài học

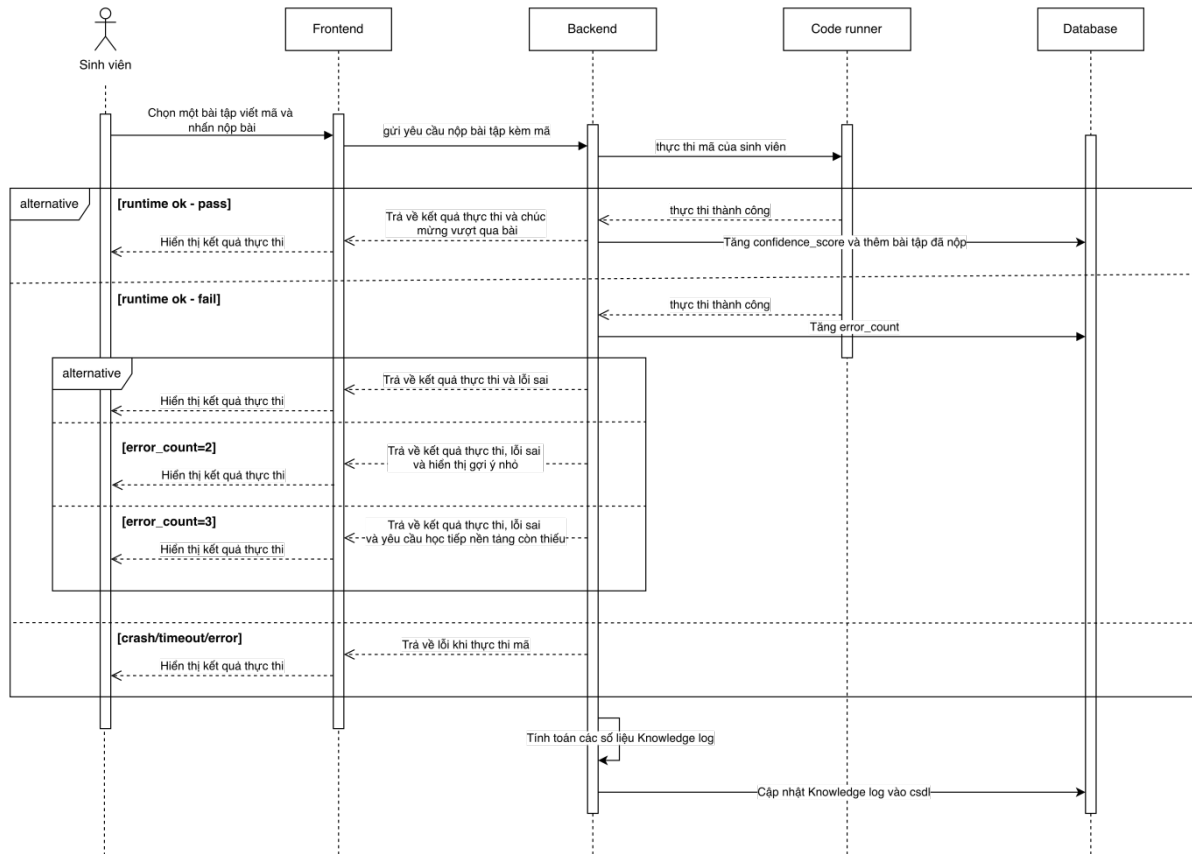
3.4.6. Code Editor (Làm bài tập)



Hình 3.19. Giao diện Code Editor (Làm bài tập)

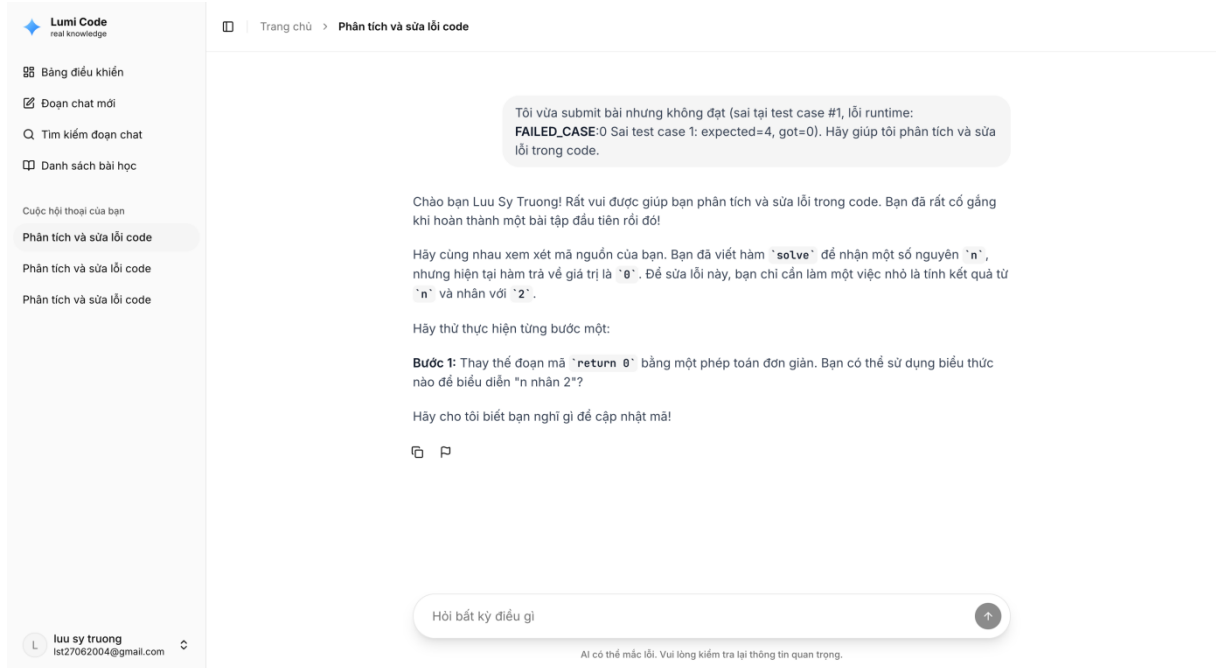


Hình 3.20. Sơ đồ hoạt động chức năng Code editor (Làm bài tập)

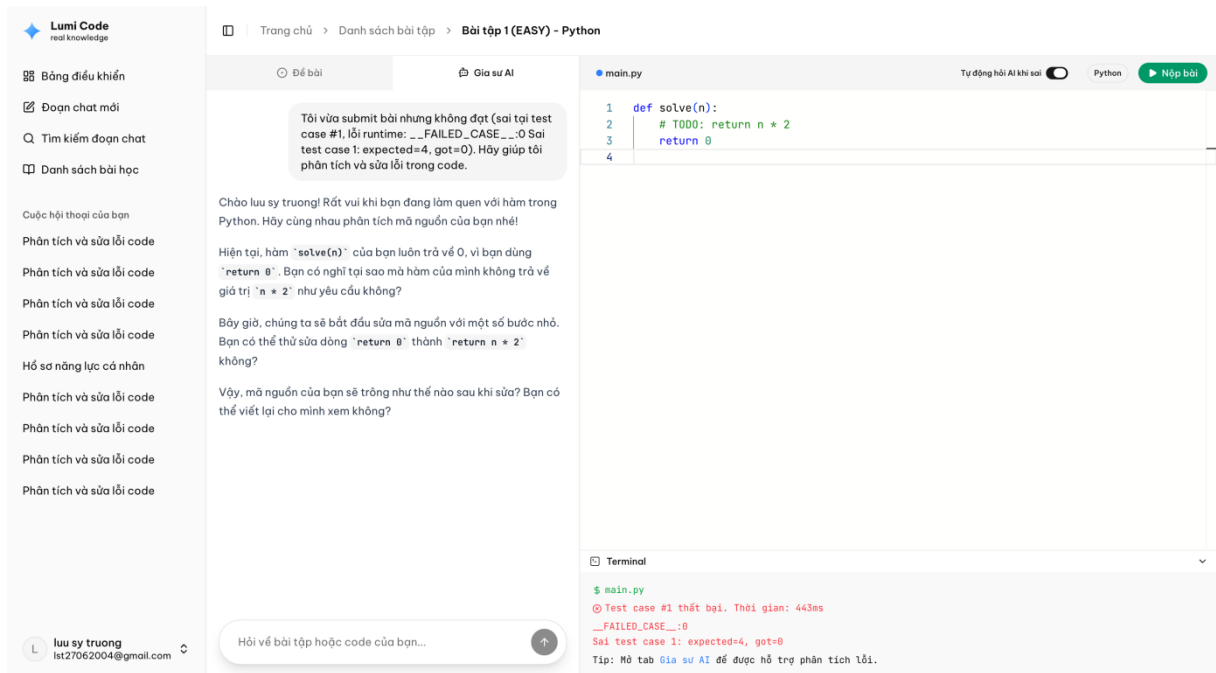


Hình 3.21. Sơ đồ trình tự chức năng Code editor (Làm bài tập)

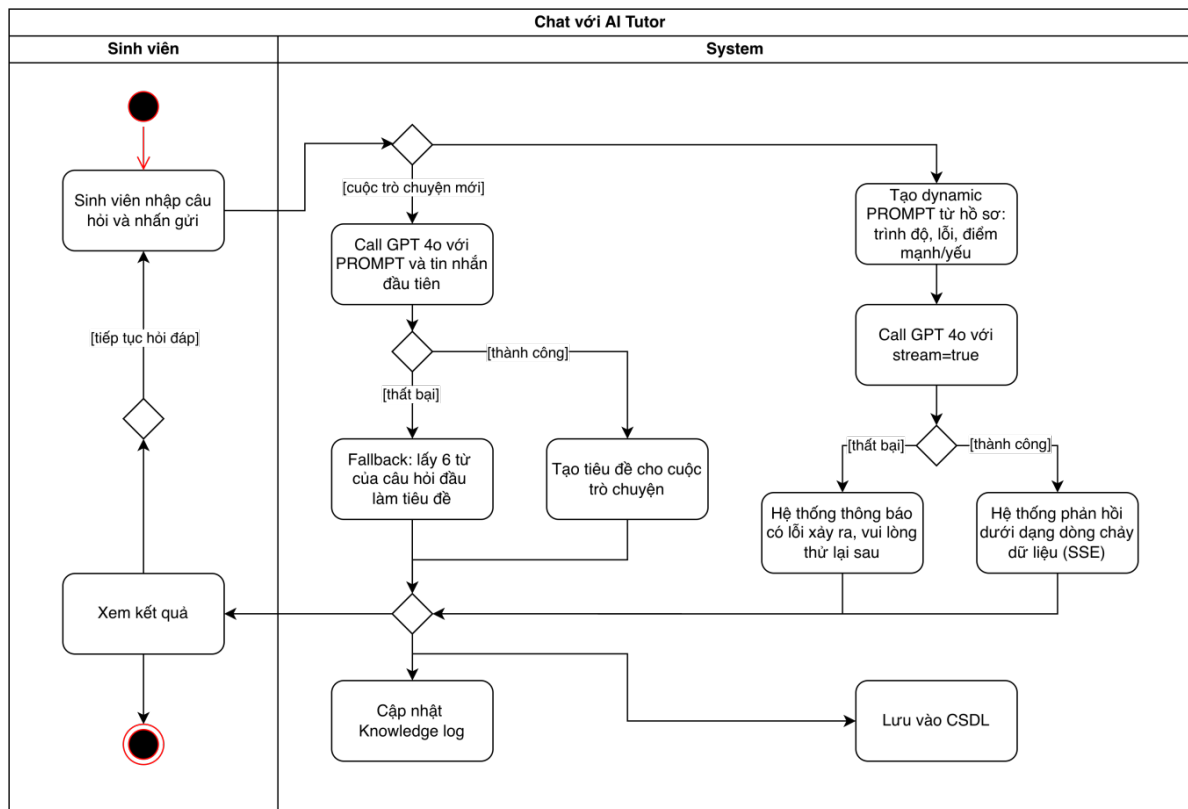
3.4.7. Chat với AI Tutor



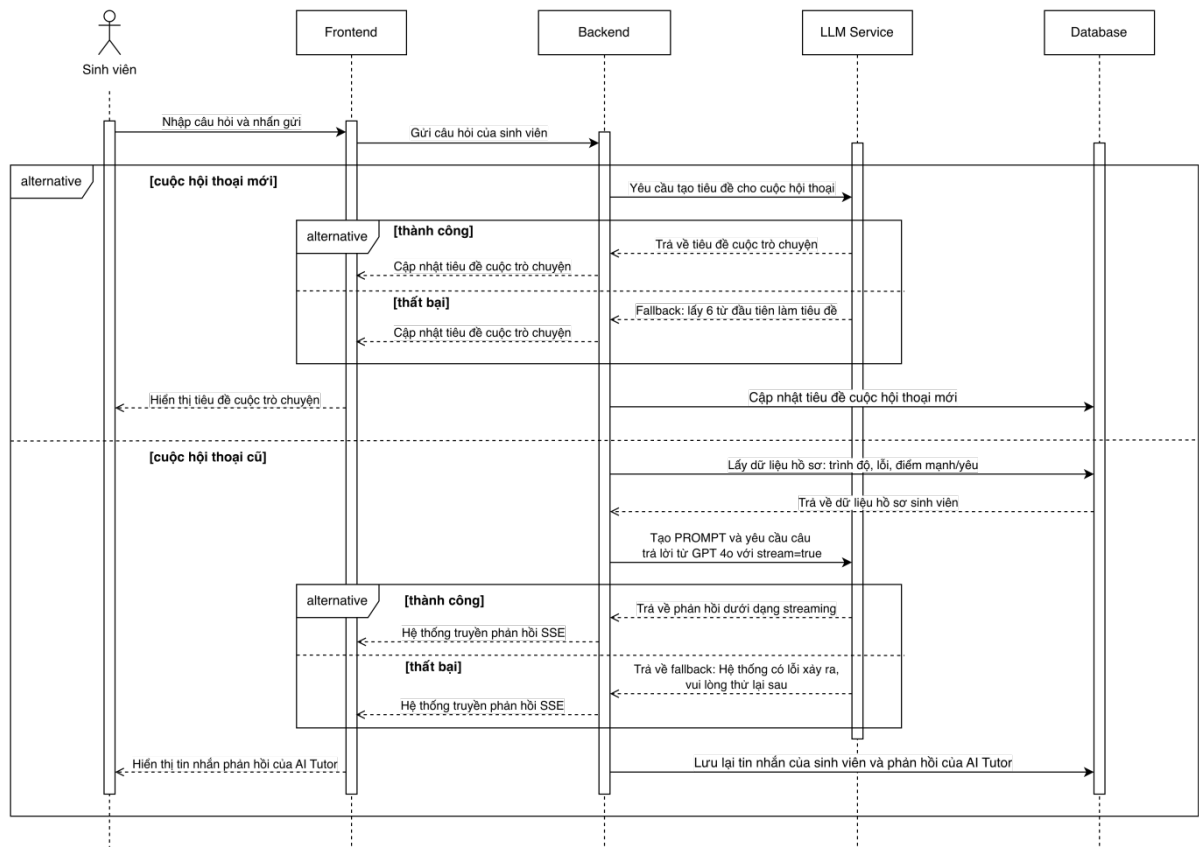
Hình 3.22. Giao diện Chat với AI Tutor



Hình 3.23. Giao diện Chat AI với Tutor trong trình Editor



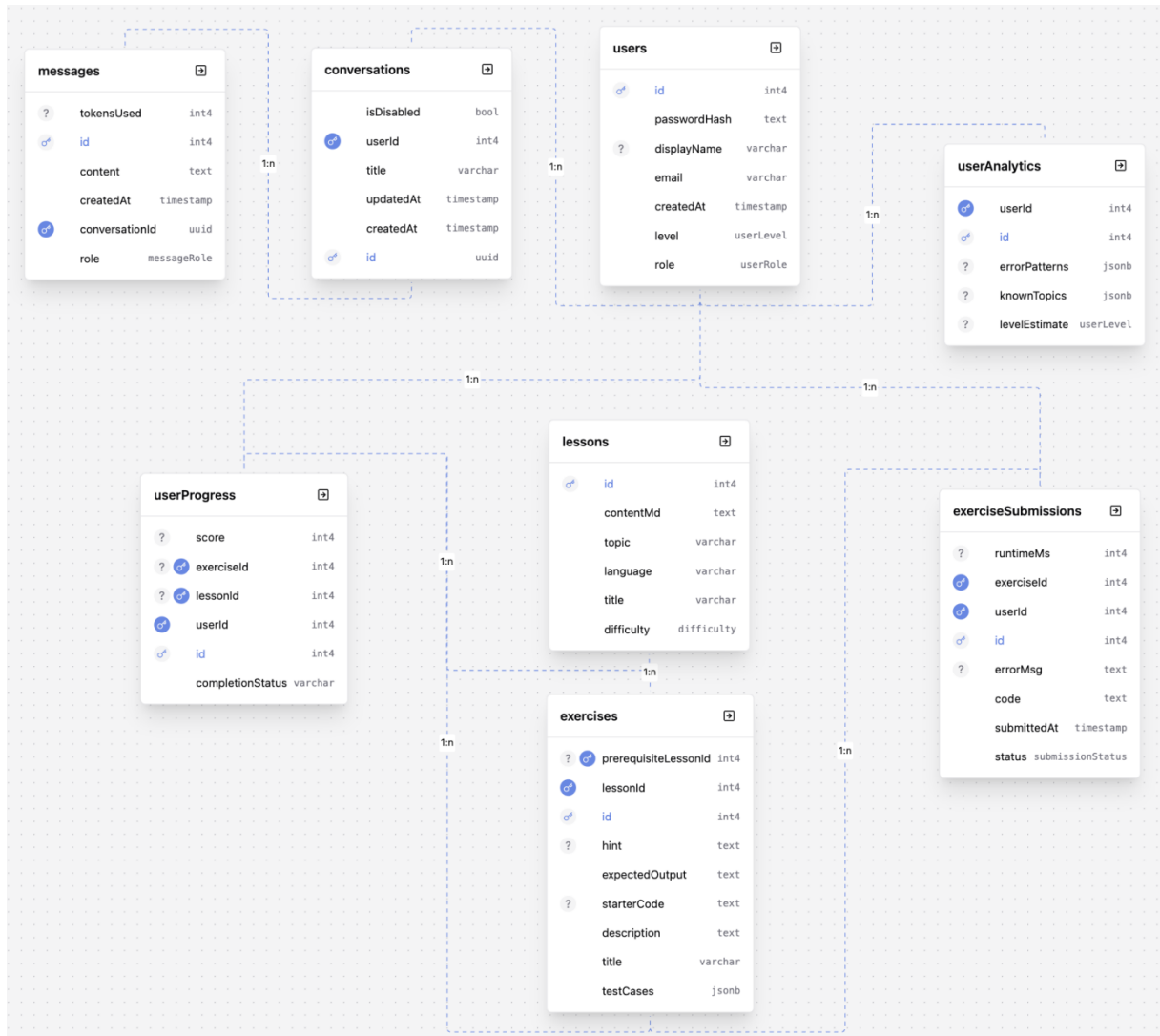
Hình 3.24. Sơ đồ hoạt động chức năng Chat AI với Tutor



Hình 3.25. Sơ đồ trình tự chức năng Chat AI với Tutor

3.5. Thiết kế Cơ sở dữ liệu

3.5.1. Sơ đồ lớp thực thể (ERD)



Hình 3.26. Sơ đồ lớp thực thể hệ thống

Cơ sở dữ liệu của hệ thống PPAT được thiết kế trên PostgreSQL gồm 8 bảng chính, thể hiện trong Hình 3.26. Các bảng được tổ chức xoay quanh thực thể trung tâm là **Users** mọi dữ liệu học tập, hội thoại và tiến độ đều gắn với một người dùng cụ thể.

Quan hệ giữa các bảng được xác định như sau: một người dùng có thể có nhiều cuộc hội thoại (1:n giữa **Users** và **Conversations**), mỗi hội thoại chứa nhiều tin nhắn (1:n giữa **Conversations** và **Messages**). Về phía học tập, mỗi bài học có thể có nhiều bài tập (1:n giữa **Lessons** và **Exercises**), và mỗi lần sinh viên nộp bài được ghi nhận riêng trong **ExerciseSubmissions**. Tiến độ học tập của từng sinh viên trên từng bài học và bài tập được theo dõi qua **UserProgress**, trong khi **UserAnalytics** lưu trữ knowledge log bao gồm các chủ đề đã nắm, lỗi thường gặp và ước tính trình độ hiện tại phục vụ cho cơ chế cá nhân hóa của AI Tutor.

3.5.2. Mô tả chi tiết các bảng

Bảng Users là trung tâm của toàn bộ hệ thống, lưu thông tin tài khoản bao gồm email, mật khẩu đã được hash, tên hiển thị, và hai trường enum quan trọng là level (trình độ: beginner, intermediate, advanced) và role (phân quyền: student, admin). Trường level được cập nhật tự động khi sinh viên hoàn thành đủ số bài tập theo ngưỡng quy định.

Bảng Conversations và Messages lưu toàn bộ lịch sử hội thoại giữa sinh viên và AI Tutor. Mỗi Message ghi nhận vai trò người gửi qua trường role (kiểu enum messageRole: user hoặc assistant), nội dung tin nhắn và số token đã tiêu thụ thông tin hữu ích để theo dõi chi phí API.

Bảng Lessons lưu nội dung bài học dưới dạng Markdown trong trường contentMd, phân loại theo ngôn ngữ lập trình (language), chủ đề (topic) và độ khó (difficulty). Thiết kế này cho phép lọc và gợi ý bài học phù hợp với trình độ từng sinh viên một cách linh hoạt.

Bảng Exercises lưu thông tin bài tập gắn với từng bài học qua lessonId. Điểm đáng chú ý là trường testCases có kiểu JSONB, cho phép lưu linh hoạt nhiều bộ test case với cấu trúc input/output khác nhau mà không cần thêm bảng riêng. Trường prerequisiteLessonId cho phép khai báo bài học điều kiện nếu sinh viên chưa hoàn thành bài học tiên quyết, AI Tutor sẽ chủ động chuyển hướng thay vì giải thích ngay.

Bảng ExerciseSubmissions ghi nhận mỗi lần nộp bài của sinh viên, bao gồm code đã nộp, trạng thái kết quả (status kiểu enum submissionStatus: pass, fail, error), thông báo lỗi nếu có và thời gian thực thi tính bằng millisecond. Dữ liệu này là đầu vào chính cho cơ chế phát hiện lỗi thường gặp trong knowledge log.

Bảng UserProgress theo dõi trạng thái hoàn thành của từng sinh viên trên từng bài học và bài tập, lưu điểm số và trạng thái hoàn thành. Bảng này được truy vấn tổng hợp khi render dashboard tiến độ.

Bảng UserAnalytics là bảng quan trọng nhất đối với tính năng cá nhân hóa. Hai trường JSONB knownTopics và errorPatterns lưu trữ dạng key-value linh hoạt ví dụ errorPatterns có thể chứa "missing_base_case": 3, "off_by_one": 2 để AI biết sinh viên hay mắc lỗi gì và mức độ thường xuyên. Trường levelEstimate là trình độ ước tính theo thời gian thực, có thể khác với level chính thức trong bảng users và được dùng để điều chỉnh độ khó phản hồi của AI.

Bảng 3.3. Các bảng thực thể trong hệ thống

Bảng	Các cột chính	Mục đích
users	id, email, password_hash, display_name, level, created_at	Tài khoản người dùng
lessons	id, title, language, topic, difficulty, content_md, order_index, created_by	Bài học lý thuyết
exercises	id, lesson_id, title, description, starter_code, expected_output, test_cases (JSONB), difficulty	Bài tập thực hành
exercise_submissions	id, user_id, exercise_id, code, status, error_msg, runtime_ms, submitted_at	Lịch sử nộp bài
user_progress	id, user_id, lesson_id, exercise_id, completion_status, score, completed_at	Tiến độ từng bài
user_analytics	id, user_id, known_topics (JSONB), error_patterns (JSONB), level_estimate, last_updated	Knowledge log cá nhân hóa
conversations	id, user_id, title, created_at, last_message_at	Phiên hội thoại với AI
messages	id, conversation_id, role (user/assistant), content, tokens_used, created_at	Tin nhắn trong hội thoại

CHƯƠNG 4: TRIỂN KHAI

4.1. Môi trường và Stack công nghệ

Hệ thống PPAT được xây dựng trên nền tảng TypeScript toàn stack, đảm bảo tính nhất quán về kiểu dữ liệu giữa frontend và backend thông qua package `@workspace/types` dùng chung trong monorepo. Bảng dưới tổng hợp các công nghệ chính được sử dụng:

Bảng 4.1. Bảng tổng hợp công nghệ của thống

Layer	Công nghệ	Version	Lý do chọn
Frontend	Next.js	16.2.2001	App Router, Server Actions, tích hợp sẵn routing
Frontend	React	19	Nền tảng của Next.js, ecosystem lớn
Frontend	TypeScript	5.9	Type safety, chia sẻ kiểu với backend qua monorepo
Frontend	TailwindCSS	4	Utility-first, tốc độ phát triển UI nhanh
Frontend	Radix UI + shaden	latest	Accessibility sẵn có, tùy chỉnh tự do
Frontend	Monaco Editor	4.7	VSCoDe engine, syntax highlighting đa ngôn ngữ
Frontend	TanStack Query	5	Server state, caching, auto-refetch
Frontend	Zustand	5	Client state, lightweight
Frontend	React Hook Form + Zod	7 + 4	Form validation, type-safe schema
Frontend	Recharts	3.8	Biểu đồ tiến độ, radar chart kỹ năng
Backend	Fastify	5.8	Hiệu năng cao, plugin architecture
Backend	TypeScript	6	Consistency với frontend
Backend	Drizzle ORM	0.45	Type-safe ORM, migration support

Backend	Zod	4	Schema validation request/response
Backend	@fastify/jwt	10	Xác thực JWT, access + refresh token
Backend	@fastify/rate-limit	10	Rate limiting bảo vệ API
Backend	@fastify/helmet	13	Bảo mật HTTP headers
Backend	bcrypt	6	Mã hóa mật khẩu
Backend	Dockerode	4	Quản lý Docker container cho code runner
AI	OpenAI SDK	6.33	Official SDK, streaming, tool calling
AI	GPT-4o	-	Reasoning mạnh, context 128k token
Database	PostgreSQL	15	JSONB cho knowledge log, quan hệ phức tạp
Cache	Redis (ioredis)	5.10	Session, rate limiting, cache
Infra	Docker + Dockerode	-	Code runner sandbox isolation
Infra	Docker Compose	2.x	Orchestrate 4 services
Test	Vitest	4	Unit test, coverage
Dev	Faker.js	10	Sinh dữ liệu thử nghiệm
Email	Resend	6	Gửi email thông báo

Một số điểm đáng chú ý so với thiết kế ban đầu trong tài liệu phân tích: frontend sử dụng **Next.js 16** thay vì **React** thuần để tận dụng Server Actions và App Router; backend dùng Fastify 5 dùng OpenAI SDK trực tiếp, giúp quản lý pipeline AI có cấu trúc hơn; code runner được xây dựng bằng Dockerode để lập trình tạo và quản lý container sandbox từ Node.js thay vì gọi Docker CLI. Ngoài ra hệ thống tích hợp Resend để gửi email xác nhận tài khoản và thông báo cho sinh viên.

4.2. Cài đặt và triển khai Docker

4.2.1. Cấu trúc Docker Compose

Hệ thống được đóng gói và triển khai bằng Docker Compose với 4 service chạy trong cùng một internal network tên ppat-network kiểu bridge. Các service giao tiếp với nhau qua tên service thay vì địa chỉ IP cố định, giúp cấu hình không thay đổi giữa các môi trường khác nhau.

Bảng 4.2. Cấu trúc file Docker Compose

Service	Image/Build	Port	Phụ thuộc
db	postgres:15-alpine	5432	
cache	redis:7-alpine	6379	
api	Build từ Dockerfile backend	3000	db, cache
web	Build từ Dockerfile frontend	80	api

Thứ tự khởi động được kiểm soát bằng depends_on kết hợp condition: service_healthy, tức service sau chỉ khởi động khi service trước đã vượt qua healthcheck, không chỉ đơn giản là đã start. Service api chờ cả db và cache healthy trước khi khởi động; web chờ api healthy. Cơ chế này tránh tình trạng backend khởi động nhưng chưa kết nối được database.

Healthcheck được cấu hình riêng cho từng service. Service db dùng lệnh pg_isready để xác nhận PostgreSQL sẵn sàng nhận kết nối. Service cache dùng redis-cli ping. Service api tự gọi endpoint /health bằng fetch, nếu response trả về OK thì mới được coi là healthy.

Dữ liệu PostgreSQL được mount vào named volume ppat-db-data, đảm bảo dữ liệu không mất khi container restart hoặc rebuild image. Redis không cần persistent volume vì chỉ đóng vai trò cache và lưu session tạm thời.

Điểm đặc biệt là service api mount /var/run/docker.sock từ máy host vào trong container. Đây là cơ chế cho phép backend tạo và quản lý container sandbox để thực thi code của sinh viên thông qua thư viện Dockerode, mà không cần cài Docker riêng bên trong container api.

4.2.2. Cấu trúc Dockerfile

Cả hai service api và web đều áp dụng kỹ thuật multi-stage build để tối ưu kích thước image production. Mỗi Dockerfile được chia làm hai giai đoạn.

Giai đoạn đầu tiên chịu trách nhiệm build ứng dụng. Toàn bộ dependencies bao gồm cả devDependencies được cài vào, sau đó build package @workspace/types trước vì cả frontend lẫn backend đều phụ thuộc vào package dùng chung này. Tiếp theo mới

build ứng dụng chính. Với backend, kết quả là thư mục `dist/` chứa JavaScript đã compile từ TypeScript. Với frontend, Next.js tạo ra thư mục `.next/` được tối ưu cho môi trường production.

Giai đoạn thứ hai tạo image production thực sự. Chỉ những file cần thiết để chạy mới được giữ lại, dependencies được cài lại với flag `-prod` để loại bỏ toàn bộ devDependencies. Nhờ cách này, image production gọn hơn đáng kể so với single-stage build thông thường.

Cả hai Dockerfile đều dùng `node:20-alpine` làm base image. Alpine Linux giúp giảm kích thước image xuống còn khoảng một phần ba so với image Ubuntu thông thường. Package manager được cố định ở `pnpm@9.15.9` kích hoạt qua `corepack` để đảm bảo version nhất quán giữa môi trường phát triển và production.

4.2.3. Hướng dẫn cài đặt

Để triển khai hệ thống, máy chủ cần cài sẵn Docker Engine và Docker Compose. Sau khi clone repository về máy, bước đầu tiên là tạo file `.env` ở thư mục gốc với các biến môi trường bắt buộc:

```
# .env
POSTGRES_USER=postgres
POSTGRES_PASSWORD=your_password
POSTGRES_DB=ppat
JWT_SECRET=your_jwt_secret_key
OPENAI_API_KEY=sk-...
CORS_ORIGIN=http://localhost
PORT=3000
RESEND_API_KEY=your_resend_api_key
FRONTEND_URL=http://localhost
LOG_LEVEL=info
```

Sau khi có file `.env`, chạy lệnh sau để build toàn bộ image và khởi động các service: `docker compose up --build -d`

Docker Compose sẽ tự động build image cho api và web, kéo image `postgres:15-alpine` và `redis:7-alpine` từ Docker Hub, tạo network và volume cần thiết, rồi khởi động các service theo đúng thứ tự healthcheck đã cấu hình.

Sau khi service api healthy, chạy migration để tạo schema database và seed dữ liệu thử nghiệm:

```
# terminal
docker compose exec api node apps/be/dist/db/migrate.js
docker compose exec api node apps/be/dist/scripts/seed.js
```

Truy cập `http://localhost` để vào giao diện web. Kiểm tra trạng thái API tại

`http://localhost:3000/health.`

Để dừng hệ thống mà vẫn giữ nguyên dữ liệu, dùng lệnh `docker compose down`. Nếu muốn dừng và xóa toàn bộ dữ liệu kể cả volume, thêm flag `-v` vào lệnh trên.

4.3. Bảo vệ hệ thống

Hệ thống áp dụng bốn lớp bảo vệ độc lập nhằm giảm nguy cơ truy cập trái phép, bảo vệ dữ liệu người dùng, ngăn lạm dụng tài nguyên và hạn chế các hình thức tấn công cơ bản như brute force, spam request hoặc gửi code độc hại.

4.3.1. Xác thực JWT

Hệ thống dùng JWT làm cơ chế xác thực chính. Sau khi đăng nhập hoặc đăng ký thành công, backend sinh hai loại token. Access token có thời hạn 24 giờ, dùng cho các request thông thường đến API. Refresh token có thời hạn 7 ngày, dùng để cấp lại access token mới khi hết hạn mà không bắt người dùng đăng nhập lại. JWT được ký bằng secret lấy từ biến môi trường `JWT_SECRET`, payload chứa các thông tin định danh tối thiểu gồm `userId`, `email` và `role`. Access token được lưu vào cookie tên `access_token` với thuộc tính `httpOnly` và `secure` khi chạy ở môi trường `production`.

Luồng xác thực diễn ra như sau: người dùng gửi email và mật khẩu, backend kiểm tra thông tin và nếu hợp lệ trả về cả hai token. Frontend lưu access token và dùng cho các request về sau. Khi access token hết hạn, client gọi endpoint `refresh` để lấy token mới. Middleware `authenticate` kiểm tra tính hợp lệ của token trước khi cho phép truy cập các route được bảo vệ như quản lý bài học, bài tập, tiến độ và chat AI.

Việc tách thành hai loại token giúp cân bằng giữa bảo mật và trải nghiệm sử dụng. Access token ngắn hạn giảm rủi ro nếu token bị lộ, trong khi refresh token dài hạn tránh việc người dùng phải đăng nhập lại liên tục.

4.3.2. Mã hóa mật khẩu

Mật khẩu người dùng không được lưu dưới dạng văn bản thuần mà được băm bằng `bcrypt` trước khi ghi vào cơ sở dữ liệu. Khi đăng ký, mật khẩu được xử lý qua `bcrypt.hash()` với `salt rounds = 12`, kết quả hash được lưu vào cột `passwordHash`. Khi đăng nhập, hệ thống dùng `bcrypt.compare()` để so khớp mật khẩu người dùng nhập vào với chuỗi hash đã lưu, mà không cần giải mã ngược.

`Bcrypt` là hàm băm một chiều có tích hợp salt riêng cho từng mật khẩu, vì vậy hai người dùng có cùng mật khẩu sẽ cho ra hai chuỗi hash khác nhau. Điều này làm giảm hiệu quả của các cuộc tấn công dùng bảng tra cứu sẵn. Mức `salt rounds = 12` làm tăng chi phí tính toán cho mỗi lần băm, từ đó làm chậm brute force offline, đồng thời vẫn không gây chậm đáng kể cho các thao tác đăng ký hoặc đổi mật khẩu trong điều kiện bình thường.

4.3.3. Cô lập môi trường chạy code

Code của sinh viên được thực thi trong container Docker riêng biệt thay vì chạy trực tiếp trên máy chủ ứng dụng. Backend dùng `Dockerode` để tạo container theo từng ngôn

ngữ lập trình, nạp mã nguồn và test case vào container dưới dạng archive, thực thi kiểm tra rồi xóa container sau khi chạy xong. Image được chọn theo ngôn ngữ: node:20-alpine cho JavaScript, python:3.11-alpine cho Python, gcc:13 cho C++ và eclipse-temurin:21-jdk-jammy cho Java.

Môi trường sandbox được cô lập theo ba chiều. Về mạng, container được đặt NetworkMode: 'none' nên không có quyền truy cập internet hay mạng nội bộ. Về tài nguyên, bộ nhớ bị giới hạn ở 128 MB và CPU ở mức 50% một nhân với CpuQuota: 50000 và CpuPeriod: 100000. Về thời gian, mỗi lần chạy có timeout 10 giây, vượt quá giới hạn này thì kết quả được tính là thất bại. Sau khi chạy xong, container bị xóa ngay bằng remove(force: true) để tránh tồn đọng tài nguyên.

Cách kiểm tra code được thiết kế theo mô hình wrapper: với JavaScript và Python, hệ thống bọc code trong chương trình chạy hàm solve; với C++ và Java, hệ thống sinh file nguồn tạm, biên dịch rồi chạy qua shell script. Test case được đưa vào theo từng bộ input và expected output để so sánh kết quả tự động.

Cơ chế này đảm bảo code lỗi hoặc code độc hại từ phía sinh viên không thể ảnh hưởng đến hoạt động của backend chính, không thể truy cập mạng bên ngoài và không thể chiếm tài nguyên hệ thống quá mức cho phép.

4.3.4. Giới hạn tần suất request

Hệ thống sử dụng @fastify/rate-limit để giới hạn tối đa 100 request mỗi phút cho mỗi client, áp dụng ở tầng server cho toàn bộ request đến backend. Cơ chế này làm chậm brute force vào các endpoint đăng nhập và refresh token, giảm spam request vào API chat và submit code, đồng thời tránh việc một client chiếm quá nhiều tài nguyên server trong thời gian ngắn.

Redis đã được tích hợp vào hệ thống và phục vụ các tác vụ lưu trạng thái dùng chung như session và cache dữ liệu. Trong cấu hình hiện tại, rate limiting hoạt động ở tầng Fastify và Redis đóng vai trò hạ tầng sẵn sàng để mở rộng rate limiting đồng bộ giữa nhiều instance nếu hệ thống cần scale out trong tương lai.

4.4. Tính năng AI Chatbot cá nhân hóa

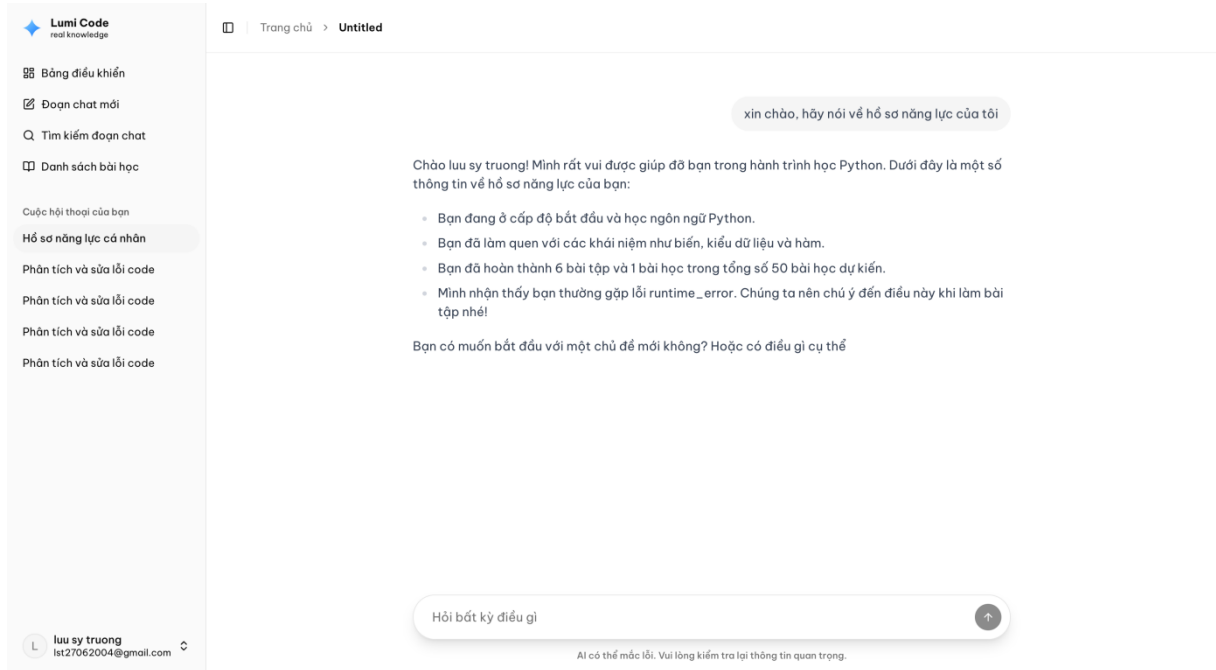
4.4.1. Ghi nhớ lịch sử và cá nhân hóa theo hồ sơ học tập

Khác với các công cụ AI thông thường như ChatGPT, mỗi cuộc hội thoại trong PPAT được lưu trữ vĩnh viễn và hiển thị ở sidebar bên trái, cho phép sinh viên quay lại xem lại bất kỳ phiên hỗ trợ nào trước đó. Hệ thống không chỉ lưu lịch sử mà còn liên tục cập nhật knowledge log bao gồm trình độ hiện tại, các chủ đề đã nắm, lỗi thường gặp và tiến độ hoàn thành bài tập sau mỗi lần tương tác.

Điểm then chốt của cơ chế cá nhân hóa là dynamic system prompt: trước mỗi lần gọi GPT-4o, hệ thống truy vấn bảng user_analytics để lấy dữ liệu hồ sơ học tập của sinh viên, sau đó ghép vào system prompt động. Nhờ vậy, AI luôn có đầy đủ ngữ cảnh về

người học trước khi sinh phản hồi.

Hình 4.1 minh họa kết quả thực tế: khi sinh viên hỏi "hãy nói về hồ sơ năng lực của tôi", AI trả lời chính xác rằng sinh viên đang ở cấp độ bắt đầu, học Python, đã quen với biến kiểu dữ liệu hàm, hoàn thành 6 bài tập và 1 bài học trong tổng số 50, đồng thời chủ động nhắc đến lỗi `runtime_error` mà sinh viên hay mắc phải. Toàn bộ thông tin này được đọc trực tiếp từ knowledge log, không phải do sinh viên khai báo trong cuộc hội thoại đó.



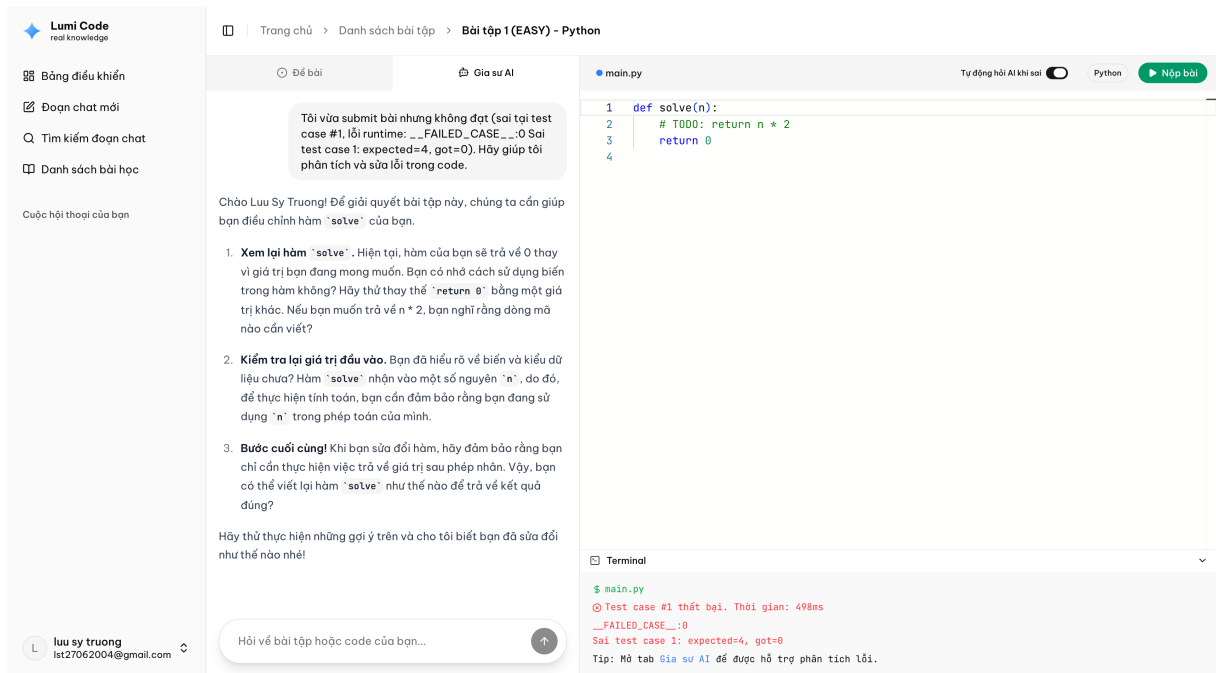
Hình 4.1. AI Tutor tóm tắt hồ sơ năng lực cá nhân của sinh viên

4.4.2. Phân tích lỗi code và gợi ý debug theo phương pháp Socratic

Khi sinh viên nộp bài và không đạt, hệ thống tự động gửi thông tin lỗi từ code runner bao gồm loại lỗi, test case thất bại, giá trị expected và giá trị thực tế vào cuộc hội thoại với AI Tutor. AI không đưa ra đáp án trực tiếp mà áp dụng phương pháp Socratic: phân tích nguyên nhân gốc rễ, sau đó đặt câu hỏi dẫn dắt để sinh viên tự tìm ra hướng sửa.

Hình 4.2 thể hiện một tình huống cụ thể: sinh viên submit hàm `solve(n)` với nội dung `return 0`, hệ thống phát hiện `FAILED_CASE`: Sai test case 1: `expected=4, got=0`. AI xác định ngay rằng hàm nhận đúng tham số `n` nhưng luôn trả về 0 thay vì tính toán. Thay vì viết ra `return n`

2, AI đặt câu hỏi gợi ý: "Bạn có thể dùng biểu thức nào để biểu diễn `n` nhân với 2?" buộc sinh viên phải tư duy và tự hoàn thiện logic. Giao diện tab "Trợ giảng AI" tích hợp ngay trong màn hình bài tập, bên cạnh Monaco Editor và terminal, giúp sinh viên không cần chuyển tab khi cần hỗ trợ.



Hình 4.2. AI Tutor phân tích lỗi và dẫn dắt sinh viên sửa từng bước

4.4.3. Điều chỉnh độ khó và lộ trình học theo performance

Hệ thống theo dõi liên tục kết quả nộp bài của sinh viên và tự động điều chỉnh phản hồi theo từng mức độ, thay vì xử lý đồng nhất mọi trường hợp.

Gợi ý bài tiếp theo sau khi hoàn thành. Khi sinh viên hoàn thành toàn bộ bài tập của một bài học (ảnh 1 "Hoàn thành 4/4 bài tập", tiến độ 100%), hệ thống hiển thị nút "Bài tiếp theo" và điều hướng sang bài học kế tiếp được gợi ý bằng LLM dựa trên profile của sinh viên, phù hợp với lộ trình. Sidebar bài học cũng hiển thị "Bài tiếp theo: Bài học 7 Xử lý chuỗi (PYTHON)", phản ánh lộ trình được sắp xếp theo trình độ từ cơ sở đến nâng cao.

The screenshot shows the 'Bài học 4: Hàm (PYTHON)' page in the Lumi Code interface. The sidebar on the left contains navigation links like 'Bảng điều khiển', 'Đoạn chat mới', 'Tìm kiếm đoạn chat', and 'Danh sách bài học'. The main content area displays the title 'Bài học 4: Hàm (PYTHON)' and a progress bar indicating 100% completion. The right sidebar shows a list of exercises and a tip about using the AI Tutor.

Hình 4.3. Hệ thống gợi ý bài tiếp theo sau khi hoàn thành 4/4 bài tập

Tự động kích hoạt Gia sư AI khi phát hiện lỗi. Khi sinh viên nộp bài không đạt lần đầu, hệ thống tự động chuyển sang tab Gia sư AI và gửi thông tin lỗi vào cuộc hội thoại sinh viên không cần mô tả lại vấn đề. Ở lần fail đầu, AI áp dụng phương pháp Socratic: đặt câu hỏi từng bước nhỏ để dẫn dắt sinh viên tự tìm hướng sửa, chưa tiết lộ đáp án (ảnh 2 "Bước 1: Bạn có hiểu rõ cách nhân số nguyên n với 2 không?").

The screenshot shows the 'Bài tập 1 (EASY) - Python' page in the Lumi Code interface. The sidebar on the left contains navigation links like 'Bảng điều khiển', 'Đoạn chat mới', 'Tìm kiếm đoạn chat', and 'Danh sách bài học'. The main content area displays the title 'Bài tập 1 (EASY) - Python' and a progress bar indicating 100% completion. The right sidebar shows a list of exercises and a tip about using the AI Tutor.

Hình 4.4. AI dẫn dắt Socratic ở lần fail đầu tiên

Tăng mức gợi ý khi fail lần thứ hai và hiển thị gợi ý từ hệ thống. Nếu sinh viên vẫn

không vượt qua sau lần thứ hai, AI chuyển sang cung cấp hướng dẫn cụ thể hơn: liệt kê các bước thực hiện rõ ràng, hiển thị code mẫu minh họa và dự đoán kết quả test case (ảnh 3 "Dưới đây là các bước cụ thể... return n * 2... solve(2) sẽ trả về 4").

The screenshot shows the Lumi Code interface. On the left is a sidebar with navigation options: "Bảng điều khiển", "Đoạn chat mới", "Tìm kiếm đoạn chat", "Danh sách bài học", and "Cuộc hội thoại của bạn". The main area is titled "Bài tập 1 (EASY) - Python" and contains instructions in Vietnamese. It mentions a failed submission and provides three steps for correction: 1. Update the function to return n * 2, 2. Rename the function to solve, and 3. Check test cases. A code editor on the right shows the current code:

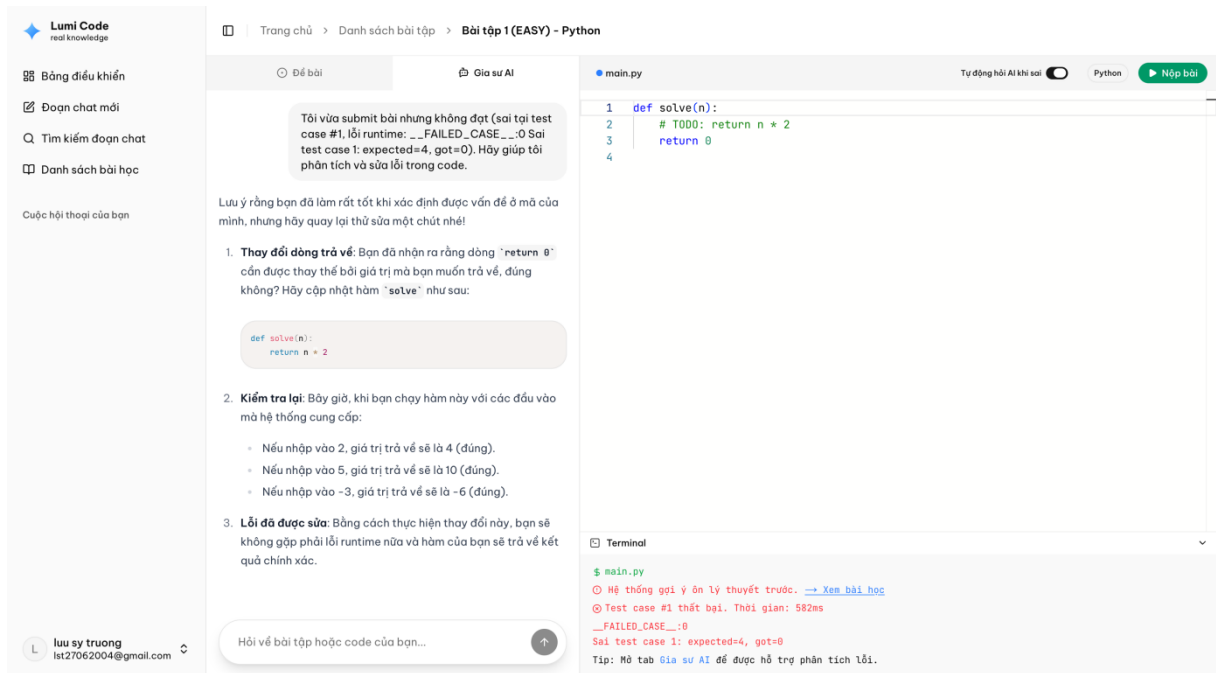
```
def solve(n):
    # TODO: return n * 2
    return 0
```

. At the bottom, a terminal window shows the output:

```
$ main.py
? Gợi ý: Biến là hộp chứa dữ liệu, hãy gán giá trị đúng cho nó.
@ Test case #1 thất bại. Thời gian: 445ms
__FAILED_CASE__:0
Sai test case 1: expected=4, got=0
Típ: Mở tab Gia sư AI để được hỗ trợ phân tích lỗi.
```

Hình 4.5. AI cung cấp hướng dẫn chi tiết và code mẫu ở lần fail thứ hai

Yêu cầu ôn lại lý thuyết khi fail nhiều lần. Ở lần fail thứ ba trở đi, terminal hiển thị cảnh báo "Hệ thống gợi ý ôn lý thuyết trước → Xem bài học" kèm link điều hướng trực tiếp về bài học liên quan (ảnh 4). Đây là cơ chế phát hiện lỗ hổng kiến thức: thay vì tiếp tục sửa code bề mặt, hệ thống xác định sinh viên cần củng cố nền tảng lý thuyết trước khi quay lại bài tập.



Hình 4.6. Hệ thống yêu cầu ôn lại bài học sau nhiều lần fail liên tiếp

Bốn mức phản hồi này hoàn thành mở bài mới, Socratic lần một, hướng dẫn chi tiết lần hai, redirect lý thuyết lần ba tạo thành cơ chế adaptive learning tự động, không cần giảng viên can thiệp thủ công.

4.5. Dữ liệu thử nghiệm

Dữ liệu thử nghiệm được tạo tự động thông qua script seed.ts đặt tại apps/be/scripts/seed.ts, chạy bằng lệnh:

- `pnpm db:seed`

Script thực hiện insert toàn bộ dữ liệu vào PostgreSQL và xuất báo cáo kết quả tại thời điểm chạy (25/05/2026). Kết quả seed thực tế khớp hoàn toàn với yêu cầu PI 4.2:

Bảng 4.3. Tổng hợp kết quả dữ liệu thử nghiệm

Hạng mục	Yêu cầu	Thực tế
Tổng người dùng	100	100
Beginner (student1 - 40)	40	40
Intermediate (student41 - 80)	40	40
Advanced (student81 - 100)	20	20
Tổng bài học	50	50

Tổng bài tập	200	200
Bài tập Easy	80	80
Bài tập Medium	80	80
Bài tập Hard	40	40
Bài tập có test cases rỗng	0	0

Bảng 4.4. Phân bố bài học theo ngôn ngữ

Ngôn ngữ	Số bài học	Chủ đề tiêu biểu
Python	13	Biến, điều kiện, vòng lặp, hàm, OOP, đệ quy, exception
JavaScript	13	Variables, arrow function, array methods, async/await, DOM
C++	12	Con trỏ, mảng, struct, STL vector/map, xử lý file
Java	12	Class, inheritance, interface, collections, exception

Mỗi bài tập được gắn với một bài học cụ thể, có đủ starter code, expected output và ít nhất 3 test cases (basic, edge case, large input). Mật khẩu tất cả tài khoản thử nghiệm được hash bcrypt trong DB, không lưu plaintext.

4.6. Kiểm thử hệ thống

4.6.1. Unit Test

Hệ thống có hai bộ unit test độc lập, kiểm tra các module không phụ thuộc vào network hay database.

Bộ 1 - CodeRunnerService: Kiểm tra module thực thi code trong Docker container với 3 tình huống: chạy JavaScript, chạy Python và xử lý vòng lặp vô hạn (timeout 10 giây). Thời gian TC-U03 cao do cơ chế chờ đủ 10 giây trước khi kill container, đúng với thiết kế sandbox cô lập.

Bộ 2 - LessonsService (LLM Parser): Kiểm tra logic parse JSON từ phản hồi GPT-4o. Do LLM đôi khi trả về JSON lẫn trong đoạn văn bản tự nhiên, hàm parseLessonRecommendation() cần trích xuất đúng JSON dù có text rác xung quanh. TC-U04 kiểm tra trường hợp thành công, TC-U05 kiểm tra trường hợp LLM không tạo ra JSON hợp lệ thì trả về null.

Bảng 4.5. Kết quả Unit Test

Test case	Module	Mô tả	Kết quả	Thời gian
TC-U01	CodeRunnerService	Chạy JavaScript trong container	Pass	403ms
TC-U02	CodeRunnerService	Chạy Python trong container	Pass	434ms
TC-U03	CodeRunnerService	Xử lý timeout vòng lặp vô hạn	Pass	10.308ms
TC-U04	LessonsService	Parse JSON từ LLM có text rác xung quanh	Pass	2ms
TC-U05	LessonsService	Trả về null khi LLM không sinh JSON hợp lệ	Pass	0ms

```
• truong@MacBook-Air be % npx vitest run src/services/code-runner.test.ts
```

```
RUN v4.1.4 /Users/truong/code/ai-agent/ai-totur/apps/be
```

```
✓ src/services/code-runner.test.ts (3 tests) 11146ms
✓ CodeRunnerService (Dockerized) (3)
  ✓ should run JavaScript code in a container 403ms
  ✓ should run Python code in a container 434ms
  ✓ should handle timeout correctly 10308ms

Test Files 1 passed (1)
Tests 3 passed (3)
Start at 00:18:02
Duration 11.48s (transform 40ms, setup 0ms, import 205ms, tests 11.15s, environment 0ms)
```

```
• truong@MacBook-Air be % npx vitest run src/modules/lessons/lessons.service.test.ts
```

```
RUN v4.1.4 /Users/truong/code/ai-agent/ai-totur/apps/be
```

```
✓ src/modules/lessons/lessons.service.test.ts (2 tests) 3ms
✓ LessonsService Unit Tests (Logic LLM parser) (2)
  ✓ kiểm tra parse JSON từ kết quả của LLM thành công dù có text rác xung quanh 2ms
  ✓ trả về null nếu LLM không tạo ra JSON hợp lệ hoặc thiếu lesson id 0ms

Test Files 1 passed (1)
Tests 2 passed (2)
Start at 00:18:20
Duration 561ms (transform 58ms, setup 0ms, import 442ms, tests 3ms, environment 0ms)
```

Hình 4.7. Kết quả chạy Unit Test trên terminal

4.6.2. Integration Test

Integration test kiểm tra toàn bộ luồng API thực tế trên môi trường dev đang chạy, bao gồm Fastify, PostgreSQL và Redis. 10 test case được tổ chức theo 5 nhóm endpoint, mỗi test case xác nhận HTTP status code, cấu trúc response và thời gian phản hồi dưới 200ms.

Bảng 4.6. Kết quả Integration Test

Nhóm	Test case	Mô tả	Kết quả	Thời gian
Health	TC-I01	GET /health trả về status 200	Pass	28ms
Auth	TC-I02	Đăng ký email trùng trả về 409	Pass	14ms
Auth	TC-I03	Đăng nhập đúng trả về accessToken	Pass	105ms
Auth	TC-I04	Đăng nhập sai mật khẩu trả về 401	Pass	109ms
Auth	TC-I05	Gọi API không có token trả về 401	Pass	2ms
Lessons	TC-I06	GET /api/lessons trả về danh sách	Pass	137ms
Lessons	TC-I07	GET /api/lessons/:id trả về chi tiết	Pass	120ms
Exercises	TC-I08	GET /api/exercises/:id trả về chi tiết	Pass	113ms
Progress	TC-I09	GET /api/progress/me trả về tiến độ	Pass	113ms
Analytics	TC-I10	GET /api/analytics/me trả về knowledge log	Pass	118ms

Tất cả 10 test case đều pass, thời gian phản hồi từ 2ms đến 137ms, đáp ứng yêu cầu phi chức năng dưới 200ms. Tổng: 10/10 passed, 0 failed.

```

● truong@MacBook-Air be % npx vitest run src/integration.test.ts

RUN v4.1.4 /Users/truong/code/ai-agent/ai-totur/apps/be

✓ src/integration.test.ts (10 tests) 861ms
  ✓ Health Check (1)
    ✓ GET /health – status 200, phản hồi < 200ms 28ms
  ✓ Auth API (4)
    ✓ POST /api/auth/register – email trùng trả về 409 14ms
    ✓ POST /api/auth/login – đăng nhập đúng trả về token 105ms
    ✓ POST /api/auth/login – sai mật khẩu trả về 401 109ms
    ✓ GET /api/lessons – không có token trả về 401 2ms
  ✓ Lessons API (2)
    ✓ GET /api/lessons – trả về danh sách, phản hồi < 200ms 137ms
    ✓ GET /api/lessons/:id – trả về chi tiết bài học 120ms
  ✓ Exercises API (1)
    ✓ GET /api/exercises/:id – trả về chi tiết bài tập 113ms
  ✓ Progress API (1)
    ✓ GET /api/progress/me – trả về tiến độ cá nhân, phản hồi < 200ms 113ms
  ✓ Analytics API (1)
    ✓ GET /api/analytics/me – trả về knowledge log, phản hồi < 200ms 118ms

Test Files 1 passed (1)
Tests 10 passed (10)
Start at 00:18:40
Duration 1.02s (transform 28ms, setup 0ms, import 39ms, tests 861ms, environment 0ms)

```

Hình 4.8. Kết quả chạy *Integration Test* trên terminal

4.6.3. Performance Test

Performance test mô phỏng 50 người dùng đồng thời gửi request liên tục trong 30 giây đến 4 endpoint thực tế: /health, /api/lessons, /api/progress/me và /api/analytics/me. Token xác thực được lấy một lần trong hàm setup() và dùng chung cho toàn bộ virtual users nhằm tập trung đo hiệu năng xử lý dữ liệu thay vì chi phí đăng nhập. Công cụ sử dụng là k6, chạy trực tiếp trên môi trường MacBook Air M3.

Kết quả được lấy từ lần chạy có điều kiện tải ổn định nhất trong 4 lần thử nghiệm.

Bảng 4.7. Kết quả *Performance Test* (50 VUs, 30 giây)

Chỉ số	Giá trị	Ngưỡng yêu cầu	Đạt yêu cầu
p(95) response time	80.66ms	Dưới 200ms	Đạt
Response time trung bình	28.8ms	Không quy định	Ghi nhận
Response time trung vị	18.91ms	Không quy định	Ghi nhận
Response time tối đa	428.23ms	Không quy định	Ghi nhận
Tỷ lệ lỗi HTTP	0.00%	Dưới 1%	Đạt

Tổng requests thực hiện	5.477 requests	Không quy định	Ghi nhận
Throughput	174.07 req/s	Không quy định	Ghi nhận
Checks passed	8.215/8.215 (100%)	1	Đạt

95% request hoàn thành trong 80.66ms, thấp hơn 2.5 lần so với ngưỡng 200ms yêu cầu. Response time tối đa 428ms xảy ra ở một số request đầu tiên do cold start của connection pool, không ảnh hưởng đến p(95). Tỷ lệ lỗi HTTP duy trì 0% trong suốt 1.369 iterations, xác nhận hệ thống ổn định dưới tải 50 người dùng đồng thời.

```
● truong@MacBook-Air be % k6 run k6-test.js
```



```

      / \   / \   / \
     /   \ /   \ /   \
    /_____/_____/_____\

execution: local
script: k6-test.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration (incl. graceful stop):
    * default: 50 looping VUs for 30s (gracefulStop: 30s)


THRESHOLDS

http_req_duration
✓ 'p(95)<200' p(95)=80.66ms

http_req_failed
✓ 'rate<0.01' rate=0.00%


TOTAL RESULTS

checks_total.....: 8215      261.082557/s
checks_succeeded...: 100.00% 8215 out of 8215
checks_failed.....: 0.00%    0 out of 8215

✓ login successful in setup
✓ GET /health - 200
✓ GET /health < 200ms
✓ GET /api/lessons - 200
✓ GET /api/lessons < 200ms
✓ GET /api/progress/me - 200
✓ GET /api/analytics/me - 200


HTTP
http_req_duration.....: avg=28.8ms min=141µs med=18.91ms max=428.23ms p(90)=66.86ms p(95)=80.66ms
{ expected_response:true }...: avg=28.8ms min=141µs med=18.91ms max=428.23ms p(90)=66.86ms p(95)=80.66ms
http_req_failed.....: 0.00%    0 out of 5477
http_reqs.....: 5477      174.065632/s


EXECUTION
iteration_duration.....: avg=1.11s min=1s med=1.07s max=1.37s p(90)=1.21s p(95)=1.22s
iterations.....: 1369      43.508463/s
vus.....: 19      min=19      max=50
vus_max.....: 50      min=50      max=50


NETWORK
data_received.....: 21 MB      653 kB/s
data_sent.....: 1.5 MB      49 kB/s
```

Hình 4.9. Kết quả chạy Performance Test với k6 trên terminal

4.6.4. Tổng kết kiểm thử

Bảng 4.8. Tổng hợp kết quả kiểm thử

Loại test	File	Số test case	Passed	Failed
Unit Test	code-runner.test.ts	3	3	0
Unit Test	lessons.service.test.ts	2	2	0
Integration Test	integration.test.ts	10	10	0
Performance Test	k6-test.js	7 checks	7	0
Tổng cộng	4 file	22	22	0

4.7. Tài liệu hướng dẫn sử dụng

4.7.1. Hướng dẫn cài đặt và khởi động hệ thống

Yêu cầu môi trường: Docker Desktop đang chạy, Node.js 20 trở lên, pnpm và OpenAI API key hợp lệ.

Các bước khởi động:

- Clone repository về máy và di chuyển vào thư mục dự án.
- Sao chép file cấu hình môi trường: `cp apps/be/.env.example apps/be/.env` và điền OPENAI_API_KEY, RESEND_API_KEY cùng các biến còn lại vào file .env.
- Khởi động PostgreSQL và Redis bằng Docker Compose: `docker compose up -d db cache`.
- Cài đặt dependencies từ thư mục gốc: `pnpm install`.
- Chạy migration database: `pnpm db:migrate`.
- Seed dữ liệu thử nghiệm: `pnpm db:seed`.
- Khởi động toàn bộ hệ thống từ thư mục gốc: `pnpm dev`. Backend Fastify khởi động trên cổng 3001, frontend Next.js khởi động trên cổng 3000.
- Truy cập hệ thống tại `<http://localhost:3000>`.

4.7.2. Hướng dẫn dành cho sinh viên

Bước 1 - Đăng ký tài khoản: Sinh viên truy cập hệ thống và chọn "Đăng ký". Điền đầy đủ tên hiển thị, email, mật khẩu và chọn trình độ ban đầu phù hợp (Mới bắt đầu, Trung bình hoặc Nâng cao). Trình độ ban đầu được dùng để khởi tạo knowledge log và định hướng lộ trình học từ phiên đầu tiên.

Bước 2 - Xem danh sách bài học: Sau khi đăng nhập, sinh viên vào mục "Danh sách bài học" để xem toàn bộ bài học theo ngôn ngữ lập trình (Python, JavaScript, C++, Java) và mức độ khó. Mỗi bài học hiển thị tiến độ hoàn thành và số bài tập đã làm.

Bước 3 - Học lý thuyết và làm bài tập: Sinh viên chọn bài học để đọc lý thuyết, sau đó chuyển sang danh sách bài tập đính kèm. Giao diện bài tập chia làm hai phần: đề bài bên trái và Monaco Editor bên phải. Sinh viên viết code vào editor và nhấn "Nộp bài" để chạy test cases. Terminal phía dưới hiển thị kết quả từng test case cùng thông báo pass hoặc fail.

Bước 4 - Sử dụng Gia sư AI: Khi gặp khó khăn, sinh viên chuyển sang tab "Gia sư AI" ngay trong màn hình bài tập để được hỗ trợ theo ngữ cảnh bài đang làm. Sinh viên cũng có thể vào mục "Đoạn chat mới" ở sidebar để bắt đầu cuộc hội thoại độc lập về bất kỳ chủ đề lập trình nào. AI Tutor tự động nhận biết trình độ và lỗi thường gặp từ knowledge log mà không cần sinh viên tự giới thiệu lại. Khi nộp bài sai, hệ thống tự động gửi thông tin lỗi vào cuộc hội thoại để AI phân tích và dẫn dắt sửa lỗi.

Bước 5 - Theo dõi tiến độ: Sinh viên vào mục "Bảng điều khiển" để xem tổng quan tiến độ học tập, bao gồm số bài học và bài tập đã hoàn thành, các lỗi thường gặp và lộ trình bài học được gợi ý tiếp theo dựa trên knowledge log hiện tại.

4.7.3. Hướng dẫn dành cho Admin

Bước 1 - Đăng nhập tài khoản Admin: Tài khoản admin mặc định được tạo sẵn với thông tin đăng nhập: email **admin@gmail.com**, mật khẩu **11111**. Sau khi đăng nhập lần đầu, admin nên đổi mật khẩu trong phần cài đặt tài khoản.

Bước 2 - Quản lý người dùng: Admin vào mục quản lý người dùng để xem danh sách tài khoản, kiểm tra trình độ và trạng thái hoạt động của từng sinh viên.

Bước 3 - Quản lý bài học: Admin tạo mới, chỉnh sửa hoặc xóa bài học. Mỗi bài học cần điền tiêu đề, ngôn ngữ lập trình, chủ đề, mức độ khó và nội dung lý thuyết định dạng Markdown.

Bước 4 - Quản lý bài tập và test cases: Admin tạo bài tập gắn với bài học tương ứng, bao gồm đề bài, starter code, expected output và danh sách test cases. Mỗi bài tập cần có ít nhất 3 test cases gồm basic, edge case và large input để đảm bảo chất lượng kiểm tra.

4.8. Kết quả và Đánh giá

4.8.1. Chức năng hoàn thành

Bảng 4.9. Checklist thi công các Use Case theo yêu cầu PI

Use Case	Yêu cầu PI	Nội dung	Trạng thái
UC01	PI 3.3	Quản lý người dùng: đăng ký, đăng nhập, cập nhật profile	Hoàn thành
UC02	PI 3.3	Quản lý bài học: xem danh sách, xem chi tiết, lọc theo chủ đề	Hoàn thành

UC03	PI 3.3	Quản lý bài tập: xem bài tập, submit code, kiểm tra kết quả	Hoàn thành
UC04	PI 3.3	Tương tác chat với Gia sư AI: gửi tin nhắn, nhận hướng dẫn, lưu lịch sử	Hoàn thành
UC05	PI 3.3	Theo dõi tiến độ: xem thống kê, phân tích điểm yếu	Hoàn thành
PI 3.4	PI 3.4	Tích hợp OpenAI GPT-4o, ghi nhớ lịch sử, điều chỉnh độ khó, phân tích lỗi code	Hoàn thành
PI 4.1	PI 4.1	Triển khai Docker Compose với Fastify, PostgreSQL, Redis	Hoàn thành
PI 4.2	PI 4.2	Dữ liệu thử nghiệm: 100 users, 50 bài học, 200 bài tập	Hoàn thành

4.8.2. Ưu điểm

Cá nhân hóa phản hồi dựa trên knowledge log. Đây là điểm khác biệt cốt lõi so với các công cụ AI thông thường. Thay vì trả lời theo từng câu hỏi đơn lẻ, hệ thống theo dõi toàn bộ hành trình học tập của sinh viên và điều chỉnh từng phản hồi phù hợp với trình độ, lỗi đã mắc và tốc độ tiến bộ của từng cá nhân. Kết quả thực tế được minh họa qua 3 sinh viên cùng đặt một câu hỏi nhưng nhận được 3 phản hồi hoàn toàn khác nhau.

Gợi ý bài học thông minh bằng LLM. Hệ thống không chỉ gợi ý bài tiếp theo theo thứ tự cố định mà sử dụng GPT-4o để phân tích hồ sơ học tập và đề xuất bài học phù hợp nhất với trình độ và lỗ hổng kiến thức hiện tại của từng sinh viên.

Tích hợp Gia sư AI trực tiếp trong trình soạn thảo code. Sinh viên không cần chuyển sang công cụ khác khi cần hỗ trợ. Tab Gia sư AI nằm ngay cạnh Monaco Editor, khi nộp bài sai hệ thống tự động đẩy thông tin lỗi vào cuộc hội thoại và AI phân tích ngay lập tức. Trải nghiệm này tương tự một IDE thông minh có trợ lý tích hợp sẵn.

Code runner đa ngôn ngữ trong sandbox cô lập. Hệ thống hỗ trợ thực thi code Python, JavaScript, C++ và Java trực tiếp trên trình duyệt thông qua Docker container với timeout 10 giây và không có network access. Sinh viên học lý thuyết và thực hành trong cùng một môi trường mà không cần cài đặt IDE hay compiler trên máy cá nhân.

Kiến trúc SOA dễ mở rộng. Bốn service độc lập (Frontend Next.js, Backend Fastify, PostgreSQL, Redis) chạy trong Docker Compose, cho phép thêm service mới như analytics nâng cao hay multi-agent AI mà không ảnh hưởng đến các thành phần hiện có.

Hiệu năng đáp ứng yêu cầu phi chức năng. Kết quả kiểm thử thực tế cho thấy p(95) response time đạt 80.66ms với 50 người dùng đồng thời, thấp hơn 2.5 lần so với ngưỡng 200ms yêu cầu. Tỷ lệ lỗi duy trì 0% trong toàn bộ quá trình kiểm thử.

4.8.3. Hạn chế

Phụ thuộc vào OpenAI API. Chi phí token tăng tuyến tính theo số lượng người dùng và độ dài hội thoại. Trong môi trường triển khai thực tế với hàng trăm sinh viên đồng thời, chi phí vận hành có thể trở thành rào cản đáng kể.

Chưa có mobile app. Hệ thống hiện tại chỉ hỗ trợ trình duyệt web. Monaco Editor và giao diện code runner chưa được tối ưu cho màn hình nhỏ, ảnh hưởng đến trải nghiệm code trực tiếp trên thiết bị di động.

Chưa tích hợp LMS của trường. Hệ thống hoạt động độc lập, chưa kết nối với hệ thống quản lý học tập của nhà trường nên điểm số và tiến độ không được đồng bộ với hồ sơ học tập chính thức.

Knowledge log phụ thuộc vào dữ liệu tương tác. Với sinh viên mới đăng ký, knowledge log còn ít dữ liệu nên chất lượng cá nhân hóa trong các phiên đầu tiên chưa cao, cần tích lũy thêm tương tác để AI hiểu chính xác hơn về người học.

4.8.4. Hướng phát triển

Fine-tuning model cho tiếng Việt và lập trình. GPT-4o hiện tại hoạt động tốt nhưng chưa được tối ưu cho ngữ cảnh giảng dạy lập trình bằng tiếng Việt. Một model được fine-tune trên dữ liệu hội thoại thực tế của sinh viên Việt Nam sẽ cho phản hồi tự nhiên và chính xác hơn đồng thời giảm chi phí API.

Tích hợp RAG với tài liệu của trường. Sử dụng Retrieval-Augmented Generation để AI có thể tham chiếu trực tiếp giáo trình, đề cương và tài liệu nội bộ của trường, giúp phản hồi bám sát chương trình học chính thức hơn.

Multi-agent AI. Thay vì một AI duy nhất xử lý tất cả, kiến trúc multi-agent gồm AI Tutor hướng dẫn lý thuyết, AI Reviewer chấm và nhận xét code, và AI Exercise Generator tự động sinh bài tập mới phù hợp với trình độ từng sinh viên.

Nhận diện trạng thái cảm xúc. Tích hợp phân tích cảm xúc từ hành vi tương tác như thời gian phản hồi, tần suất thử lại và ngôn ngữ sử dụng để AI điều chỉnh cách dạy phù hợp hơn khi sinh viên đang căng thẳng hoặc mất động lực.

Tích hợp LMS của trường. Kết nối với hệ thống quản lý học tập của nhà trường để đồng bộ điểm số, tiến độ và lịch học, giúp giảng viên theo dõi toàn diện hơn mà không cần chuyển đổi giữa nhiều hệ thống.

KẾT LUẬN

Đồ án đã nghiên cứu và xây dựng thành công Hệ thống Gia sư Lập trình Cá nhân hóa sử dụng Mô hình Ngôn ngữ Lớn với các kết quả chính sau:

- Thiết kế và triển khai giao diện chat-based tích hợp Monaco Code Editor, hỗ trợ trải nghiệm học lập trình trực tiếp.
- Tích hợp OpenAI GPT-4o với cơ chế Dynamic System Prompt, cá nhân hóa phản hồi dựa trên knowledge log của từng sinh viên.
- Xây dựng module theo dõi tiến độ với Knowledge Enrichment, hệ thống tự động cập nhật hồ sơ học tập và cải thiện chất lượng phản hồi theo thời gian.
- Xây dựng code runner đa ngôn ngữ (Python, JavaScript, C++, Java) trong Docker sandbox cô lập, tích hợp Gia sư AI trực tiếp trong trình soạn thảo code.
- Triển khai thành công trên Docker Compose với 4 service độc lập theo kiến trúc SOA.
- Tạo dữ liệu thử nghiệm đầy đủ: 100 người dùng, 50 bài học, 200 bài tập có test cases.
- Kiểm thử đạt 15/15 test cases passed, performance test với 50 người dùng đồng thời đạt $p(95) = 80.66\text{ms}$, tỷ lệ lỗi 0%.

Kết quả khảo sát 47 sinh viên cho thấy hơn 94% đã sử dụng AI trong học tập nhưng chỉ dừng ở mức AI tổng quát, không cá nhân hóa. Hơn 90% gặp ít nhất một trong các khó khăn mà một gia sư cá nhân có thể giải quyết trực tiếp. Hệ thống PPAT được xây dựng để lấp đầy khoảng trống này: không chỉ trả lời câu hỏi đơn lẻ như ChatGPT thông thường, mà theo dõi toàn bộ hành trình học tập và điều chỉnh từng phản hồi phù hợp với trình độ, lỗi đã mắc và tốc độ tiến bộ của từng sinh viên.

Trong tương lai, hệ thống có thể được mở rộng với fine-tuning model tiếng Việt, tích hợp RAG với tài liệu của trường, và phát triển kiến trúc multi-agent AI để nâng cao hơn nữa chất lượng hướng dẫn cá nhân hóa.

TÀI LIỆU THAM KHẢO

- [1] Đ. N. Đức, “Xây dựng website học lập trình online,” M.S. thesis, Trường đại học Công nghệ thông tin và Truyền thông - Đại học Thái Nguyên, 2017. Accessed: Jun. 8, 2026. [Online]. Available: <https://repository.ictu.edu.vn/wp-content/uploads/2026/05/25167.pdf>
- [2] ICTU_STUDENT, “Xây dựng chương trình hỗ trợ dạy học mầm non,” M.S. thesis, Trường đại học Công nghệ thông tin và Truyền thông - Đại học Thái Nguyên, 2013.
- [3] B. C. Ngọc, “Xây dựng hệ thống tư vấn thông minh cho giáo dục sử dụng Deep Learning và LLM (RAG),” M.S. thesis, Trường đại học Công nghệ thông tin và Truyền thông - Đại học Thái Nguyên, 2024. Accessed: Jun. 8, 2026. [Online]. Available: <https://repository.ictu.edu.vn/wp-content/uploads/2026/05/22520.pdf>
- [4] E. Grassucci, G. Grassucci, A. Uncini, and D. Comminiello, “Beyond answers: How LLMs can pursue strategic thinking in education,” *IEEE Signal Processing Magazine*, 2026. Accessed: May 27, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2504.04815>
- [5] R. Schmelzer. “How AI is shaping the future of education,” Forbes, Accessed: May 27, 2026. [Online]. Available: <https://www.forbes.com/sites/ronschmelzer/2024/05/28/how-ai-is-shaping-the-future-of-education/>
- [6] B. S. Bloom, “The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring,” *Educational Researcher*, vol. 13, no. 6, pp. 4–16, 1984.
- [7] B. D. Nye, A. C. Graesser, and X. Hu, “Autotutor and family: A review of 17 years of natural language tutoring,” *International Journal of Artificial Intelligence in Education*, 2014.
- [8] E. Kasneci et al., “ChatGPT for good? on opportunities and challenges of large language models for education,” *Learning and Individual Differences*, vol. 103, 2023.
- [9] *Hello GPT-4o*, OpenAI, 2024. Accessed: May 27, 2026. [Online]. Available: <https://openai.com/vi-VN/index/hello-gpt-4o>
- [10] *OpenAI API documentation*, OpenAI, 2024. Accessed: May 27, 2026. [Online]. Available: <https://platform.openai.com/docs>
- [11] *Fastify documentation*, Fastify, 2024. Accessed: May 27, 2026. [Online]. Available: <https://fastify.dev>
- [12] *PostgreSQL 15 documentation*, PostgreSQL Global Development Group, 2024. Accessed: May 27, 2026. [Online]. Available: <https://www.postgresql.org/docs/15>
- [13] *Docker and docker compose documentation*, Docker Inc., 2024. Accessed: May 27, 2026. [Online]. Available: <https://docs.docker.com>
- [14] *Monaco editor documentation*, Microsoft, 2024. Accessed: May 27, 2026. [Online]. Available: <https://microsoft.github.io/monaco-editor>

- [15] *Redis documentation*, Redis Ltd., 2024. Accessed: May 27, 2026. [Online]. Available: <https://redis.io/docs>
- [16] *Drizzle ORM documentation*, Drizzle Team, 2024. Accessed: May 27, 2026. [Online]. Available: <https://orm.drizzle.team>
- [17] TopDev. “Báo cáo thị trường IT Việt Nam năm 2023,” Accessed: May 27, 2026. [Online]. Available: <https://topdev.vn/blog/bao-cao-thi-truong-it-viet-nam-nam-2023>

PHỤ LỤC

Bảng tổng hợp kết quả khảo sát

Dấu thời gian	Bạn hiện đang là?	Bạn đã học lập trình trong bao lâu?	Bạn đang học ngôn ngữ lập trình nào?	Bạn thường học lập trình thông qua nguồn nào?	Bạn thường làm gì sau khi học lý thuyết lập trình?	Khó khăn lớn nhất của bạn khi học lập trình là gì?	Khi gặp lỗi code, bạn thường làm gì?	Bạn có sử dụng AI để hỗ trợ học lập trình không?	Bạn mong muốn hệ thống AI tutor (Gia sư AI) hỗ trợ những chức năng nào?	Bạn có sẵn sàng sử dụng một hệ thống AI tutor để hỗ trợ học lập trình không?	Theo bạn, một hệ thống AI tutor (Gia sư AI) nên có chức năng gì để hỗ trợ học lập trình hiệu quả?	Bạn có muốn tham gia test hệ thống AI tutor trong tương lai không?
3/14/2026 23:26:51	Sinh viên ngành khác	Trên 2 năm	Java	Udemy / Coursera	Làm bài tập thực hành	Không hiểu khái niệm lập trình	Hỏi bạn bè / giảng viên	Thường xuyên	Gợi ý cách sửa lỗi	Có	Không biết	Có
3/14/2026 23:31:20	Sinh viên ngành khác	Dưới 6 tháng	C++	Youtube	Không làm gì thêm	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thỉnh thoảng	Phân tích lỗi code, Trả lời câu hỏi khi học	Có		Có
3/14/2026 23:31:48	Sinh viên ngành khác	6 tháng - 1 năm	JavaScript	Tài liệu chính thức (Documentation)	Viết project nhỏ	Không biết bắt đầu từ đâu	Hỏi bạn bè / giảng viên	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo, Trả lời câu hỏi khi học	Có		Có
3/14/2026 23:34:12	Sinh viên ngành khác	Dưới 6 tháng	C++	Youtube	Làm bài tập thực hành	Không hiểu khái niệm lập trình	Tìm lỗi trên Google	Thường xuyên	Giải thích khái niệm lập trình	Có		Có
3/15/2026 6:28:44	Sinh viên CNTT	Trên 2 năm	Java	Youtube	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi	Có	Giúp giải thích bài toán	Có
3/15/2026 9:34:20	Sinh viên CNTT	Trên 2 năm	Python	Youtube	Làm bài tập thực hành	Không có người hướng dẫn	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo, Trả lời câu hỏi khi học	Có		Có
3/15/2026 10:02:25	Sinh viên ngành khác	6 tháng - 1 năm	JavaScript	Hỏi đáp AI (ChatGPT, Gemini, ...)	Làm bài tập thực hành	Không biết sửa lỗi code	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Phân tích lỗi code, Gợi ý cách sửa lỗi	Có thể		Có
3/15/2026 13:03:31	Sinh viên CNTT	Trên 2 năm	Java	Youtube	Làm bài tập thực hành	Không có người hướng dẫn	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Phân tích lỗi code	Có thể		Có

3/15/2026 13:43:22	Người tự học lập trình	Dưới 6 tháng	C++	Hỏi đáp AI (ChatGPT, Gemini, ...)	Không làm gì thêm	Không có người hướng dẫn	Sử dụng AI (ChatGPT, Gemini, ...)	Chưa từng	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo, Trả lời câu hỏi khi học	Không	Gửi link	Có
3/15/2026 14:52:43	Sinh viên CNTT	Dưới 6 tháng	JavaScript	Hỏi đáp AI (ChatGPT, Gemini, ...)	Làm bài tập thực hành	Không có người hướng dẫn	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Phân tích lỗi code, Gợi ý bài học tiếp theo, Trả lời câu hỏi khi học	Có thể	giải thích code chi tiết, phát hiện và sửa lỗi lập trình, đưa ra bài tập thực hành phù hợp với trình độ, gợi ý cách giải khi người học gặp khó khăn, và theo dõi tiến độ học tập	Có
3/15/2026 14:55:11	Sinh viên CNTT	6 tháng - 1 năm	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Tìm lỗi trên Google	Thường xuyên	Phân tích lỗi code	Có		Có
3/15/2026 14:55:16	Sinh viên CNTT	Dưới 6 tháng	Python	Khóa học ở trường	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo, Trả lời câu hỏi khi học	Có		Có
3/15/2026 14:58:55	Sinh viên CNTT	Trên 2 năm	Python	Tài liệu chính thức (Documentation)	Làm bài tập thực hành	Thiếu bài tập thực hành	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo	Có thể		Có
3/15/2026 14:59:07	Sinh viên CNTT	Trên 2 năm	C++	Youtube	Làm bài tập thực hành	Không hiểu khái niệm lập trình	Tìm lỗi trên Google	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi	Có		Có
3/15/2026 15:12:11	Sinh viên CNTT	Dưới 6 tháng	Python	Hỏi đáp AI (ChatGPT, Gemini, ...)	Không làm gì thêm	Không có người hướng dẫn	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Phân tích lỗi code, Gợi ý cách sửa lỗi, Trả lời câu hỏi khi học	Có		Có
3/15/2026 15:13:11	Sinh viên CNTT	1 - 2 năm	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo, Trả lời câu hỏi khi học	Có		Có
3/15/2026 15:18:48	Sinh viên CNTT	Trên 2 năm	dart	Hỏi đáp AI (ChatGPT, Gemini, ...)	Đọc thêm tài liệu	Thiếu bài tập thực hành	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Trả lời câu hỏi khi học	Có thể	nhắc nhở học tập và đưa cho người học lộ trình học tập đầy đủ	Có

3/15/2026 15:20:23	Sinh viên CNTT	Trên 2 năm	Python	Khóa học ở trường	Không làm gì thêm	Không có người hướng dẫn	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Gợi ý cách sửa lỗi	Không	Đỗ biết	Có
3/15/2026 15:39:49	Sinh viên CNTT	Trên 2 năm	Java	Khóa học ở trường	Viết project nhỏ	Thiếu bài tập thực hành	Sử dụng AI (ChatGPT, Gemini, ...)	Thỉnh thoảng	Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo	Có thể		Có
3/15/2026 15:53:18	Sinh viên CNTT	Trên 2 năm	Java	Hỏi đáp AI (ChatGPT, Gemini, ...)	Làm bài tập thực hành	Không hiểu khái niệm lập trình	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình	Có		Có
3/15/2026 16:07:45	Sinh viên CNTT	Trên 2 năm	Python	Youtube	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình	Có		Có
3/15/2026 16:22:46	Sinh viên CNTT	Trên 2 năm	Python	Tài liệu chính thức (Documentation)	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Trả lời câu hỏi khi học	Có thể		Có
3/15/2026 18:11:49	Sinh viên CNTT	Trên 2 năm	JavaScript	Hỏi đáp AI (ChatGPT, Gemini, ...)	Làm bài tập thực hành	Không hiểu khái niệm lập trình	Sử dụng AI (ChatGPT, Gemini, ...)	Thỉnh thoảng	Phân tích lỗi code	Có		Có
3/15/2026 19:38:46	Sinh viên CNTT	Trên 2 năm	JavaScript	Youtube	Viết project nhỏ	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo	Có thể	Tư vấn lộ trình, lên kế hoạch chi tiết và bài học cho từng giai đoạn để người sử dụng nắm bắt được quá trình học tập và mục tiêu cụ thể	Có
3/15/2026 19:50:36	Sinh viên CNTT	Trên 2 năm	C++	Tài liệu chính thức (Documentation)	Viết project nhỏ	Không hiểu khái niệm lập trình	Tìm lỗi trên Google	Thường xuyên	Giải thích khái niệm lập trình	Có	Tuỳ	Có
3/15/2026 23:50:13	Sinh viên CNTT	6 tháng - 1 năm	C++	Khóa học ở trường	Đọc thêm tài liệu	Không biết sửa lỗi code	Hỏi bạn bè / giảng viên	Thỉnh thoảng	Giải thích khái niệm lập trình	Có		Có
3/15/2026 23:57:25	Sinh viên CNTT	6 tháng - 1 năm	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết sửa lỗi code	Hỏi bạn bè / giảng viên	Thường xuyên	Phân tích lỗi code, Gợi ý bài học tiếp theo	Có thể	Sửa lỗi và đưa ra lịch trình	Có
3/16/2026 0:00:11	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Hỏi bạn bè / giảng viên	Thường xuyên	Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo	Có	giải thích kĩ về cách giải 1 bài tập nào đó	Có

3/16/2026 0:42:33	Sinh viên CNTT	Dưới 6 tháng	C	Youtube	Đọc thêm tài liệu	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo, Trả lời câu hỏi khi học	Có thể	Tôi thấy nếu như là 1 giá sư thì phải có chức năng giúp sinh viên cải thiện bài tập và thêm bài tập cho sinh viên để sinh viên hiểu hơn về lập trình . Giải thích cần kể chi tiết cho Sinh viên	Có
3/16/2026 2:47:43	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình	Có		Có
3/16/2026 5:22:20	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết sửa lỗi code	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Phân tích lỗi code	Có	Nhiều trang	Có
3/16/2026 5:37:17	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Viết project nhỏ	Không biết bắt đầu từ đâu	Tìm lỗi trên Google	Thỉnh thoảng	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi	Có thể		Không
3/16/2026 7:42:47	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thỉnh thoảng	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo	Có thể		Không
3/16/2026 7:43:14	Sinh viên CNTT	6 tháng - 1 năm	C++	Hỏi đáp AI (ChatGPT, Gemini, ...)	Làm bài tập thực hành	Không biết sửa lỗi code	Sử dụng AI (ChatGPT, Gemini, ...)	Thỉnh thoảng	Phân tích lỗi code, Gợi ý cách sửa lỗi	Có		Có
3/16/2026 9:04:58	Sinh viên CNTT	6 tháng - 1 năm	C++	Hỏi đáp AI (ChatGPT, Gemini, ...)	Làm bài tập thực hành	Không biết sửa lỗi code	Hỏi bạn bè / giảng viên	Thỉnh thoảng	Phân tích lỗi code	Có		Không
3/16/2026 10:59:53	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thỉnh thoảng	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo, Trả lời câu hỏi khi học	Có		Không
3/16/2026 11:14:38	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Đọc thêm tài liệu	Không biết sửa lỗi code	Hỏi bạn bè / giảng viên	Thỉnh thoảng	Phân tích lỗi code	Có thể		Có
3/17/2026 8:43:17	Sinh viên CNTT	Dưới 6 tháng	C++	Youtube	Làm bài tập thực hành	Không biết sửa lỗi code	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Phân tích lỗi code	Có thể	có	Có

3/17/2026 9:29:15	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Tìm lỗi trên Google	Thỉnh thoảng	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo	Có thể		Có
3/17/2026 11:05:16	Sinh viên CNTT	Dưới 6 tháng	C++	Youtube	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo	Có		Có
3/17/2026 11:18:16	Sinh viên CNTT	Dưới 6 tháng	C++	Tài liệu chính thức (Documentation)	Đọc thêm tài liệu	Không biết sửa lỗi code	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code, Gợi ý cách sửa lỗi, Gợi ý bài học tiếp theo, Trả lời câu hỏi khi học	Có	Giải đáp nhanh đúng và chính xác	Có
3/17/2026 12:56:30	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết sửa lỗi code	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Phân tích lỗi code	Có thể		Không
3/17/2026 15:50:16	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết bắt đầu từ đâu	Sử dụng AI (ChatGPT, Gemini, ...)	Hiếm khi	Giải thích khái niệm lập trình	Có		Có
3/17/2026 16:15:10	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Làm bài tập thực hành	Không biết sửa lỗi code	Sử dụng AI (ChatGPT, Gemini, ...)	Thường xuyên	Giải thích khái niệm lập trình, Phân tích lỗi code	Có thể		Có
3/17/2026 16:57:56	Sinh viên CNTT	Dưới 6 tháng	C++	Youtube	Làm bài tập thực hành	Không có người hướng dẫn	Sử dụng AI (ChatGPT, Gemini, ...)	Thỉnh thoảng	Giải thích khái niệm lập trình	Có	Kĩ năng giải thích	Có
3/18/2026 10:08:21	Sinh viên CNTT	Dưới 6 tháng	C++	Hỏi đáp AI (ChatGPT, Gemini, ...)	Làm bài tập thực hành	Không biết sửa lỗi code	Tìm trên Stack-Overflow	Thường xuyên	Gợi ý cách sửa lỗi	Có		Có
3/18/2026 10:33:33	Sinh viên CNTT	Dưới 6 tháng	C++	Khóa học ở trường	Làm bài tập thực hành	Không hiểu khái niệm lập trình	Hỏi bạn bè / giảng viên	Hiếm khi	Giải thích khái niệm lập trình	Không		Có

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày ... tháng ... năm 2026

Giáo viên hướng dẫn